

本节内容

查找

基本概念

王道考研/CSKAOYAN.COM

知识总览

查找的基本概念

基本概念

查找算法的效率评价

王道考研/CSKAOYAN.COM

基本概念

查找 —— 在数据集中寻找满足某种条件的数据元素的过程称为查找

查找表（查找结构） —— 用于查找的数据集合称为查找表，它由同一类型的数据元素（或记录）组成

关键字 —— 数据元素中唯一标识该元素的某个数据项的值，使用基于关键字的查找，查找结果应该是唯一的。

学号	姓名	语文	数学	英语
202001	铁柱	99	97	98
202002	狗剩	86	23	90
202003	旺财	43	66	76
202004	愣子	12	16	11
202005	铁柱	9	17	25

Eg:

查找表——学生成绩信息（线性结构、可顺序可链式存储）

数据元素(记录)——每个学生的信息

关键字——学号

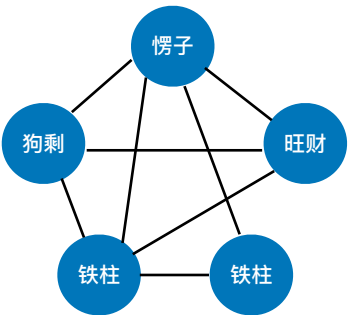
王道考研/CSKAOYAN.COM

基本概念

查找 —— 在数据集中寻找满足某种条件的数据元素的过程称为查找

查找表（查找结构） —— 用于查找的数据集合称为查找表，它由同一类型的数据元素（或记录）组成

关键字 —— 数据元素中唯一标识该元素的某个数据项的值，使用基于关键字的查找，查找结果应该是唯一的。



Eg:

查找表——微信用户数据集（图结构）

数据元素(记录)——每个用户用户的信息

关键字——微信号

王道考研/CSKAOYAN.COM

对查找表的常见操作

- ①查找符合条件的数据元素
- ②插入、删除某个数据元素

学号	姓名	语文	数学	英语
202001	铁柱	99	97	98
202002	狗剩	86	23	90
202003	旺财	43	66	76
202004	愣子	12	16	11
202005	铁柱	9	17	25

只需进行操作① —— 静态查找表

仅关注查找速度即可

订单号	菜品	下单时间	已完成
1	黄焖Jimmy饭	30min	是
2	鱼香Rose	20min	否
3	皮蛋solo粥	15min	是
4	梅菜Crow	10min	否
5	过桥Michelle	2min	否

也要进行操作② —— 动态查找表

除了查找速度，也要关注
插/删操作是否方便实现

王道考研/CSKAOYAN.COM

查找算法的评价指标

查找长度——在查找运算中，需要对比关键字的次数称为查找长度
平均查找长度（ASL, Average Search Length）—— 所有查找过程中进行关键字的比较次数的平均值



数据元素个数 n

查找第 i 个元素的查找长度 C_i

查找第 i 个元素的概率 P_i

$$ASL = \sum_{i=1}^n P_i C_i$$

通常认为查找任何一个元素的概率都相同

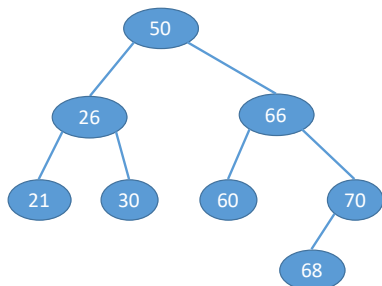
王道考研/CSKAOYAN.COM

查找算法的评价指标

查找长度——在查找运算中，需要对比关键字的次数称为查找长度

平均查找长度（ASL, Average Search Length）—— 所有查找过程中进行关键字的比较次数的平均值

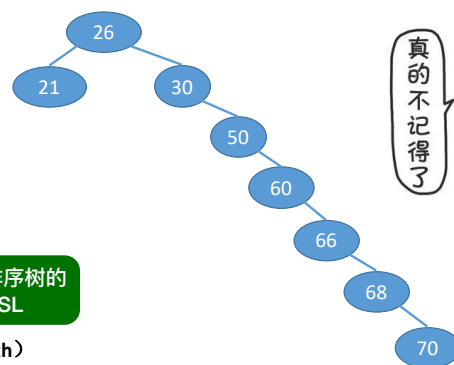
ASL 的数量级反应了
查找算法时间复杂度



二叉排序树的
ASL

查找成功的平均查找长度 ASL (Average Search Length)

$$ASL = (1*1 + 2*2 + 3*4 + 4*1)/8 = 2.625$$



$$ASL = (1*1 + 2*2 + 3*1 + 4*1 + 5*1 + 6*1 + 7*1)/8 = 3.75$$

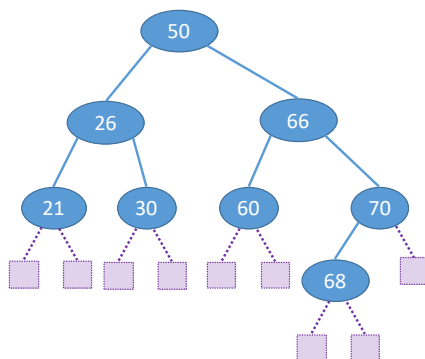
王道考研/CSKAOYAN.COM

查找算法的评价指标

查找长度——在查找运算中，需要对比关键字的次数称为查找长度

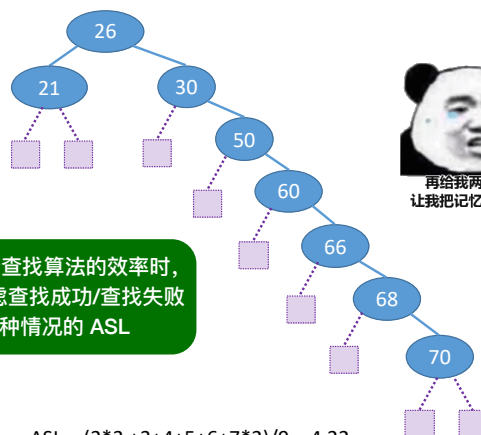
平均查找长度（ASL, Average Search Length）—— 所有查找过程中进行关键字的比较次数的平均值

ASL 的数量级反应了
查找算法时间复杂度



查找失败的平均查找长度 ASL (Average Search Length)

$$ASL = (3*7 + 4*2)/9 = 3.22$$



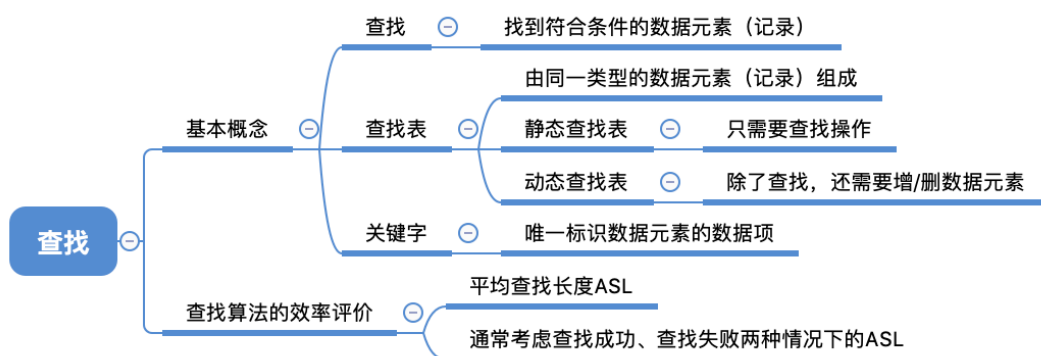
评价一个查找算法的效率时，
通常考虑查找成功/查找失败
两种情况的 ASL

$$ASL = (2*3 + 3+4+5+6+7*2)/9 = 4.22$$



王道考研/CSKAOYAN.COM

知识回顾与重要考点

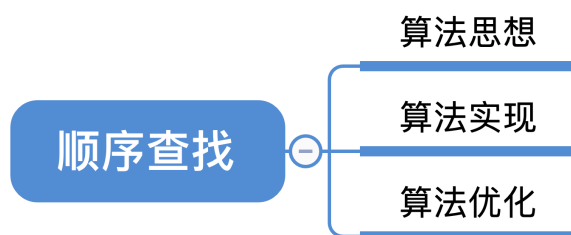


本节内容

顺序查找

王道考研/CSKAOYAN.COM

知识总览



王道考研/CSKAOYAN.COM

顺序查找的算法思想

顺序查找，又叫“线性查找”，通常用于线性表。

算法思想：从头到 jio 挨个找（或者反过来也OK）

33	10	13	29	16	19	32	7	43	41	37
----	----	----	----	----	----	----	---	----	----	----



查找目标：43

顺序查找的算法思想

顺序查找，又叫“线性查找”，通常用于线性表。

算法思想：从头到 jio 挨个找（或者反过来也OK）

33	10	13	29	16	19	32	7	43	41	37
----	----	----	----	----	----	----	---	----	----	----

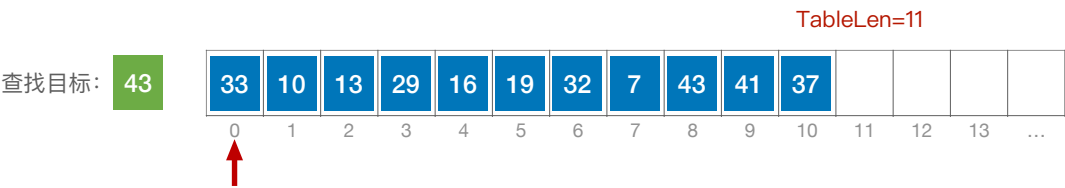


查找目标：43

顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

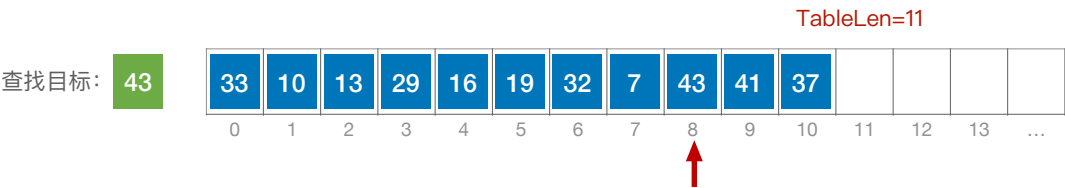
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```



顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

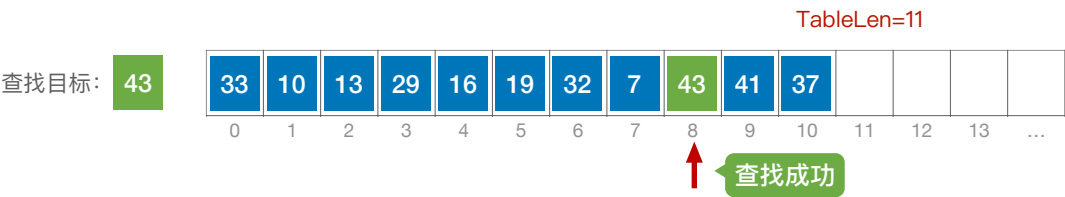
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```



顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

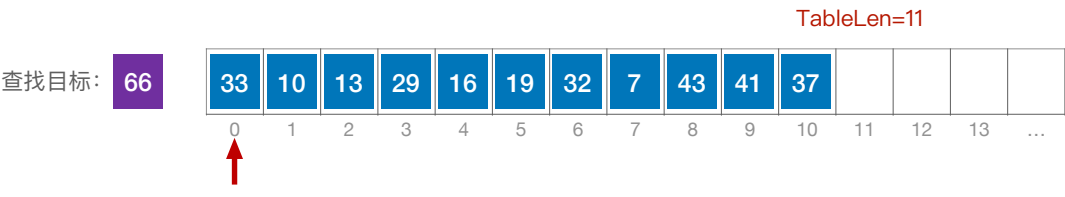
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```



顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

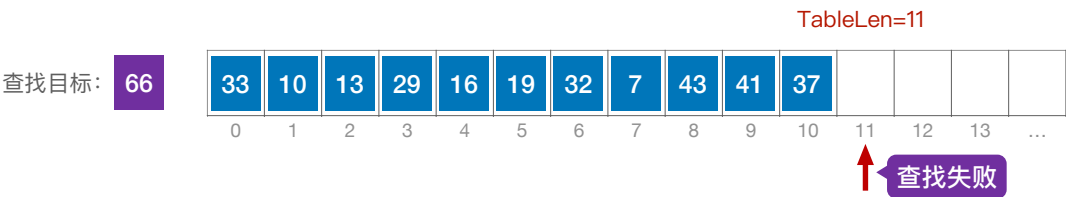
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```



顺序查找的实现

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

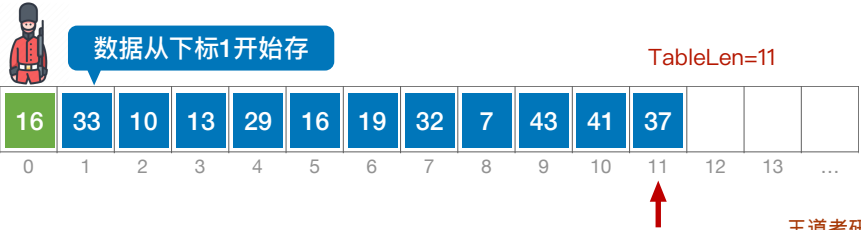
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    int i;
    for(i=0;i<ST.TableLen && ST.elem[i]!=key; ++i);
    //查找成功，则返回元素下标；查找失败，则返回-1
    return i==ST.TableLen? -1 : i;
}
```



顺序查找的实现（哨兵）

```
typedef struct{           //查找表的数据结构（顺序表）
    ElemType *elem;       //动态数组基址
    int TableLen;         //表的长度
}SSTable;

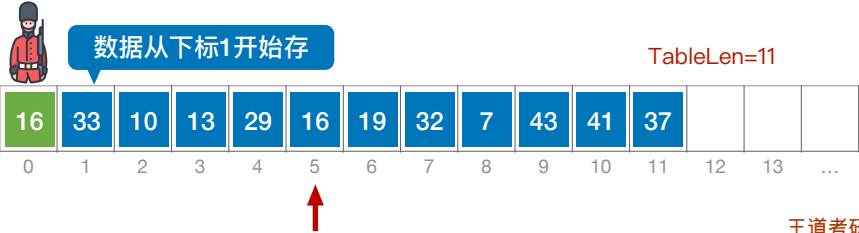
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;       //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;             //查找成功，则返回元素下标；查找失败，则返回0
}
```



顺序查找的实现（哨兵）

```
typedef struct{
    ElemType *elem; //查找表的数据结构（顺序表）
    int TableLen; //动态数组基址
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key; //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i; //查找成功，则返回元素下标；查找失败，则返回0
}
```



顺序查找的实现（哨兵）

```
typedef struct{
    ElemType *elem; //查找表的数据结构（顺序表）
    int TableLen; //动态数组基址
}SSTable;

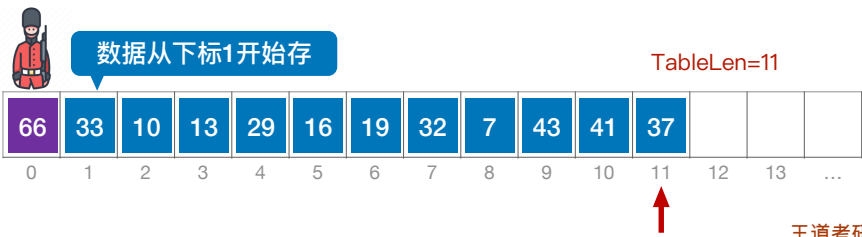
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key; //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i; //查找成功，则返回元素下标；查找失败，则返回0
}
```



顺序查找的实现（哨兵）

```
typedef struct{
    ElemType *elem;    //查找表的数据结构（顺序表）
    int TableLen;      //动态数组基址
}SSTable;

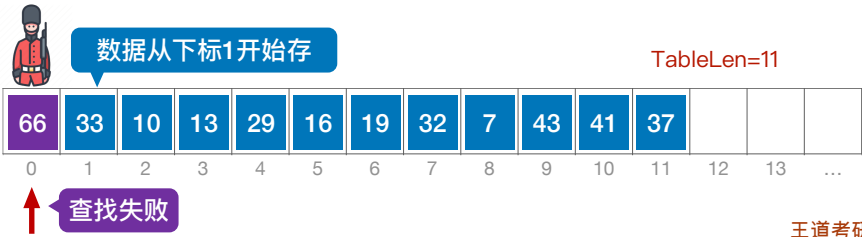
//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;    //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;          //查找成功，则返回元素下标；查找失败，则返回0
}
```



顺序查找的实现（哨兵）

```
typedef struct{
    ElemType *elem;    //查找表的数据结构（顺序表）
    int TableLen;      //动态数组基址
}SSTable;

//顺序查找
int Search_Seq(SSTable ST,ElemType key){
    ST.elem[0]=key;    //“哨兵”
    int i;
    for(i=ST.TableLen;ST.elem[i]!=key;--i); //从后往前找
    return i;          //查找成功，则返回元素下标；查找失败，则返回0
}
```





查找效率分析

TableLen=11

16	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

↑ 查找成功

$$ASL = \sum_{i=1}^n P_i C_i$$
$$ASL_{成功} = \frac{1 + 2 + 3 + \dots + n}{n} = \frac{n + 1}{2}$$
$$ASL_{失败} = n + 1$$

TableLen=11

66	33	10	13	29	16	19	32	7	43	41	37			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

↑ 查找失败

王道考研/CSKAOYAN.COM

顺序查找的优化（对有序表）

查找表中元素有序存放（递增/递减）

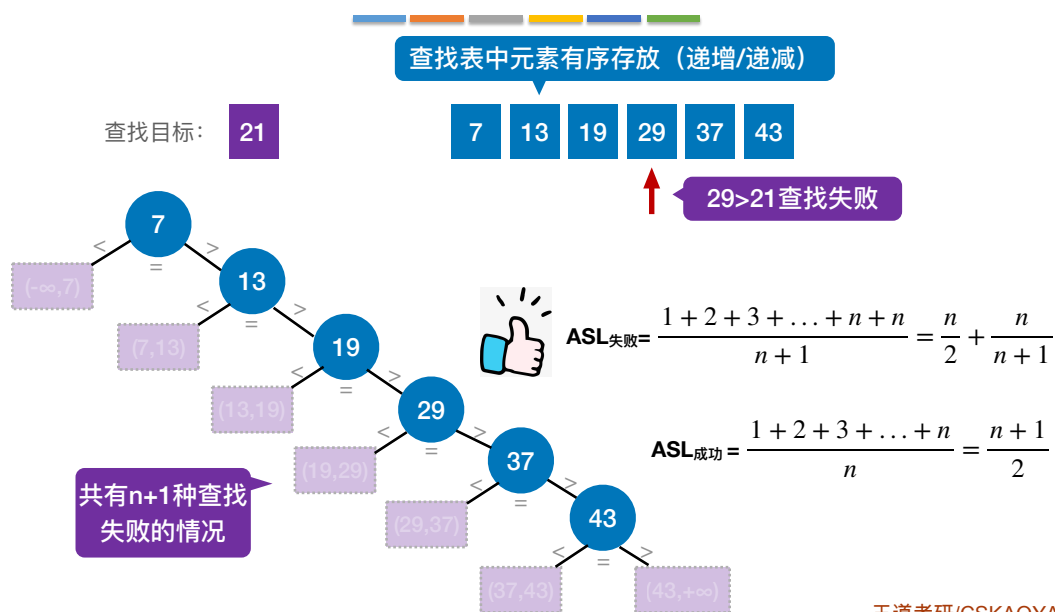
查找目标： 21

7	13	19	29	37	43
---	----	----	----	----	----

↑

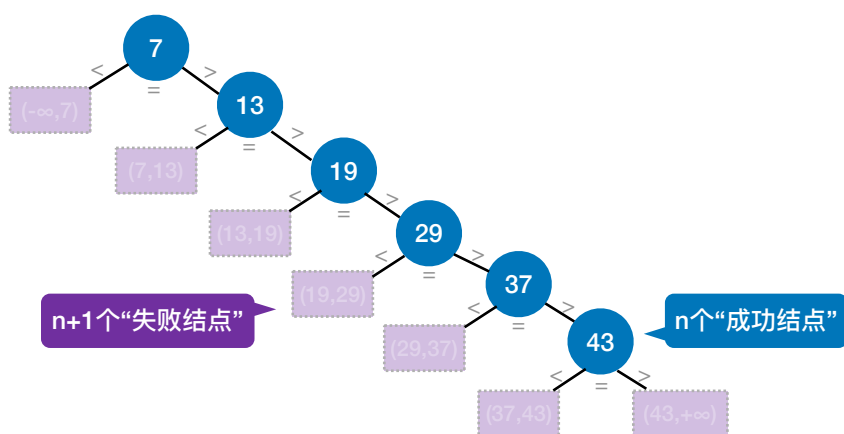
王道考研/CSKAOYAN.COM

顺序查找的优化（对有序表）



王道考研/CSKAOYAN.COM

用查找判定树分析ASL



一个成功结点的查找长度 = 自身所在层数
 一个失败结点的查找长度 = 其父节点所在层数
 默认情况下，各种失败情况或成功情况都等概率发生

王道考研/CSKAOYAN.COM

顺序查找的优化（被查概率不相等）

被查概率
7: 15%
13: 5%
19: 10%
29: 40%
37: 28%
43: 2%

7 13 19 29 37 43



$ASL_{成功} = 1 \times 0.15 + 2 \times 0.05 + 3 \times 0.1 + 4 \times 0.4 + 5 \times 0.28 + 6 \times 0.02 = 3.67$

被查概率大的放在靠前位置

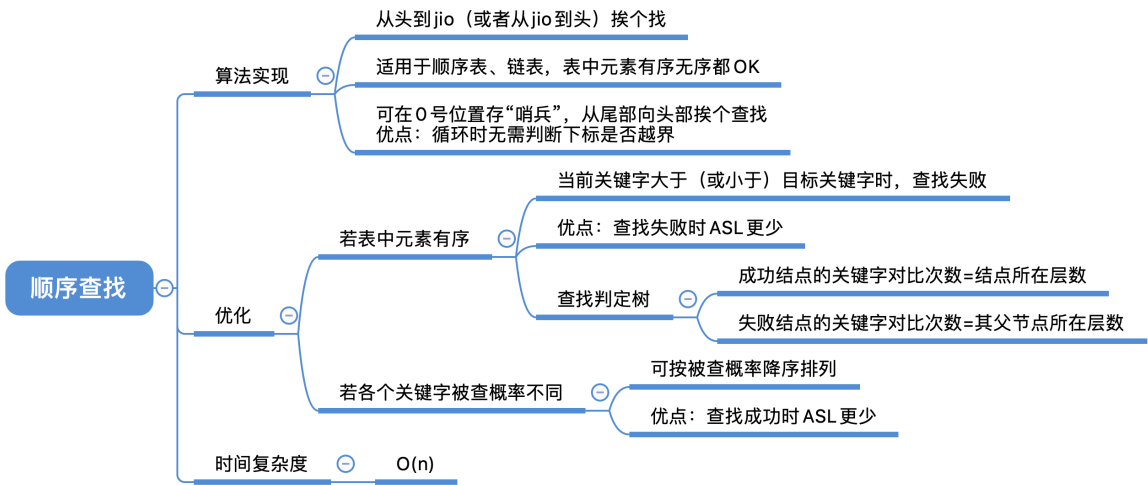
29 37 7 19 13 43



$ASL_{成功} = 1 \times 0.4 + 2 \times 0.28 + 3 \times 0.15 + 4 \times 0.1 + 5 \times 0.05 + 6 \times 0.02 = 2.18$

$ASL = \sum_{i=1}^n P_i C_i$

知识回顾与重要考点



本节内容

折半查找

王道考研/CSKAOYAN.COM

知识总览

折半查找

算法思想

算法实现

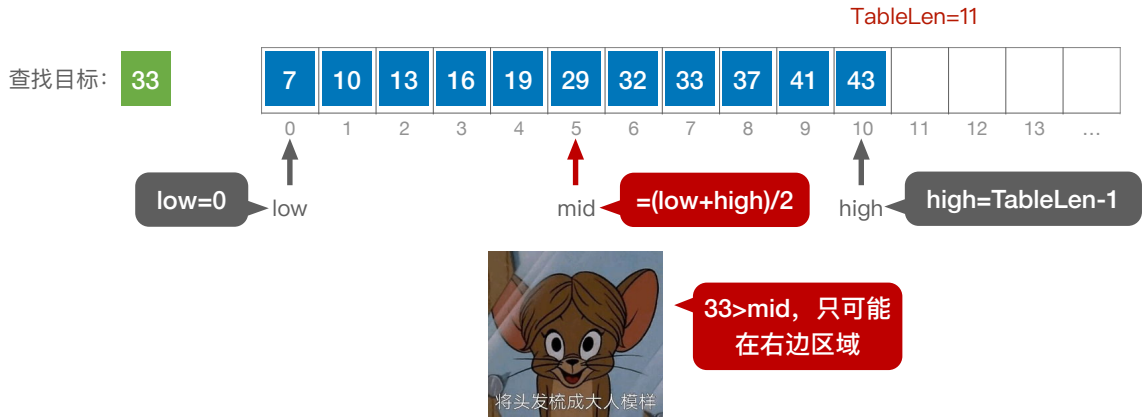
查找判定树

折半查找效率

王道考研/CSKAOYAN.COM

折半查找的算法思想

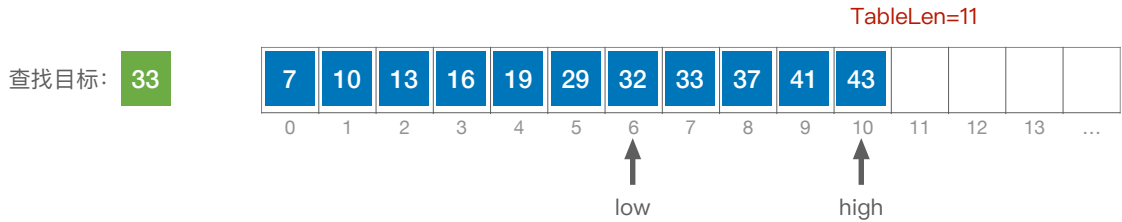
折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。

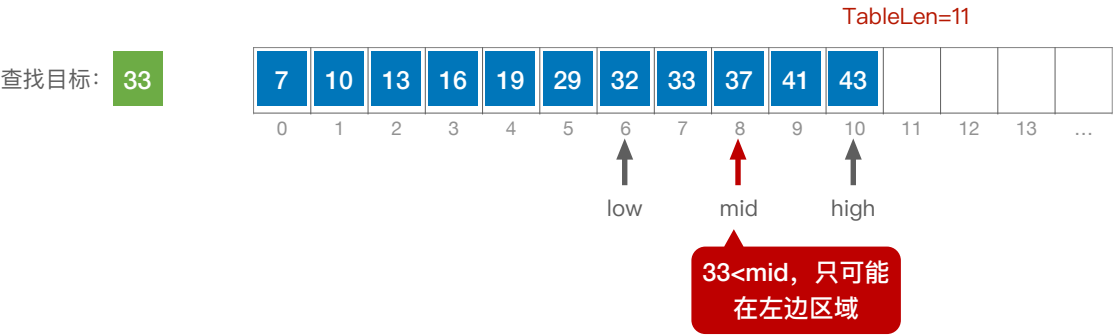


注：只有在 [low, high] 之间才有可能找到目标关键字

王道考研/CSKAOYAN.COM

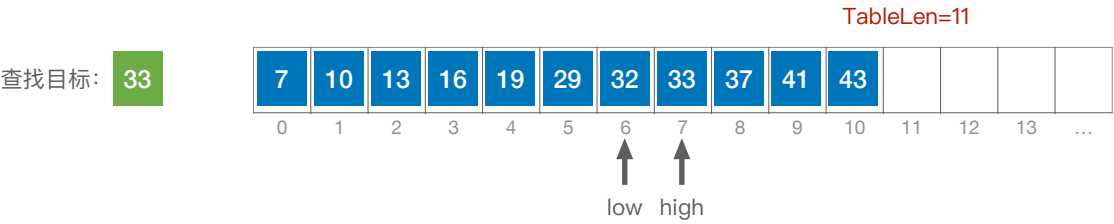
折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



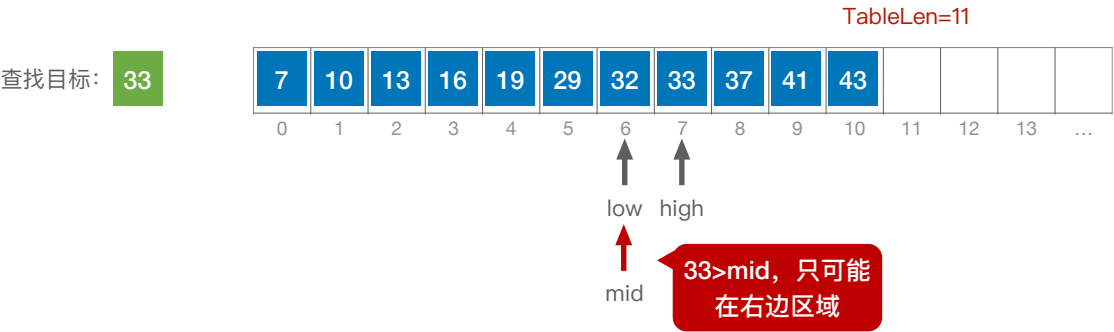
折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



折半查找的算法思想

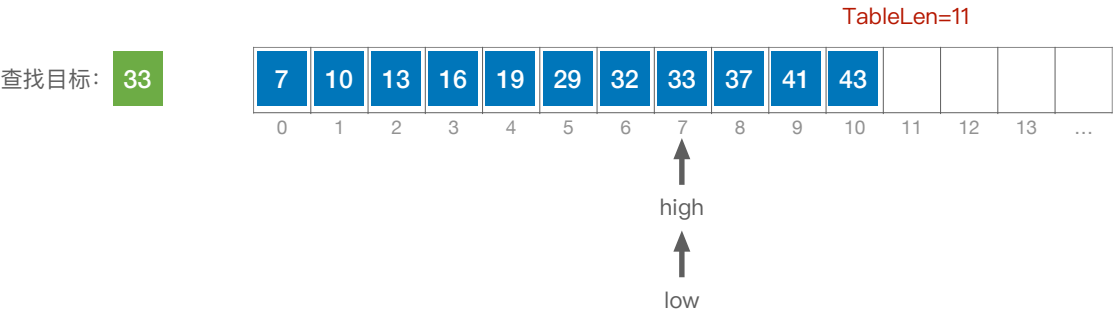
折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

折半查找的算法思想

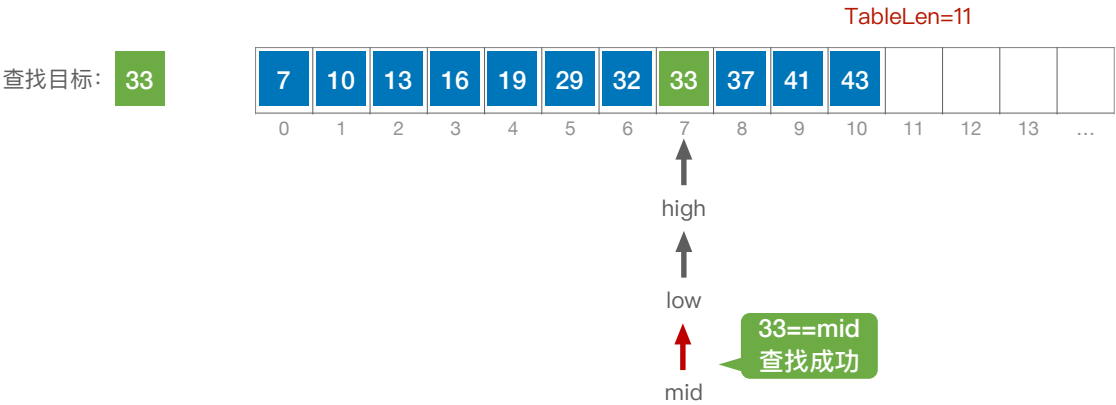
折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

折半查找的算法思想

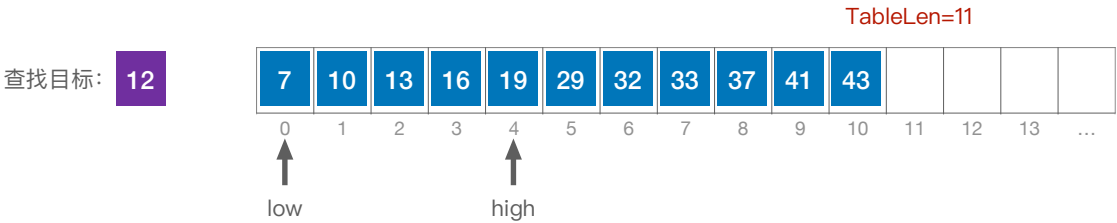
折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

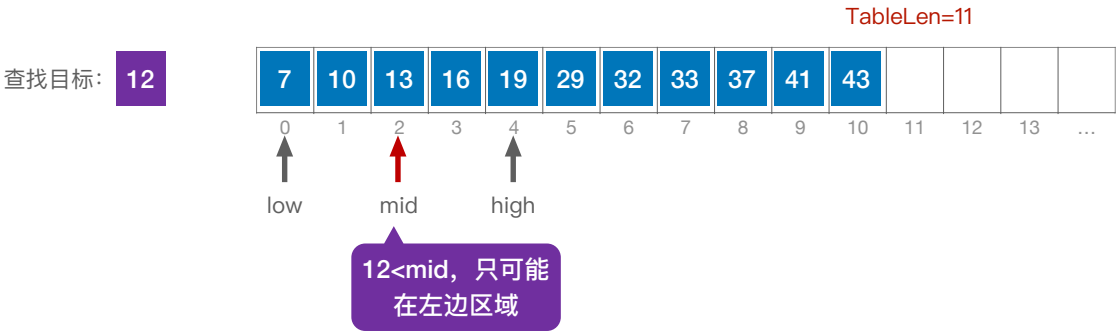
折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



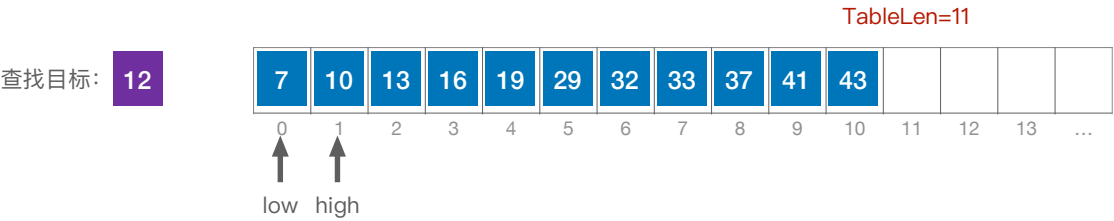
折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。

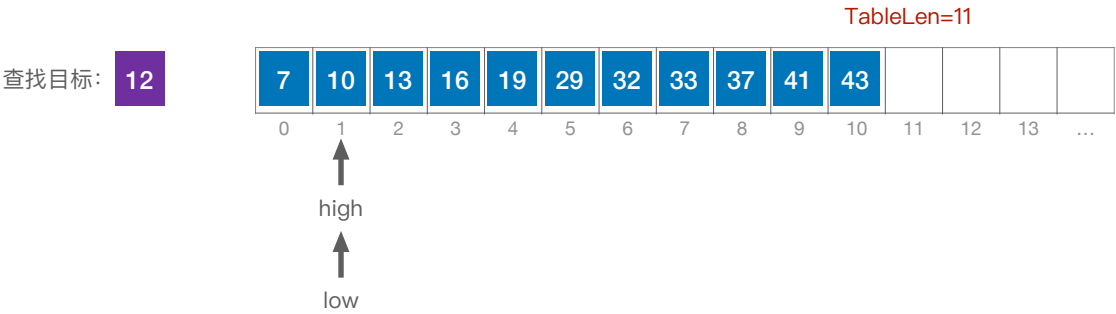


王道考研/CSKAOYAN.COM

折半查找的算法思想



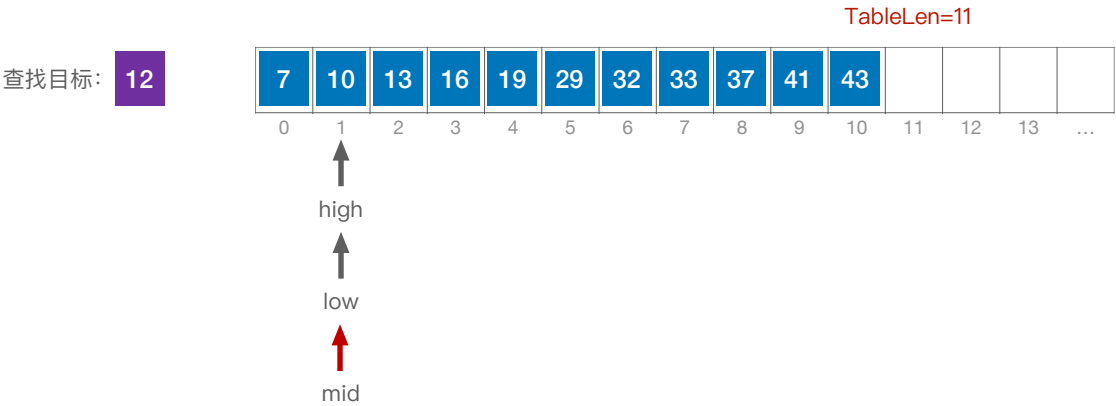
折半查找，又称“二分查找”，仅适用于有序的顺序表。



折半查找的算法思想



折半查找，又称“二分查找”，仅适用于有序的顺序表。



折半查找的算法思想

折半查找，又称“二分查找”，仅适用于有序的顺序表。



王道考研/CSKAOYAN.COM

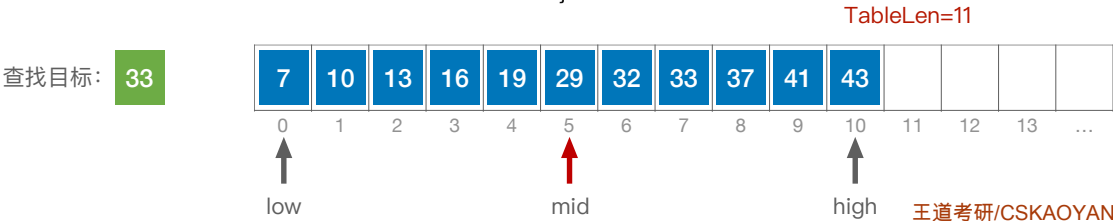
折半查找的实现

```
typedef struct{
    ElemType *elem; //查找表的数据结构（顺序表）
    int TableLen; //动态数组基址
}SSTable; //表的长度
```

折半查找，又称“二分查找”，
仅适用于有序的顺序表。

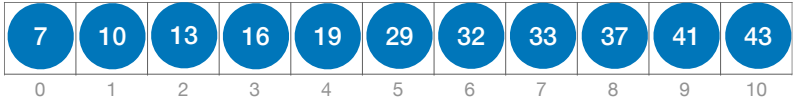
顺序表拥有随机访问
的特性，链表没有

```
//折半查找
int Binary_Search(SSTable L, ElemType key){
    int low=0, high=L.TableLen-1, mid;
    while(low<=high){
        mid=(low+high)/2; //取中间位置
        if(L.elem[mid]==key) //查找成功则返回所在位置
            return mid;
        else if(L.elem[mid]>key) //从前半部分继续查找
            high=mid-1;
        else //从后半部分继续查找
            low=mid+1;
    }
    return -1; //查找失败，返回-1
}
```

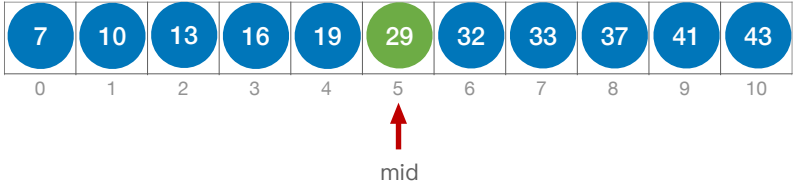


王道考研/CSKAOYAN.COM

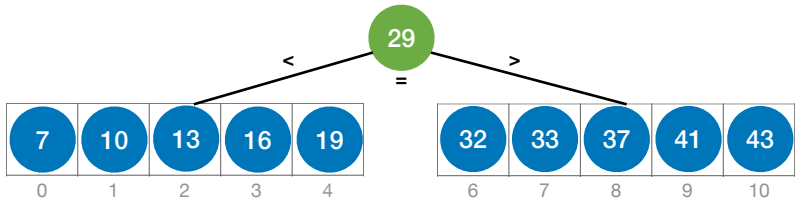
查找效率分析



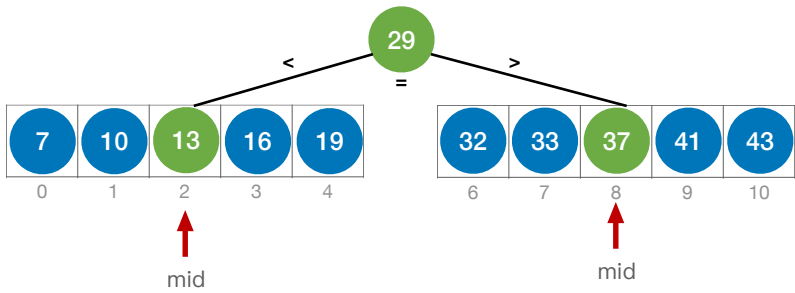
查找效率分析



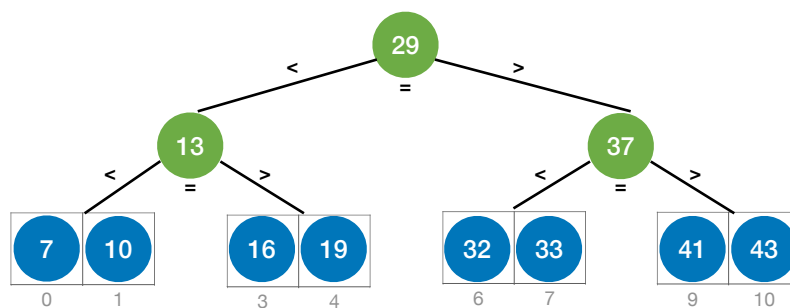
查找效率分析



查找效率分析

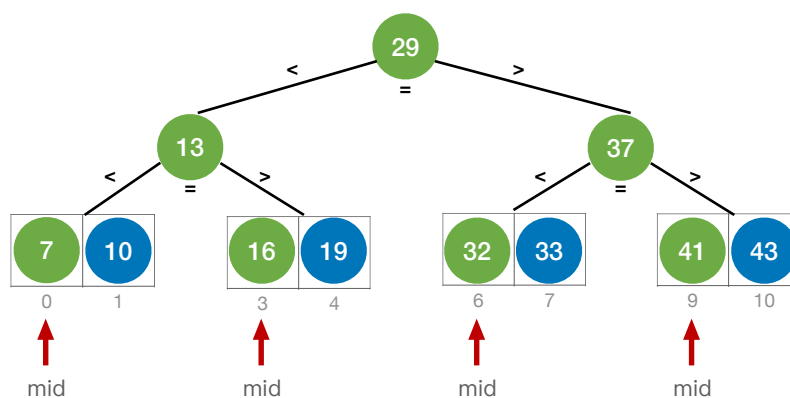


查找效率分析



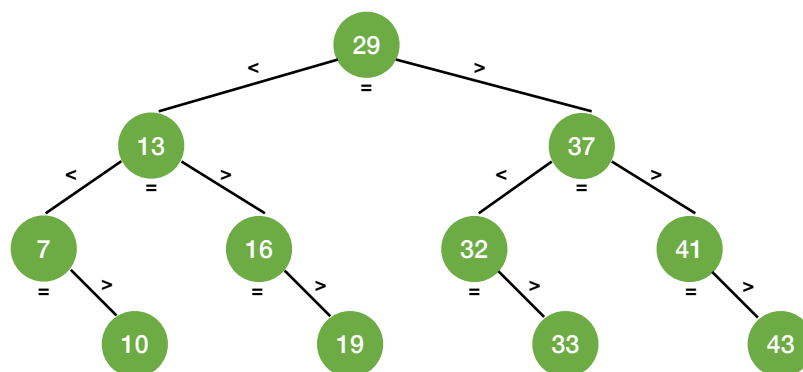
王道考研/CSKAOYAN.COM

查找效率分析



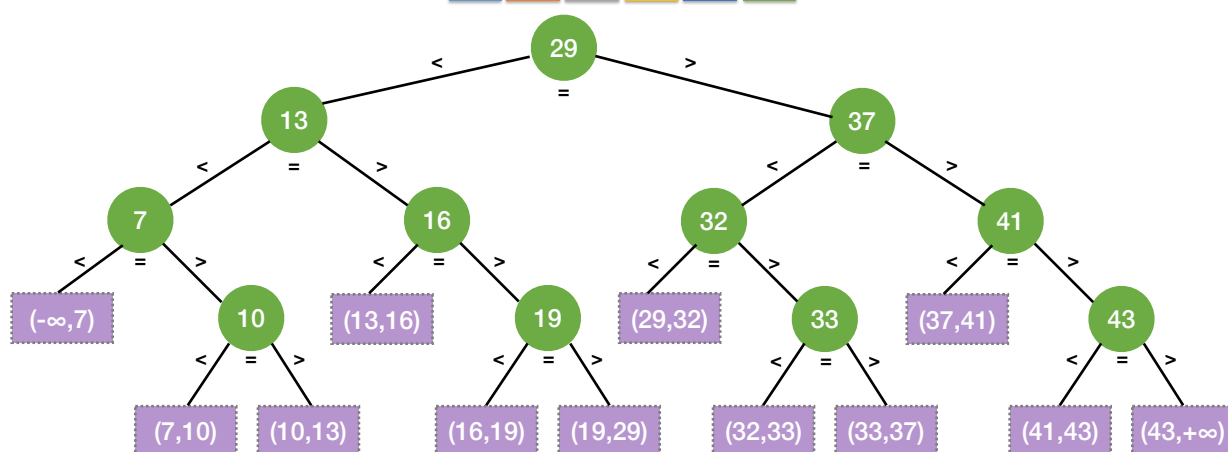
王道考研/CSKAOYAN.COM

查找效率分析



王道考研/CSKAOYAN.COM

查找效率分析

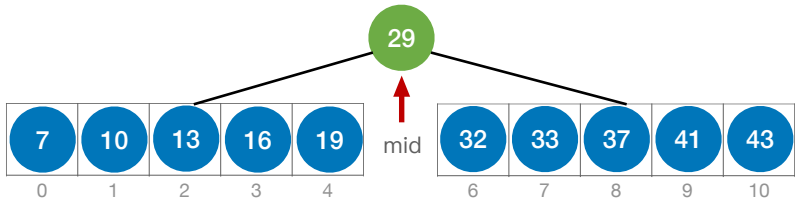


$$ASL_{成功} = (1*1 + 2*2 + 3*4 + 4*4) / 11 = 3$$

$$ASL_{失败} = (3*4 + 4*8) / 12 = 11/3$$

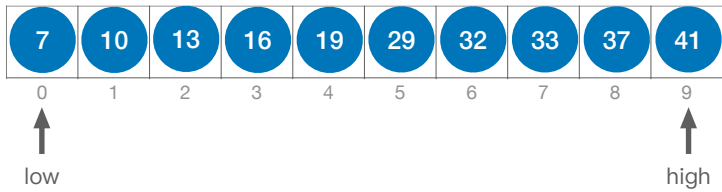
王道考研/CSKAOYAN.COM

折半查找判定树的构造

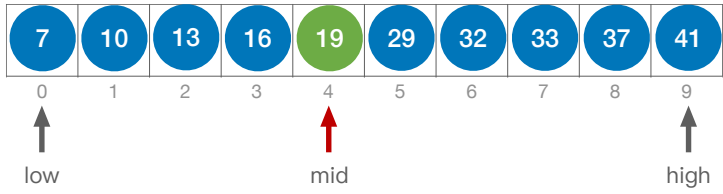


如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

折半查找判定树的构造



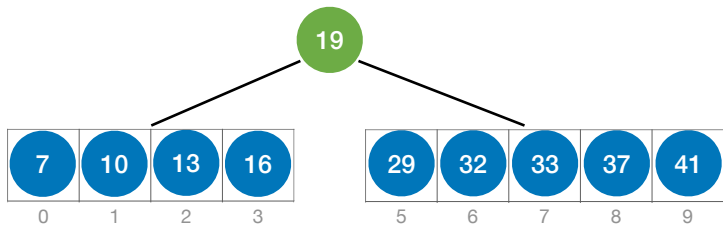
折半查找判定树的构造



$$mid = \lfloor (low + high) / 2 \rfloor$$

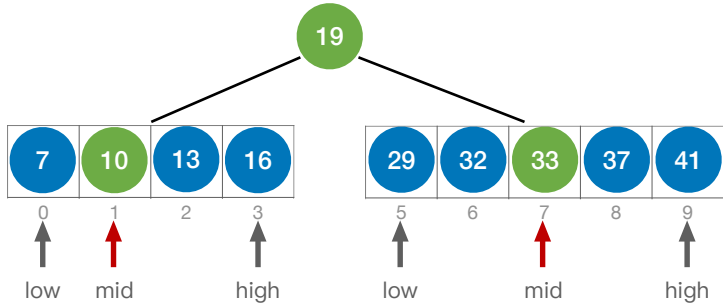
如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

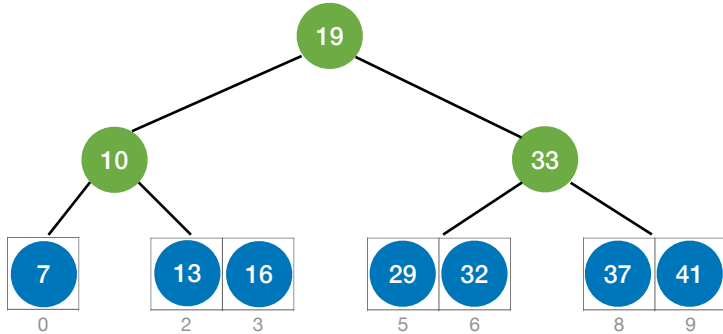
折半查找判定树的构造



$$mid = \lfloor (low + high) / 2 \rfloor$$

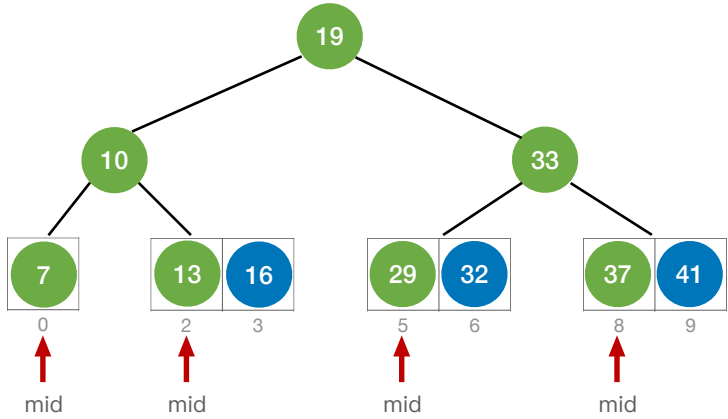
如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等
如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



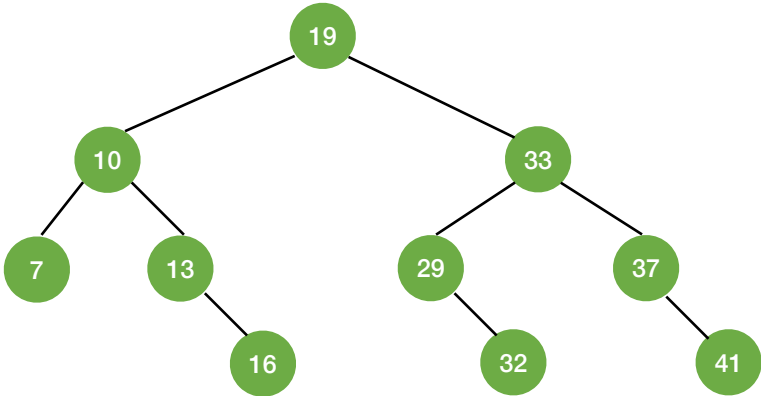
如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等
如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分少一个元素

折半查找判定树的构造



如果当前low和high之间有奇数个元素, 则 mid 分隔后, 左右两部分元素个数相等
如果当前low和high之间有偶数个元素, 则 mid 分隔后, 左半部分比右半部分少一个元素

折半查找判定树的构造



如果当前low和high之间有奇数个元素, 则 mid 分隔后, 左右两部分元素个数相等
如果当前low和high之间有偶数个元素, 则 mid 分隔后, 左半部分比右半部分少一个元素

折半查找判定树的构造

如果当前low和high之间有奇数个元素，则mid分隔后，左右两部分元素个数相等
如果当前low和high之间有偶数个元素，则mid分隔后，左半部分比右半部分少一个元素

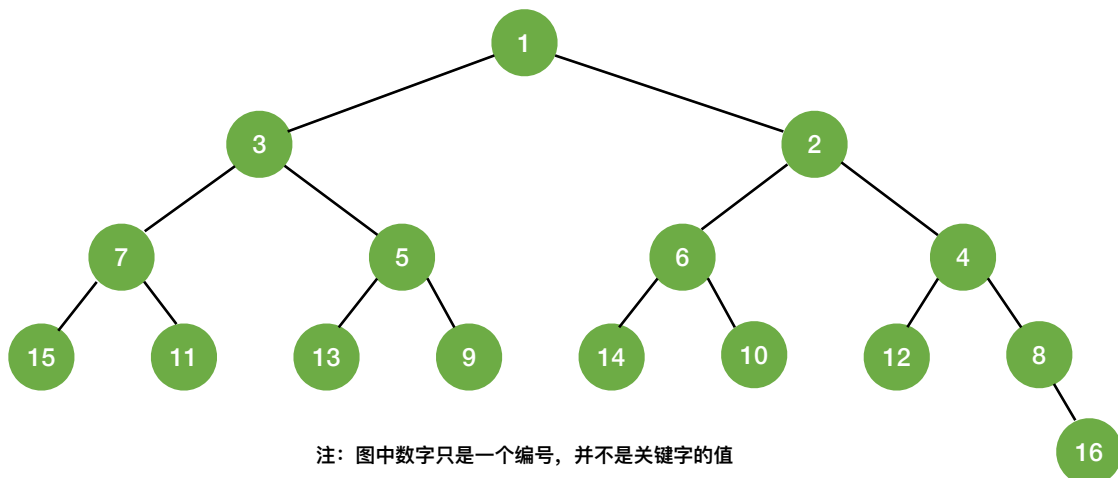


折半查找的判定树中，若 $mid = \lfloor (low + high)/2 \rfloor$ ，则对于任何一个结点，必有：
右子树结点数-左子树结点数=0或1

王道考研/CSKAOYAN.COM

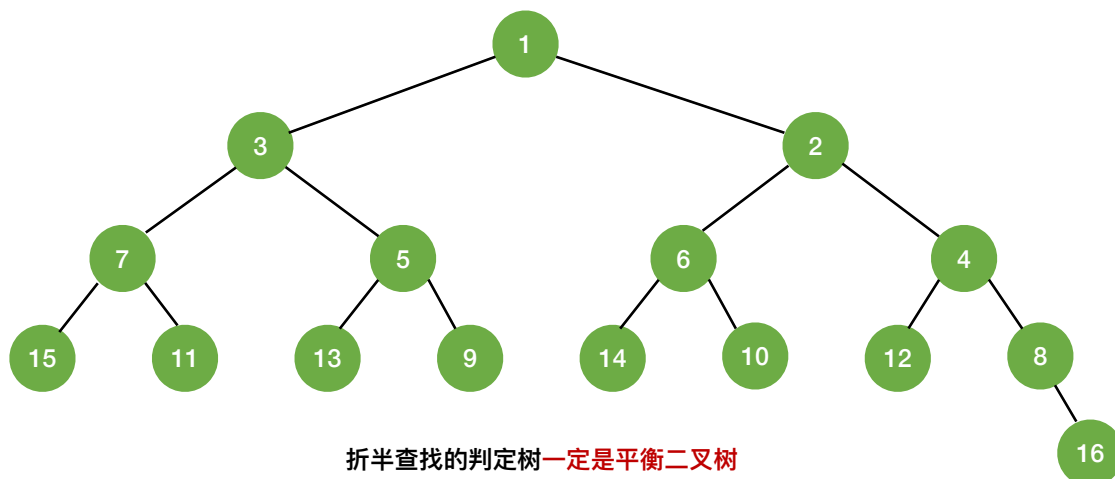
折半查找判定树的构造

练习：若 $mid = \lfloor (low + high)/2 \rfloor$ ，画出含1个元素、2个元素、3个元素....16个元素 的查找表对应的折半查找判定树，注：暂不考虑失败结点（Key：右子树结点数-左子树结点数=0或1）



王道考研/CSKAOYAN.COM

折半查找判定树的构造



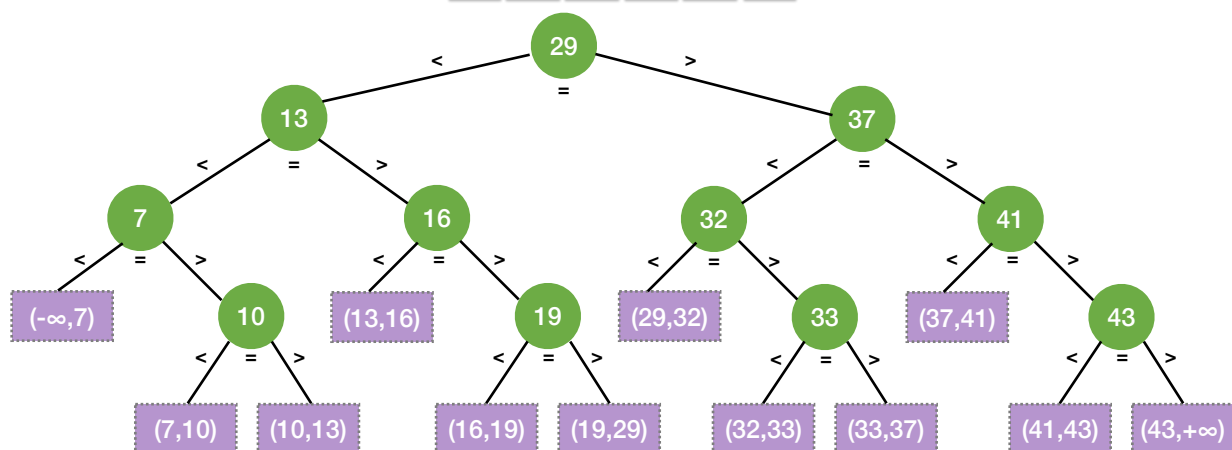
折半查找的判定树一定是平衡二叉树

折半查找的判定树中，只有最下面一层是不满的
因此，元素个数为 n 时树高 $h = \lceil \log_2(n+1) \rceil$

注：计算方法同
“完全二叉树”

王道考研/CSKAOYAN.COM

折半查找判定树的构造

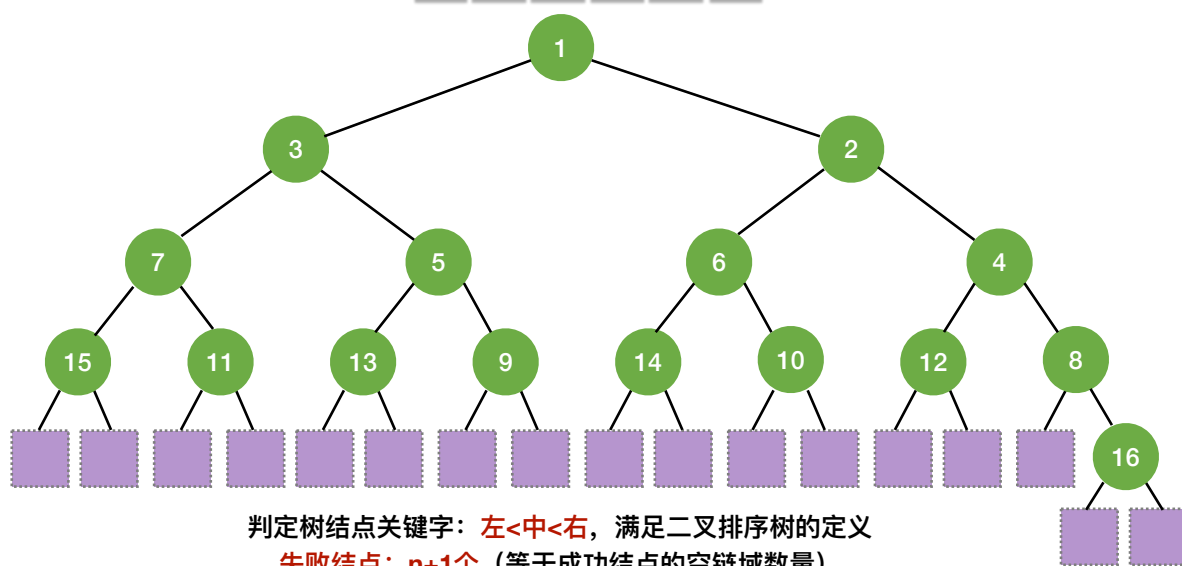


判定树结点关键字：左<中<右，满足二叉排序树的定义

失败结点： $n+1$ 个（等于成功结点的空链域数量）

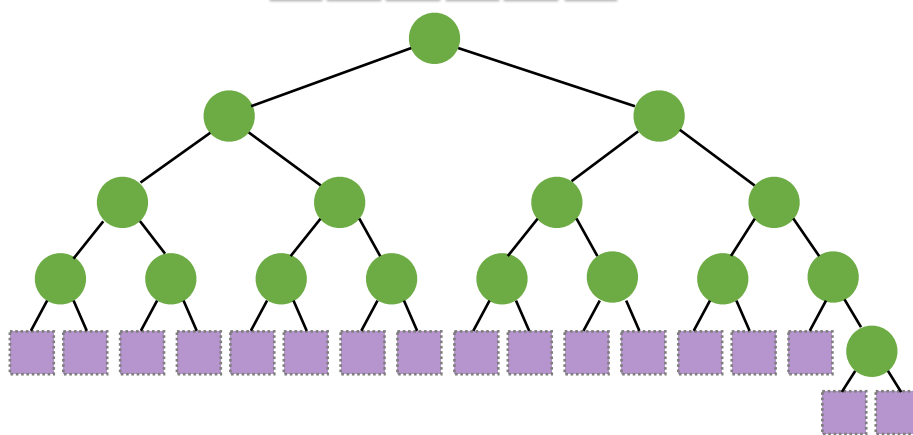
王道考研/CSKAOYAN.COM

折半查找判定树的构造



王道考研/CSKAOYAN.COM

折半查找的查找效率

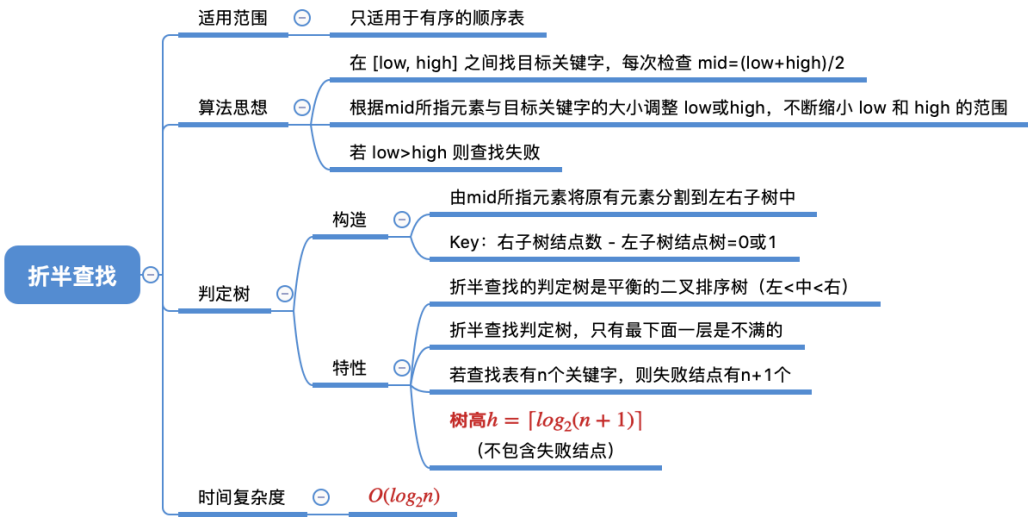


注: 该树高不包含失败结点

查找失败的ASL $\leq h$

王道考研/CSKAOYAN.COM

知识回顾与重要考点



王道考研/CSKAOYAN.COM

拓展思考



折半查找时间复杂度= $O(\log_2 n)$
顺序查找的时间复杂度= $O(n)$

辣么，折半查找的速度一定比顺序查找更快？

查找目标：

7

TableLen=11

7	10	13	16	19	29	32	33	37	41	43				
0	1	2	3	4	5	6	7	8	9	10	11	12	13	...

否认三连



我不是



我没有



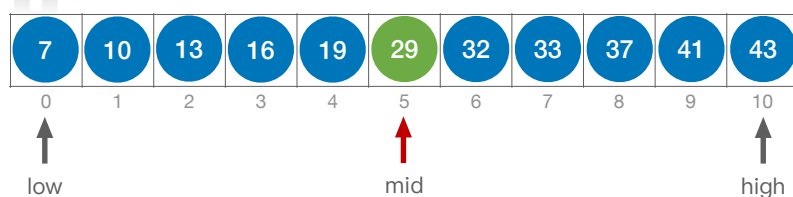
别瞎说啊

王道考研/CSKAOYAN.COM

拓展思考



如果 $mid = \lceil (low + high)/2 \rceil$
辣么，判定树是什么亚子？



$$mid = \lceil (low + high)/2 \rceil$$

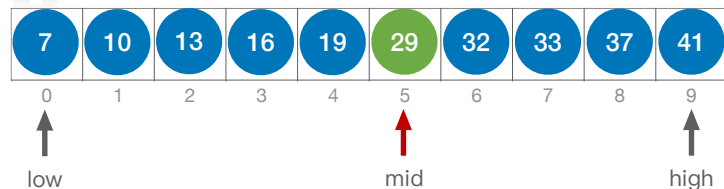
如果当前low和high之间有奇数个元素，则 mid 分隔后，左右两部分元素个数相等

王道考研/CSKAOYAN.COM

拓展思考



如果 $mid = \lceil (low + high)/2 \rceil$
辣么，判定树是什么亚子？



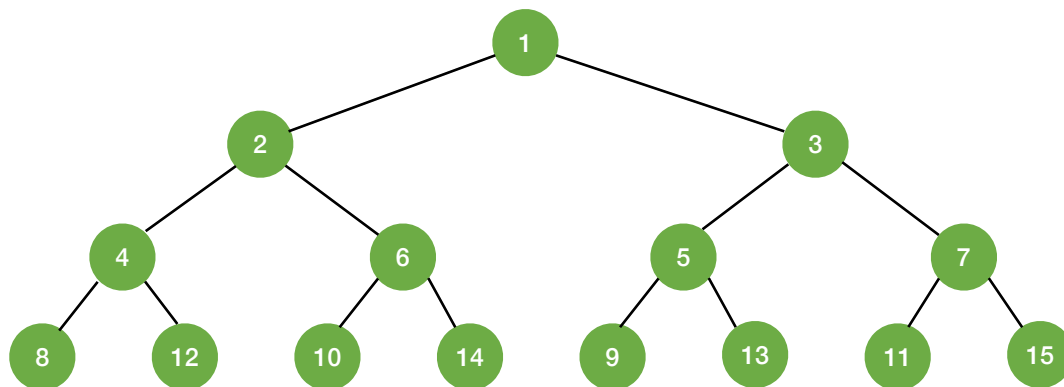
$$mid = \lceil (low + high)/2 \rceil$$

如果当前low和high之间有偶数个元素，则 mid 分隔后，左半部分比右半部分多一个元素

王道考研/CSKAOYAN.COM

拓展思考

折半查找的判定树中，若 $mid = \lceil (low + high)/2 \rceil$ ，则对于任何一个结点，必有：
左子树结点数-右子树结点数=0或1



注：图中数字只是一个编号，并不是关键字的值

本节内容

分块查找

王道考研/CSKAOYAN.COM

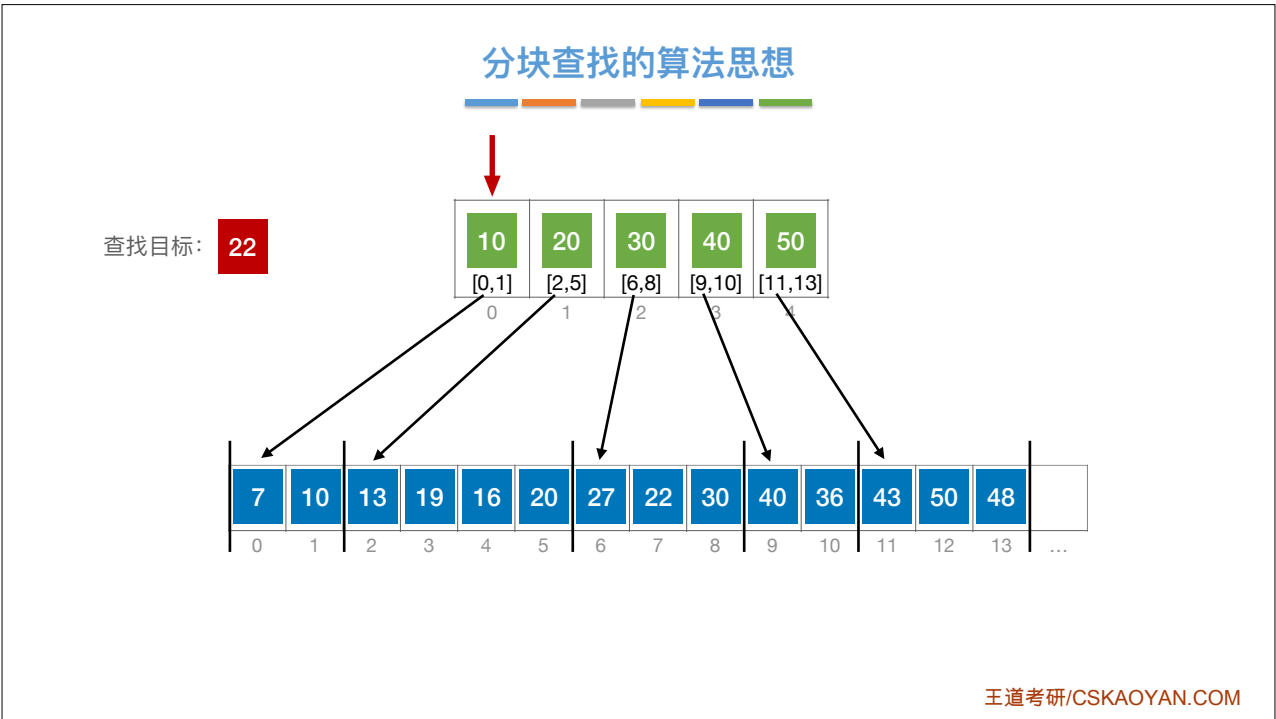
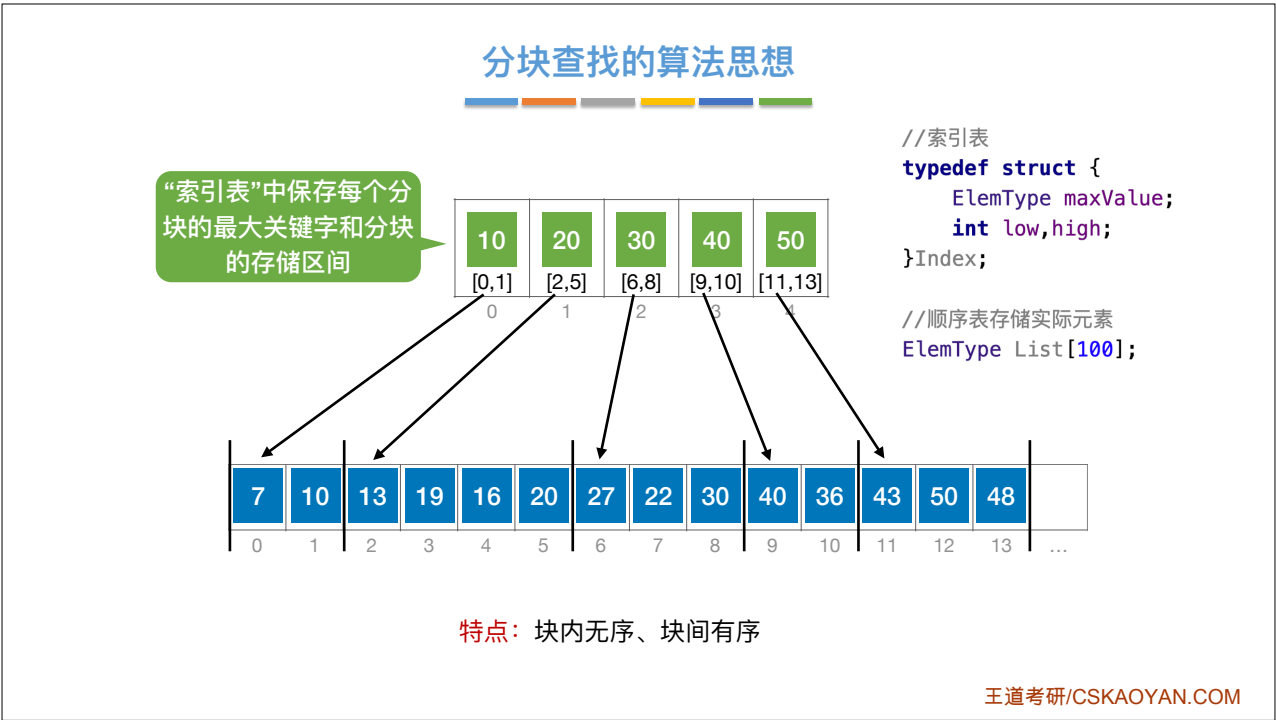
知识总览

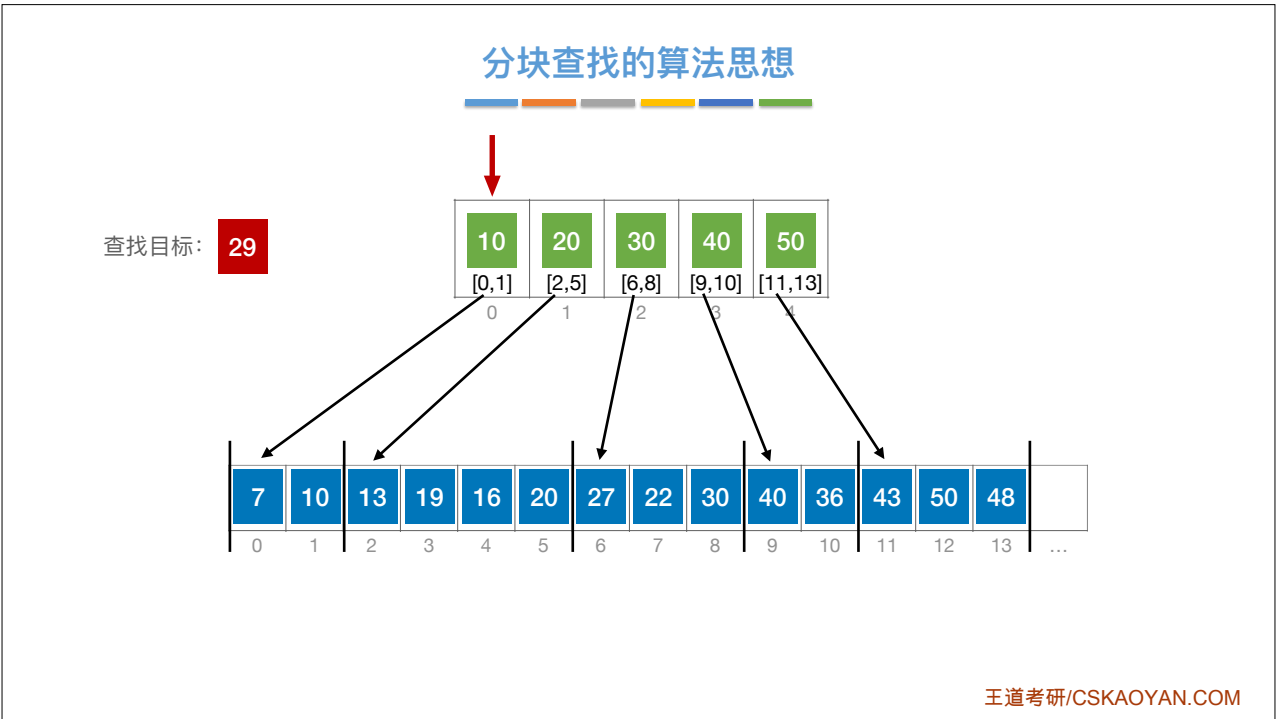
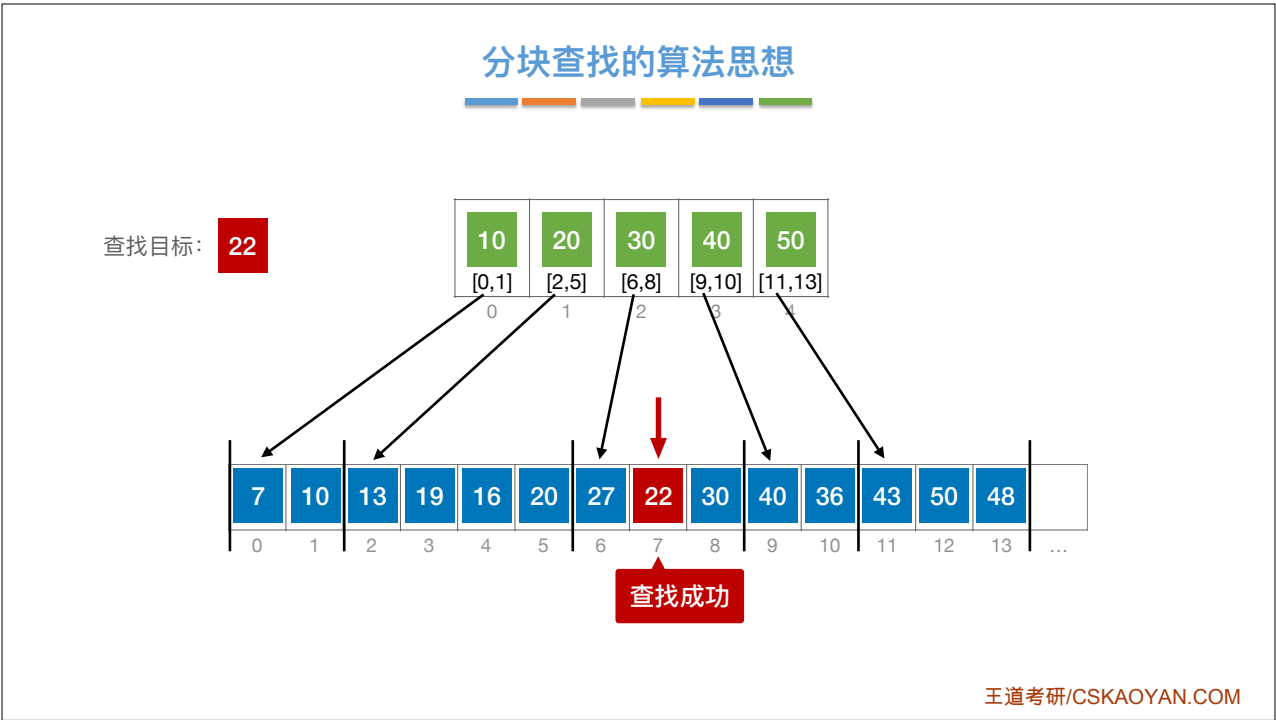
分块查找

算法思想

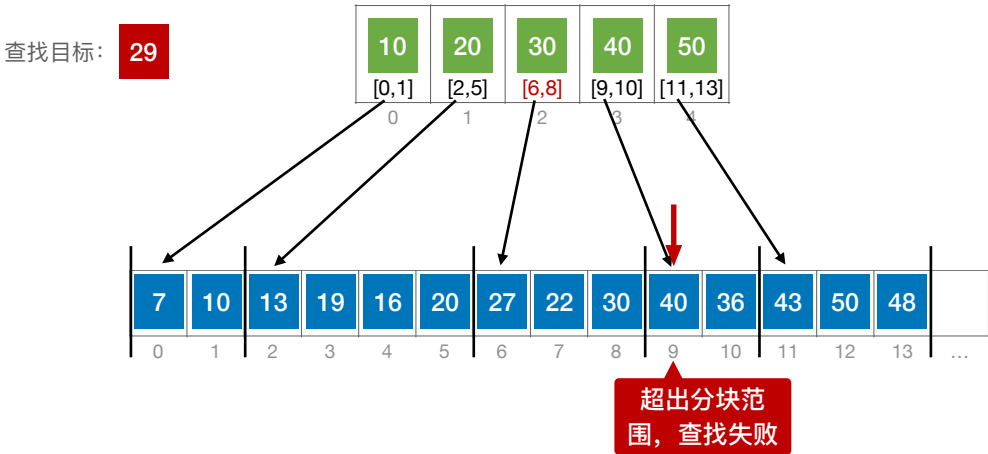
查找效率分析 (ASL)

王道考研/CSKAOYAN.COM





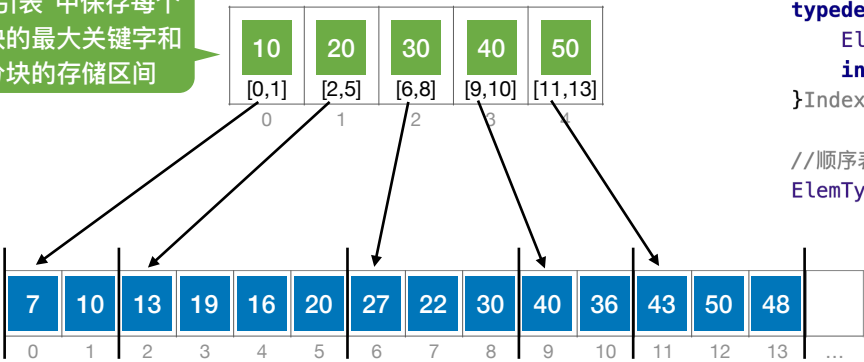
分块查找的算法思想



王道考研/CSKAOYAN.COM

分块查找的算法思想

“索引表”中保存每个分块的最大关键字和分块的存储区间



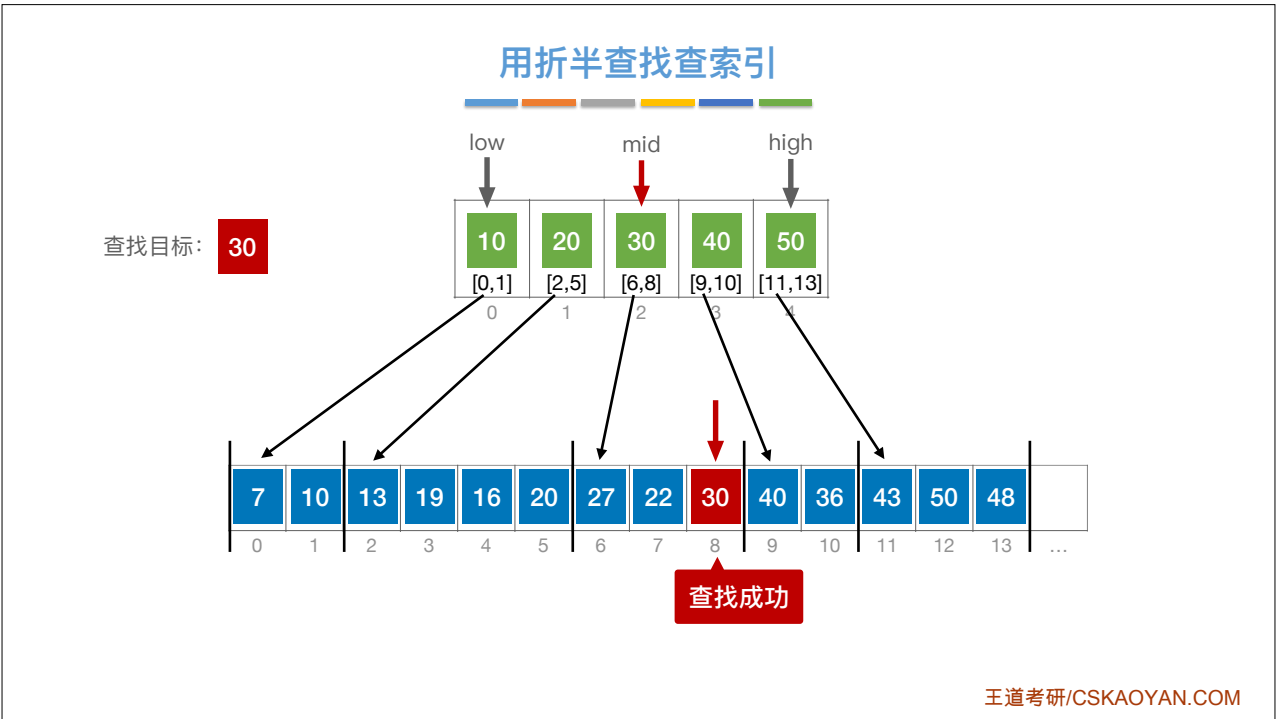
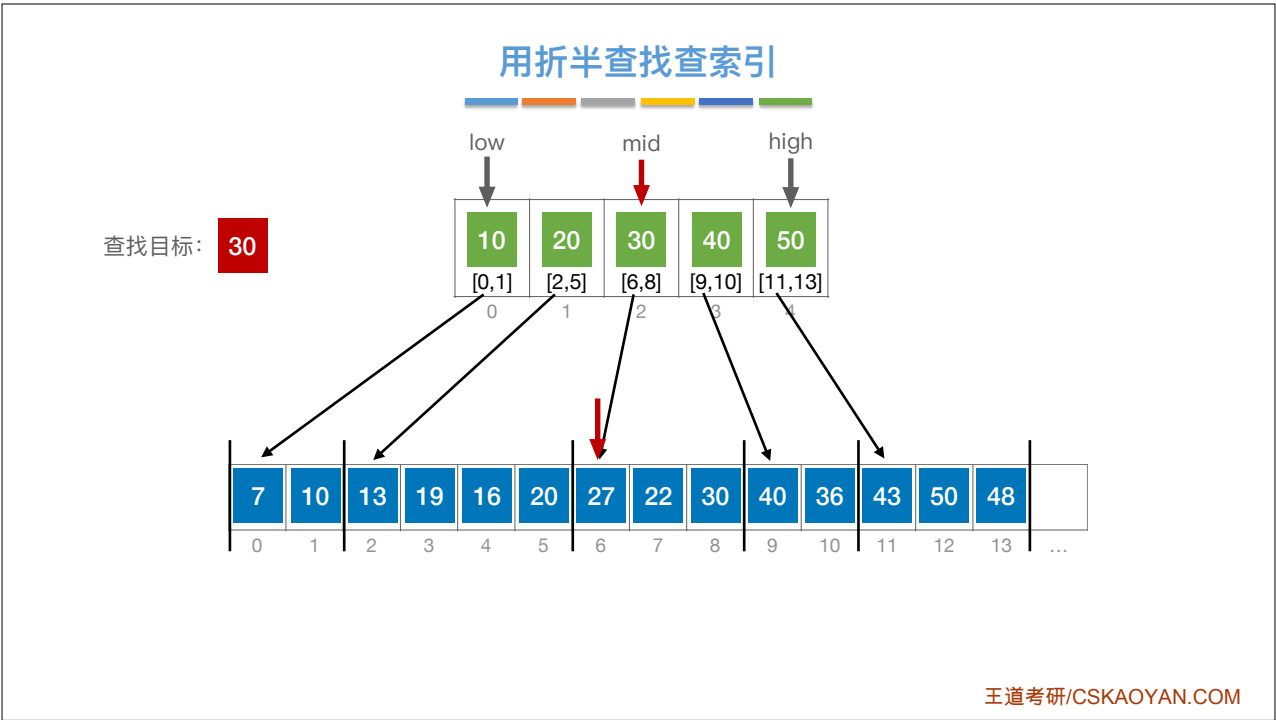
```
//索引表
typedef struct {
    ElemType maxVal;
    int low,high;
}Index;
```

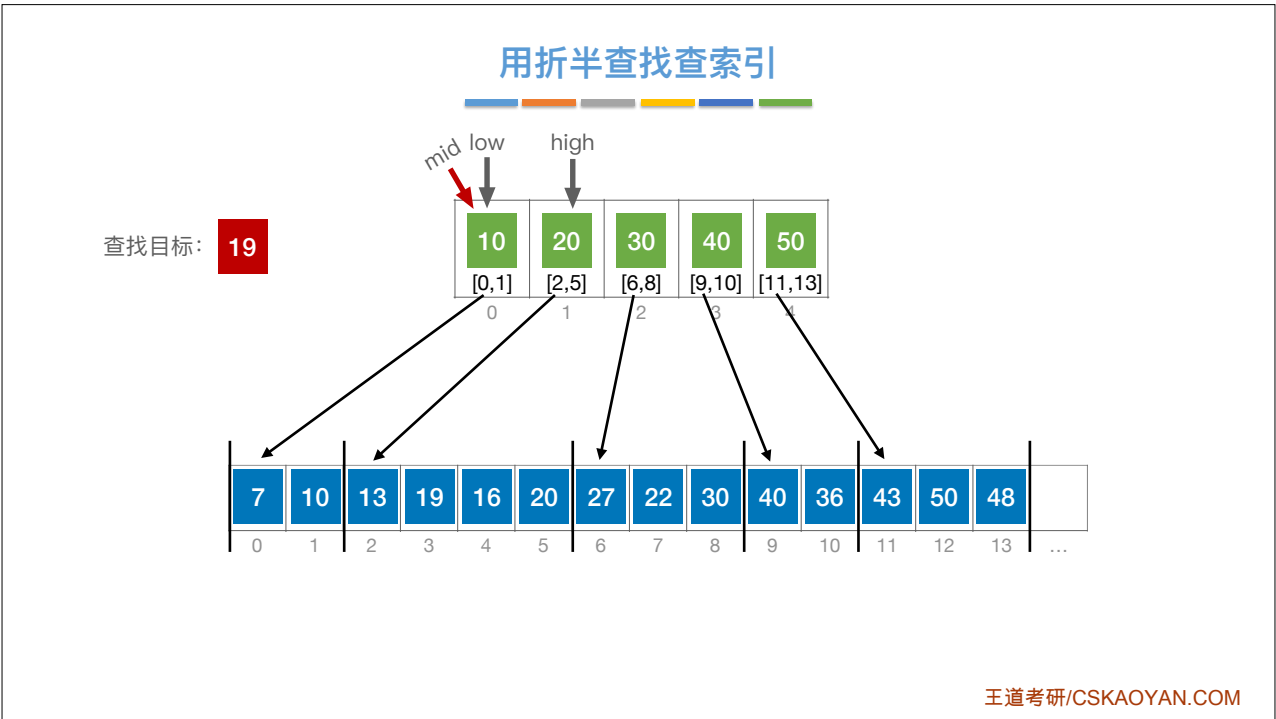
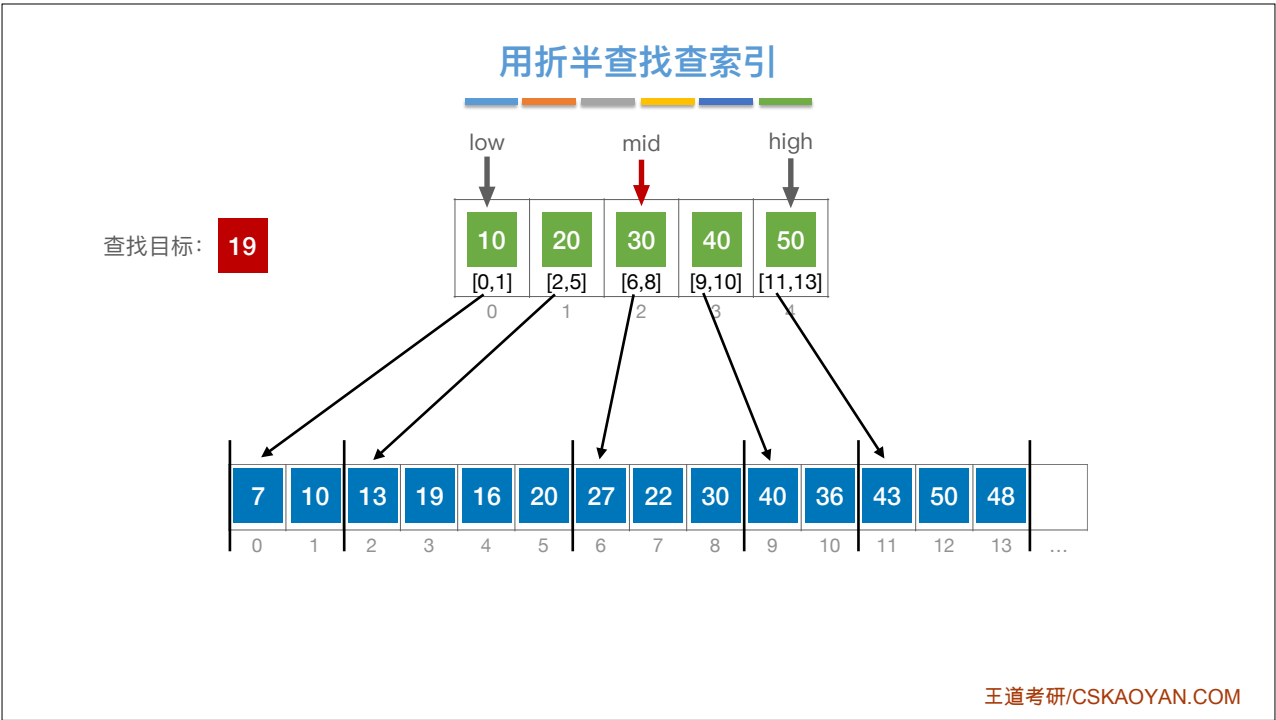
```
//顺序表存储实际元素
ElemType List[100];
```

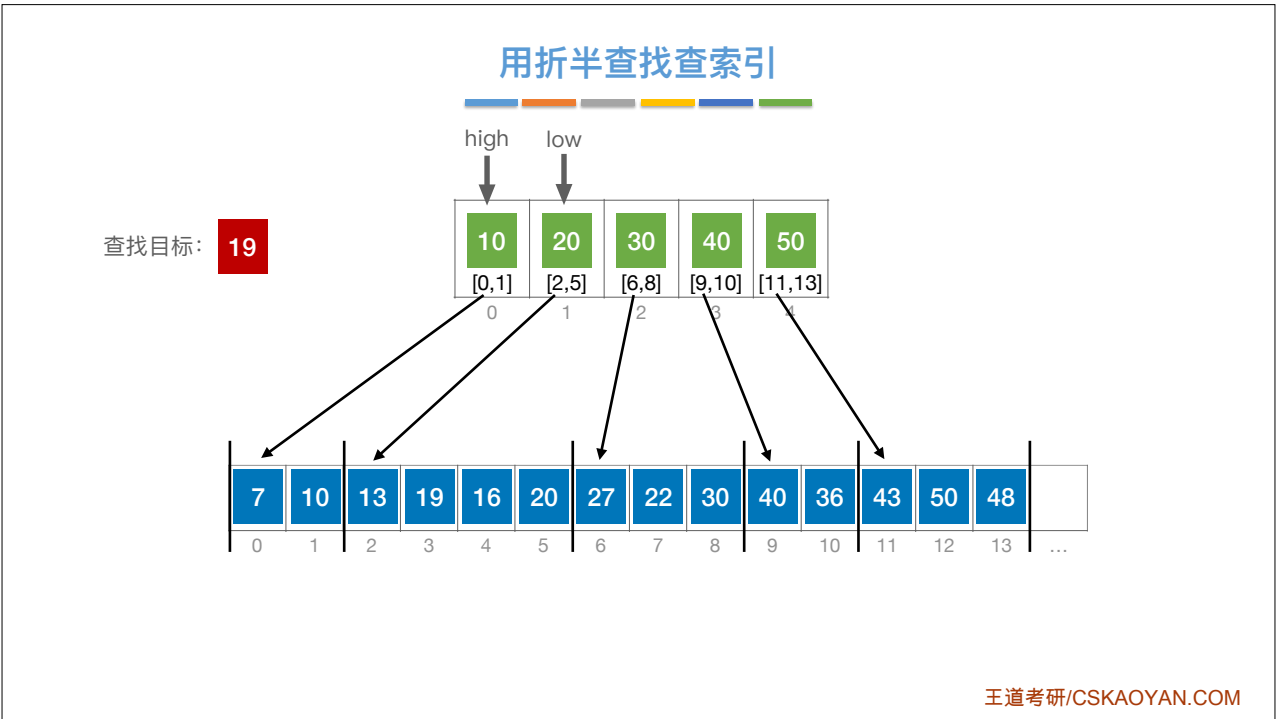
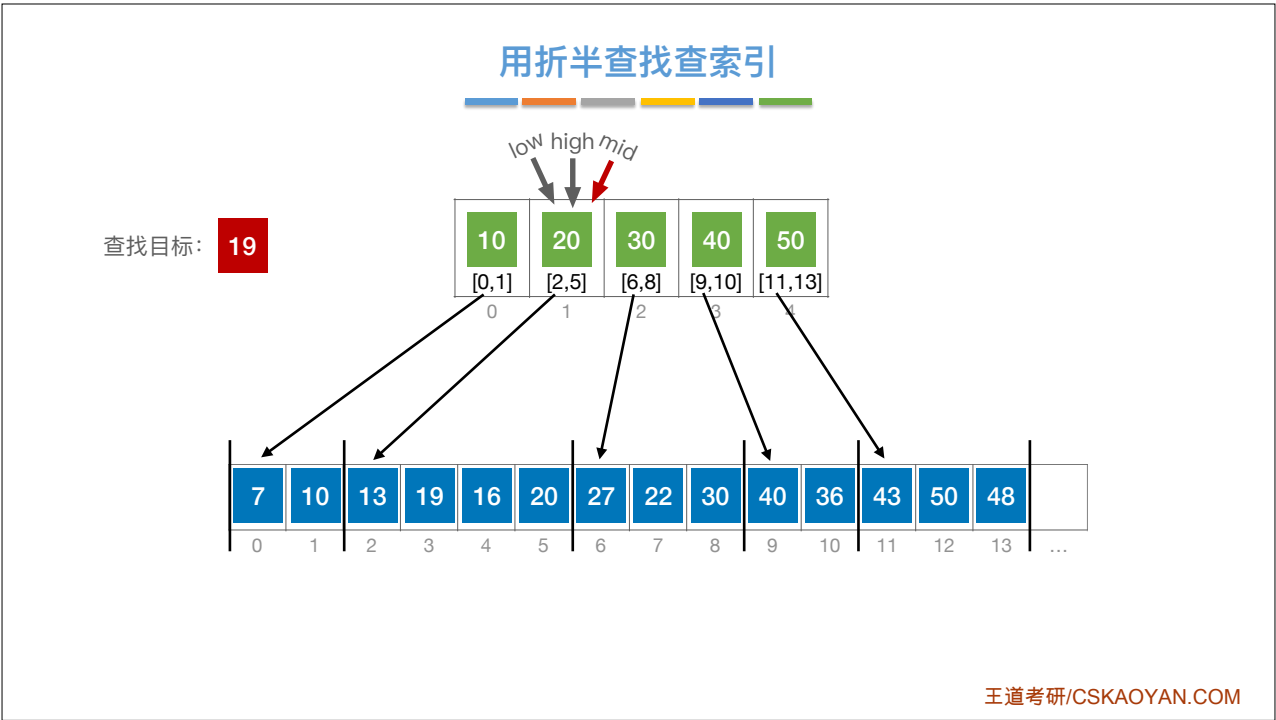
分块查找，又称索引顺序查找，算法过程如下：

- ①在索引表中确定待查记录所属的分块（可顺序、可折半）
- ②在块内顺序查找

王道考研/CSKAOYAN.COM







用折半查找查索引

查找目标：19

high

low

10

20

30

40

50

[0,1]

[2,5]

[6,8]

[9,10]

[11,13]

0

1

2

3

4

7

10

13

19

16

20

27

22

30

40

36

43

50

48

...

0

1

2

3

4

5

6

7

8

9

10

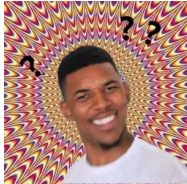
11

12

13

...

原因：最终low左边一定小于目标关键字，high右边一定大于目标关键字。而分块存储的索引表中保存的是各个分块的最大关键字



若索引表中不包含目标关键字，则折半查找索引表最终停在 $low > high$ ，要在low所指分块中查找

王道考研/CSKAOYAN.COM

用折半查找查索引

查找目标：19

high

low

10

20

30

40

50

[0,1]

[2,5]

[6,8]

[9,10]

[11,13]

0

1

2

3

4

7

10

13

19

16

20

27

22

30

40

36

43

50

48

...

0

1

2

3

4

5

6

7

8

9

10

11

12

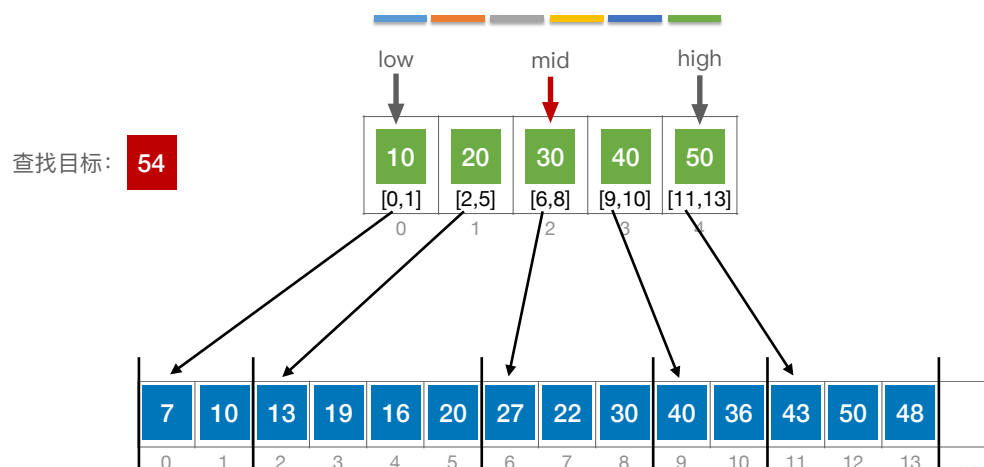
13

...

查找成功

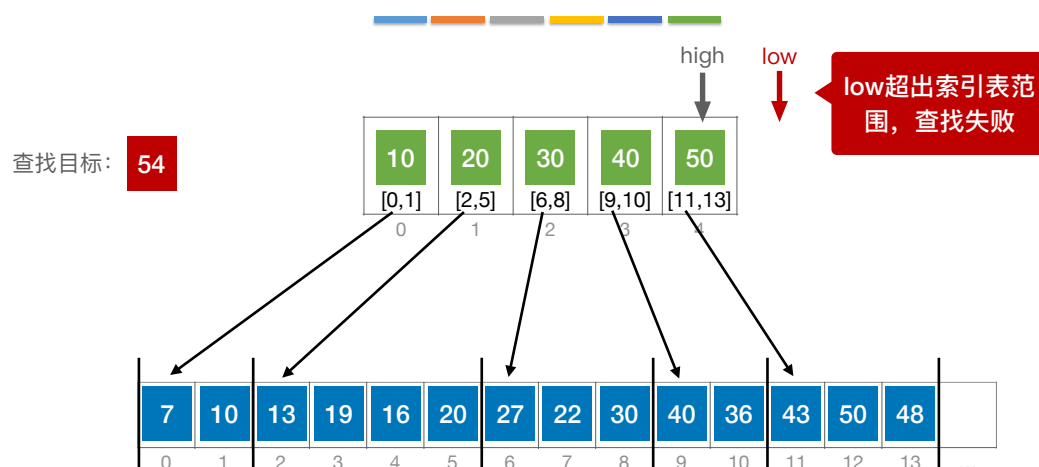
王道考研/CSKAOYAN.COM

用折半查找查索引



王道考研/CSKAOYAN.COM

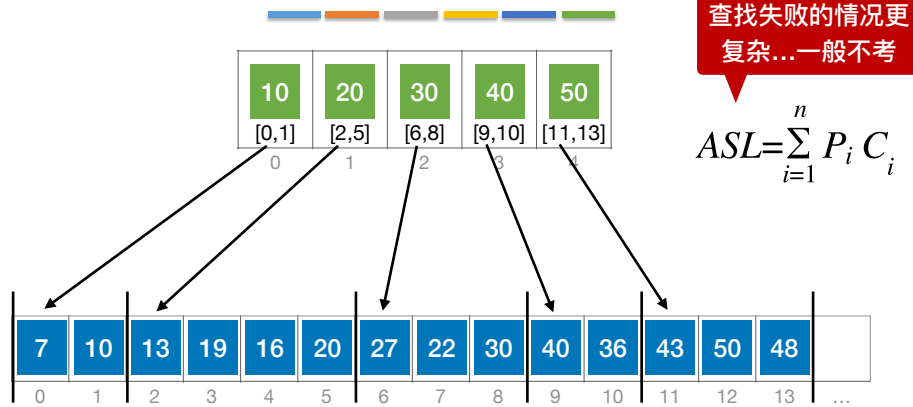
用折半查找查索引



若索引表中不包含目标关键字, 则折半查索引表最终停在 $low > high$, 要在low所指分块中查找

王道考研/CSKAOYAN.COM

查找效率分析 (ASL)



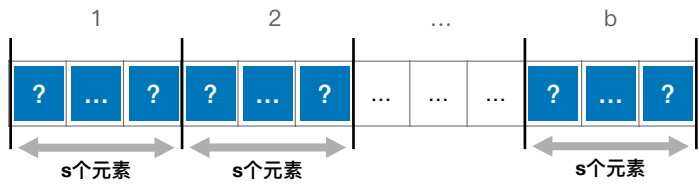
共有14个元素，各自被查概率为1/14
若索引表采用顺序查找，则 7：2次、10：3次、13：3次...
若索引表采用折半查找，则30：4次、27：2次？

放弃思考



王道考研/CSKAOYAN.COM

查找效率分析 (ASL)



假设，长度为n的查找表被均匀地分为b块，每块s个元素

设索引查找和块内查找的平均查找长度分别为 L_I 、 L_S ，则分块查找的平均查找长度为

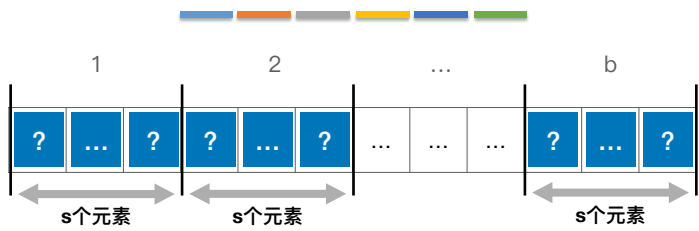
$$ASL = L_I + L_S$$

用顺序查找索引表，则 $L_I = \frac{(1+2+\dots+b)}{b} = \frac{b+1}{2}$ ， $L_S = \frac{(1+2+\dots+s)}{s} = \frac{s+1}{2}$
则 $ASL = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2+2s+n}{2s}$ ，当 $s=\sqrt{n}$ 时， $ASL_{\min}=\sqrt{n}+1$

若n=10000，则
 $ASL_{\min}=101$

王道考研/CSKAOYAN.COM

查找效率分析 (ASL)



假设，长度为n的查找表被均匀地分为b块，每块s个元素

设索引查找和块内查找的平均查找长度分别为 L_I 、 L_S ，则分块查找的平均查找长度为

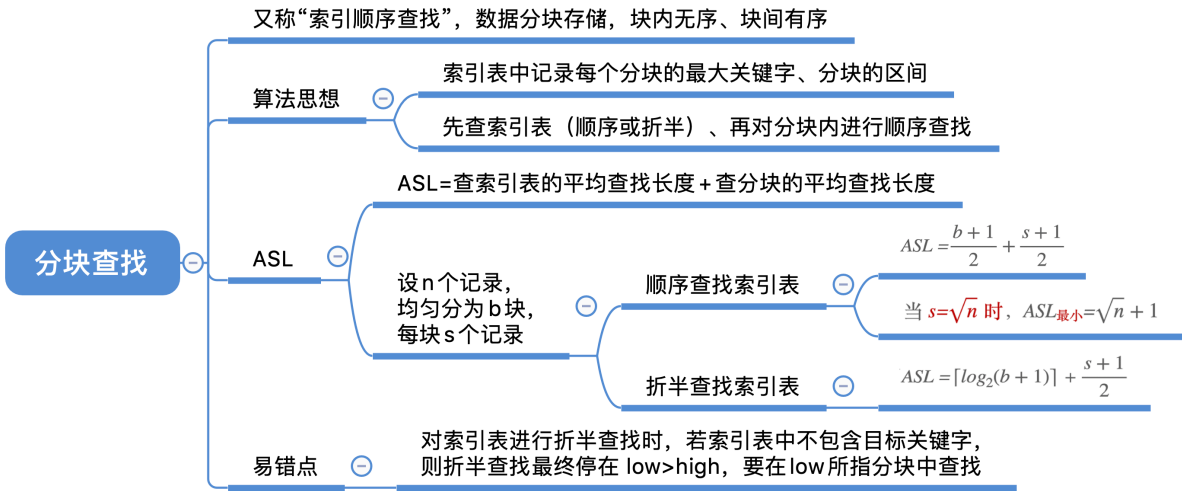
$$ASL=L_I+L_S$$

用折半查找查索引表，则 $L_I=\lceil \log_2(b+1)\rceil$ ， $L_S=\frac{(1+2+\dots+s)}{s}=\frac{s+1}{2}$

则 $ASL=\lceil \log_2(b+1)\rceil+\frac{s+1}{2}$

王道考研/CSKAOYAN.COM

知识回顾与重要考点

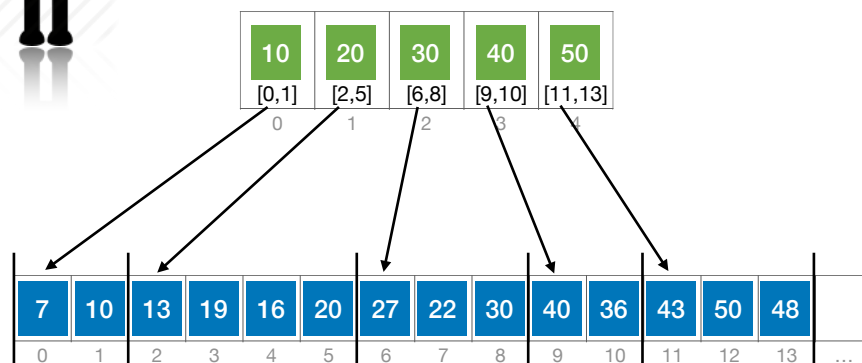


王道考研/CSKAOYAN.COM

拓展思考



若查找表是“动态查找表”，有木有更好的实现方式？

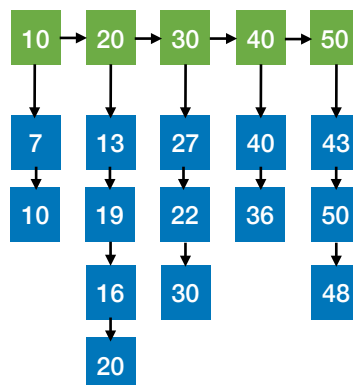


王道考研/CSKAOYAN.COM

拓展思考



若查找表是“动态查找表”，有木有更好的实现方式？——链式存储



王道考研/CSKAOYAN.COM

本节内容

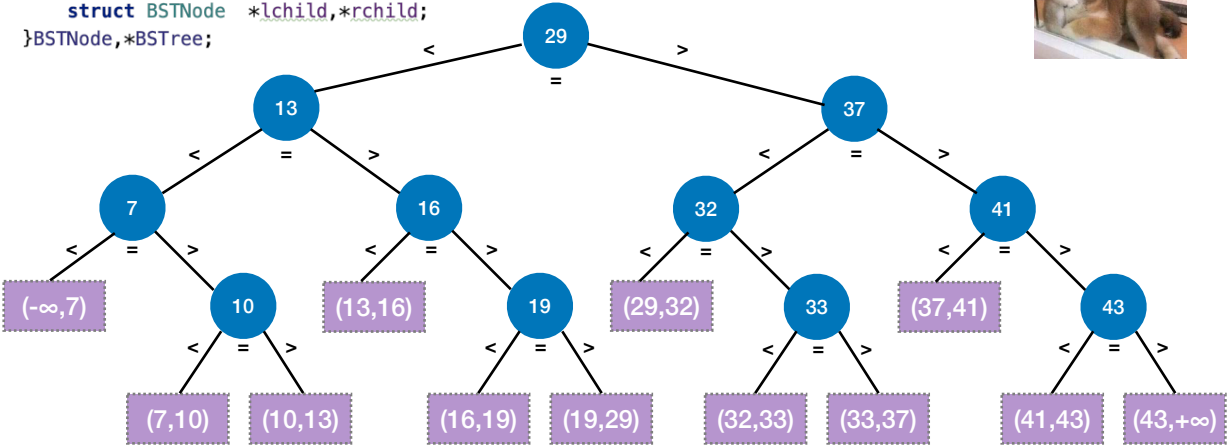
B树

王道考研/CSKAOYAN.COM

回顾：二叉查找树 (BST)

```
//二叉排序树结点
typedef struct BSTNode{
    int key;
    struct BSTNode *lchild,*rchild;
}BSTNode,*BSTree;
```

能不能变成m
叉查找树?

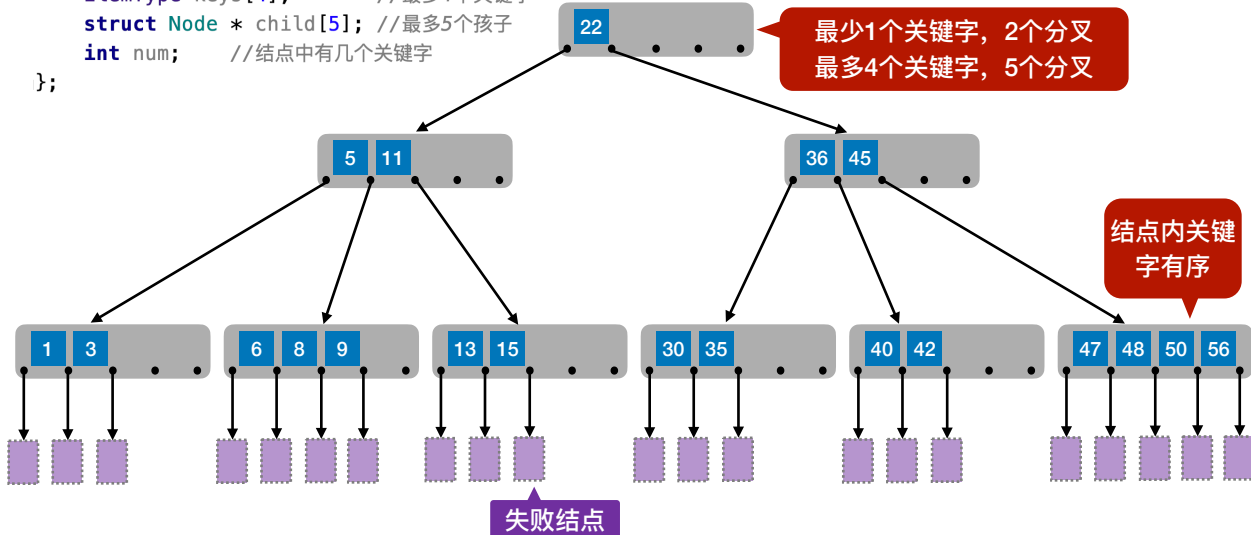


王道考研/CSKAOYAN.COM

5叉查找树

//5叉排序树的结点定义

```
struct Node {  
    ElemType keys[4]; //最多4个关键字  
    struct Node * child[5]; //最多5个孩子  
    int num; //结点中有几个关键字  
};
```

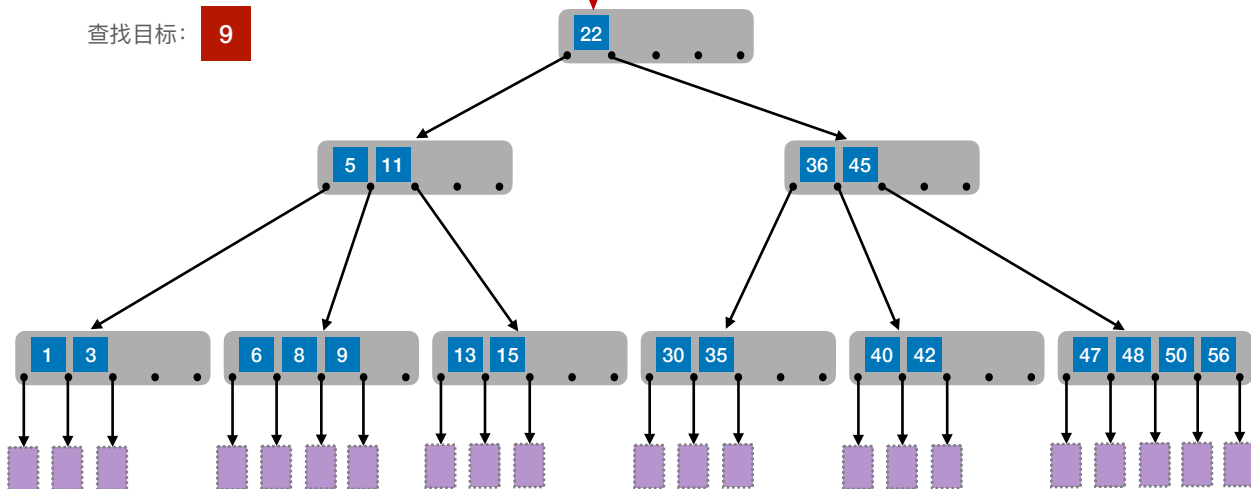


王道考研/CSKAOYAN.COM

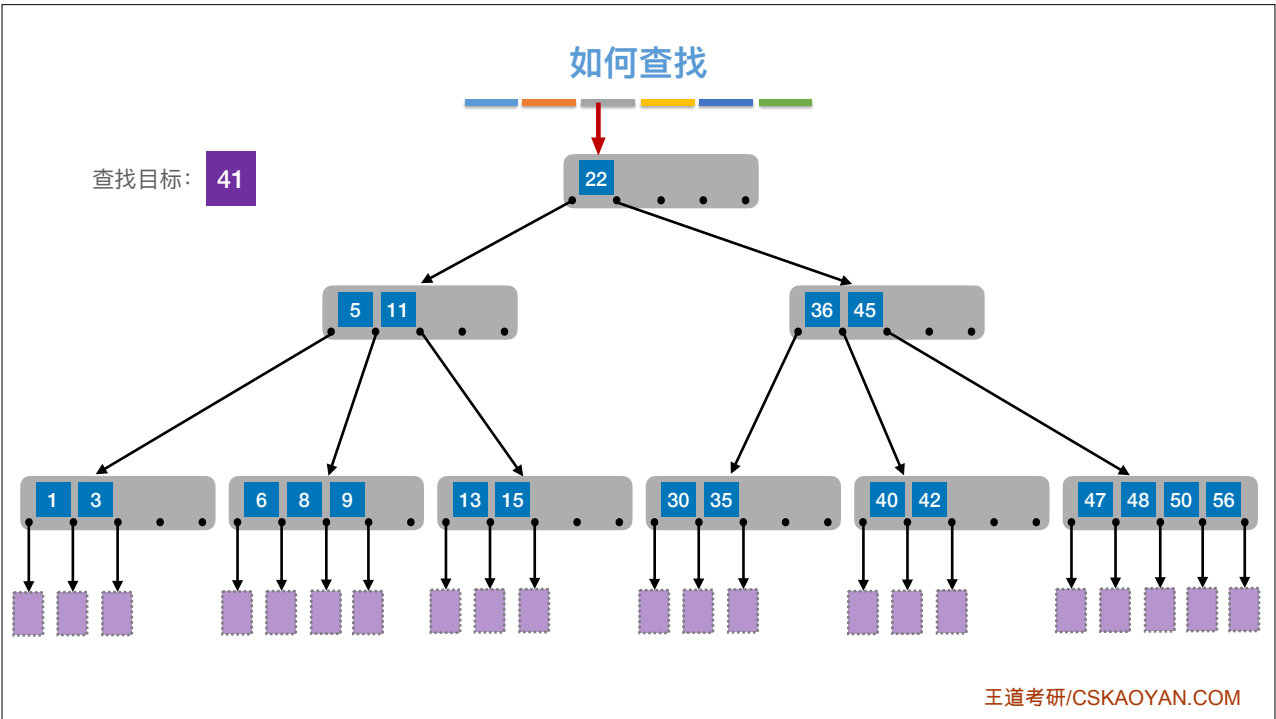
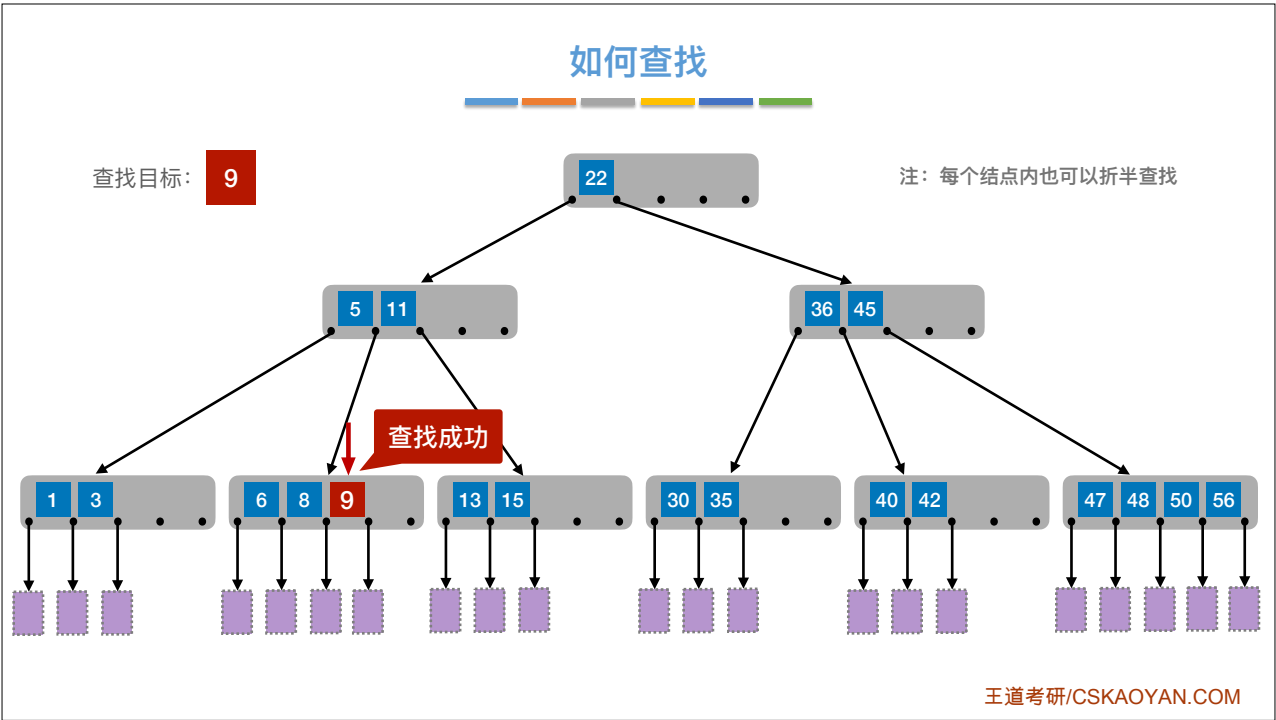
如何查找

查找目标:

9

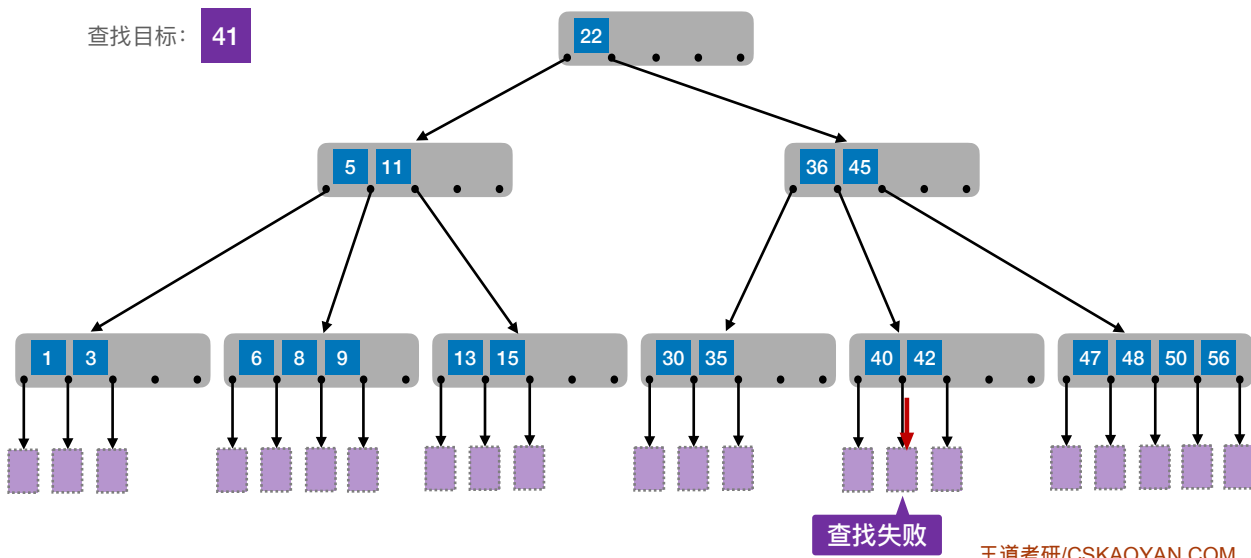


王道考研/CSKAOYAN.COM



如何查找

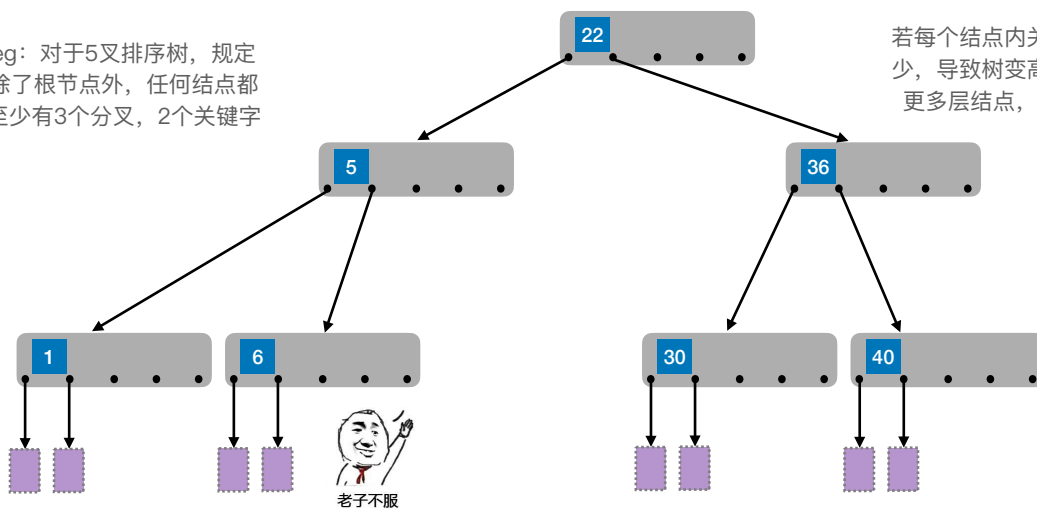
查找目标: 41



如何保证查找效率

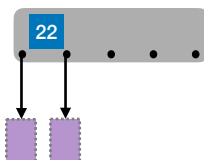
eg: 对于5叉排序树, 规定除了根节点外, 任何结点都至少有3个分叉, 2个关键字

若每个结点内关键字太少, 导致树变高, 要查更多层结点, 效率低



策略: m叉查找树中, 规定除了根节点外, 任何结点至少有 $\lceil m/2 \rceil$ 个分叉, 即至少含有 $\lceil m/2 \rceil - 1$ 个关键字

如何保证查找效率



这个我也没有 没办法

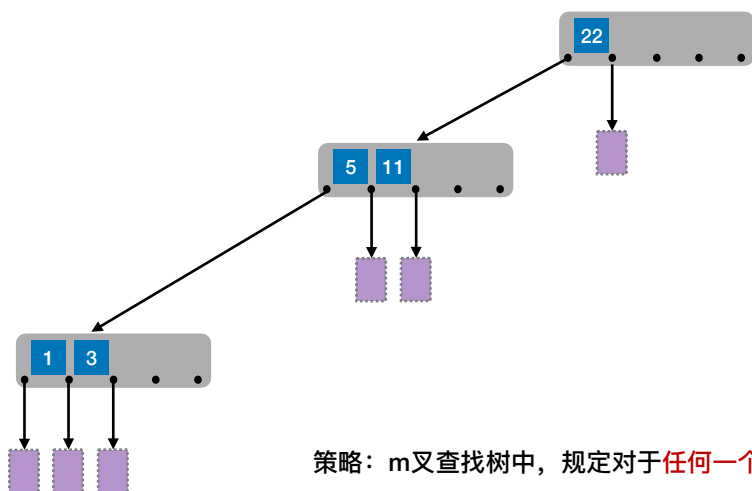


如果整个树只有1个元素，根节点只有两个分叉

策略：m叉查找树中，规定**除了根节点外**，任何结点至少有 $\lceil m/2 \rceil$ 个分叉，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字

王道考研/CSKAOYAN.COM

如何保证查找效率

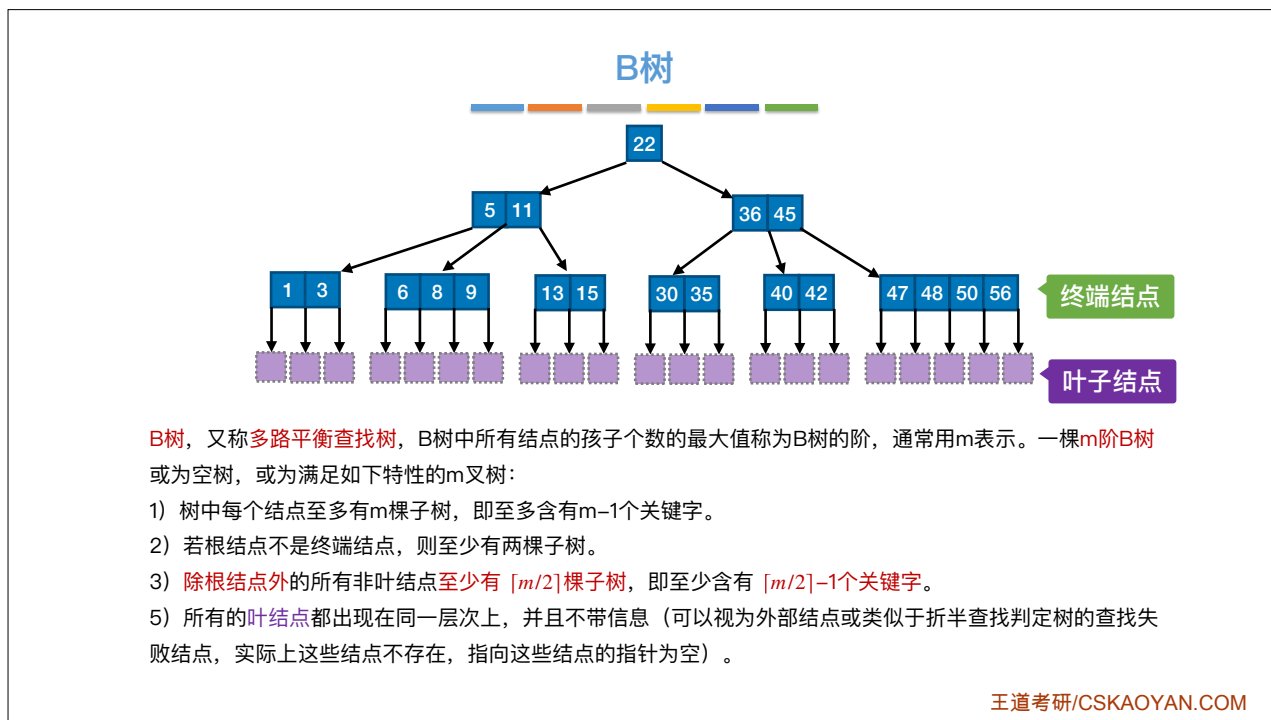
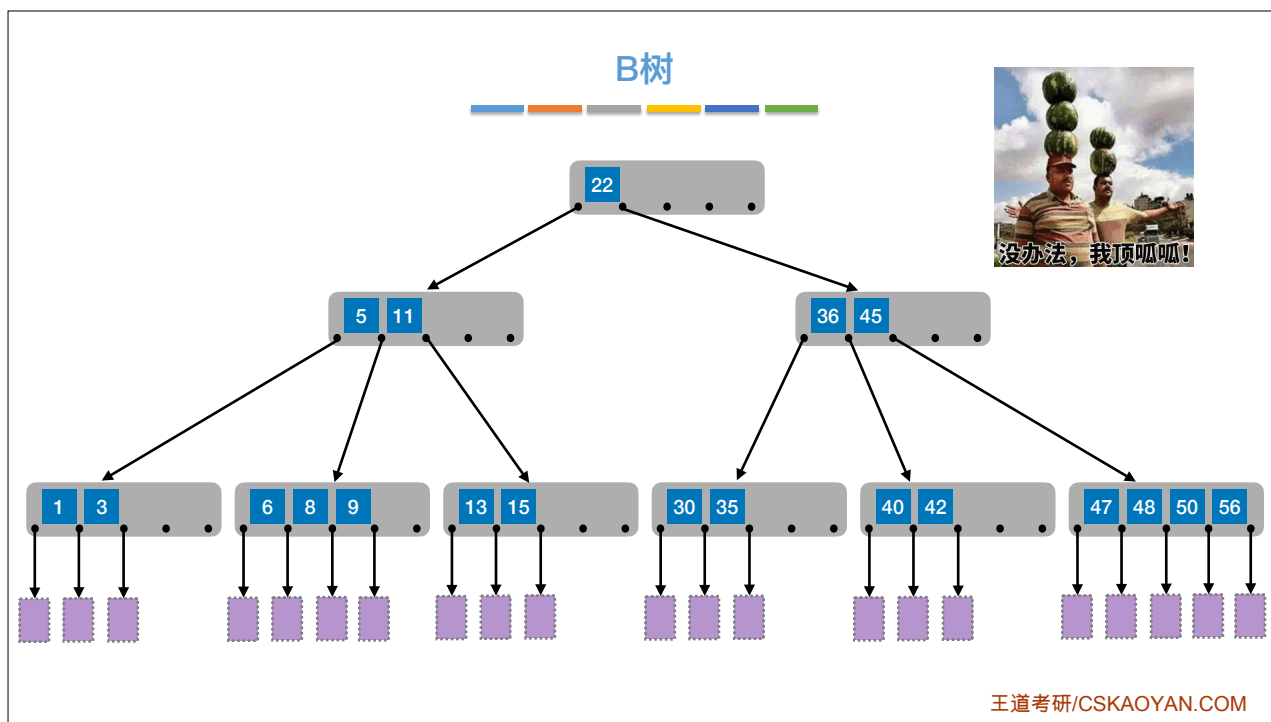


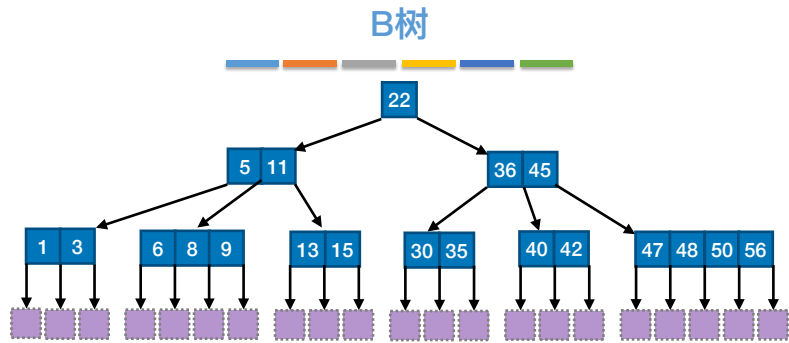
还不够

不够“平衡”，树会很高，要查很多层结点

策略：m叉查找树中，规定对于**任何一个结点**，其**所有子树的高度都要相同**。

王道考研/CSKAOYAN.COM

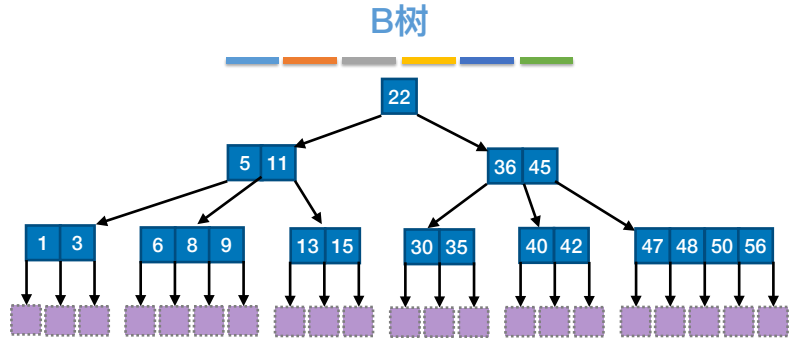




4) 所有非叶结点的结构如下：

n	P_0	K_1	P_1	K_2	P_2	$\dots$	K_n	P_n
-----	-------	-------	-------	-------	-------	---------	-------	-------

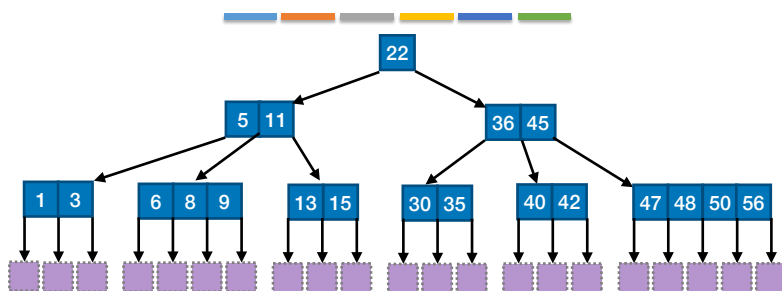
其中， K_i ($i = 1, 2, \dots, n$) 为结点的关键字，且满足 $K_1 < K_2 < \dots < K_n$ ； P_i ($i = 0, 1, \dots, n$) 为指向子树根结点的指针，且指针 P_{i-1} 所指子树中所有结点的关键字均小于 K_i ， P_i 所指子树中所有结点的关键字均大于 K_i ， n ($\lceil m/2 \rceil - 1 \leq n \leq m - 1$) 为结点中关键字的个数。



m阶B树的核心特性：

- 1) 根节点的子树数 $\in [2, m]$ ，关键字数 $\in [1, m-1]$ 。
其他结点的子树数 $\in [\lceil m/2 \rceil, m]$ ；关键字数 $\in [\lceil m/2 \rceil - 1, m-1]$
- 2) 对任一结点，其所有子树高度都相同
- 3) 关键字的值：子树0 < 关键字1 < 子树1 < 关键字2 < 子树2 < ... (类比二叉查找树 左 < 中 < 右)

B树的高度



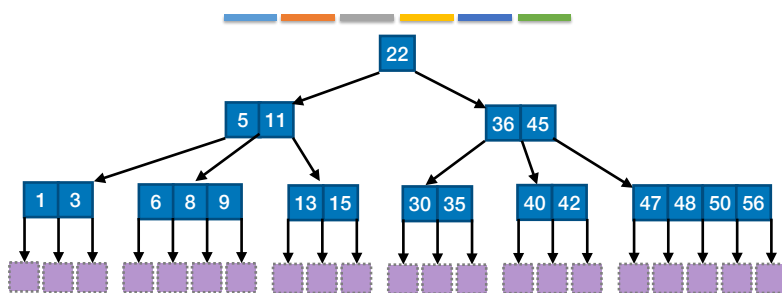
注：大部分学校算B树的高度不包括叶子结点（失败结点）

问题：含n个关键字的m阶B树，最小高度、最大高度是多少？

最小高度——让每个结点尽可能的满，有m-1个关键字，m个分叉，则有
 $n \leq (m-1)(1 + m + m^2 + m^3 + \dots + m^{h-1}) = m^h - 1$ ，因此 $h \geq \log_m(n+1)$

王道考研/CSKAOYAN.COM

B树的高度



注：大部分学校算B树的高度不包括叶子结点（失败结点）

最大高度——让各层的分叉尽可能的少，即根节点只有2个分叉，其他结点只有 $\lceil m/2 \rceil$ 个分叉
各层结点至少有：第一层 1、第二层 2、第三层 $2\lceil m/2 \rceil$... 第h层 $2(\lceil m/2 \rceil)^{h-2}$
第h+1层共有叶子结点（失败结点） $2(\lceil m/2 \rceil)^{h-1}$ 个

n个关键字的B树必有n+1个叶子结点，则 $n+1 \geq 2(\lceil m/2 \rceil)^{h-1}$ ，即 $h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

n个关键字将数域切分为n+1个区间

王道考研/CSKAOYAN.COM

B树的高度

问题：含n个关键字的m叉B树，最小高度、最大高度是多少？

最大高度——让每个结点包含的关键字、分叉尽可能的少。记 $k=\lceil m/2 \rceil$

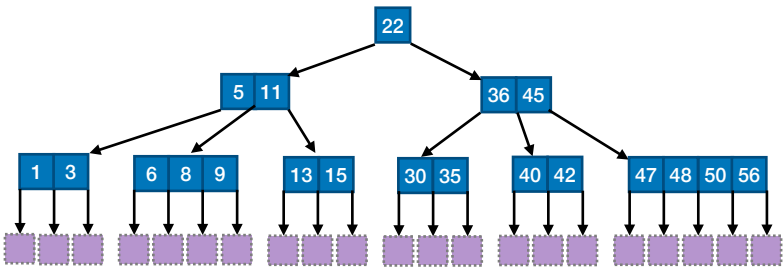
	最少结点数	最少关键字数
第一层	1	1
第二层	2	$2(k-1)$
第三层	$2k$	$2k(k-1)$
第四层	$2k^2$	$2k^2(k-1)$
...	...	
第h层	$2k^{h-2}$	$2k^{h-2}(k-1)$

h层的m阶B树至少包含关键字总数 $1+2(k-1)(k^0+k^1+k^2+\dots+k^{h-2}) = 1+2(k^{h-1}-1)$

若关键字总数少于这个值，则高度一定小于h，因此 $n \geq 1+2(k^{h-1}-1)$

得， $h \leq \log_k \frac{n+1}{2} + 1 = \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

B树的高度



注：大部分学校算B树的高度不包括叶子结点（失败结点）

问题：含n个关键字的m阶B树，最小高度、最大高度是多少？

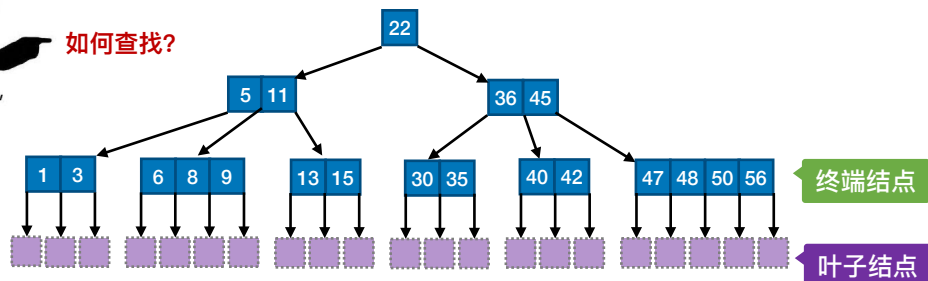
$\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

知识回顾与重要考点



如何查找?

字不重要,
看图!



m阶B树的核心特性:

尽可能“满”

1) 根节点的子树数 $\in [2, m]$, 关键字数 $\in [1, m-1]$ 。

其他结点的子树数 $\in [\lceil m/2 \rceil, m]$; 关键字数 $\in [\lceil m/2 \rceil - 1, m-1]$

尽可能“平衡”

2) 对任一结点, 其所有子树高度都相同

3) 关键字的值: 子树0<关键字1<子树1<关键字2<子树2<.... (类比二叉查找树 左<中<右)

含n个关键字的m叉B树, $\log_m(n+1) \leq h \leq \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$

王道考研/CSKAOYAN.COM

本节内容

B树

插入和删除

王道考研/CSKAOYAN.COM

B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即：2≤n≤4 （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即：2≤n≤4 （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



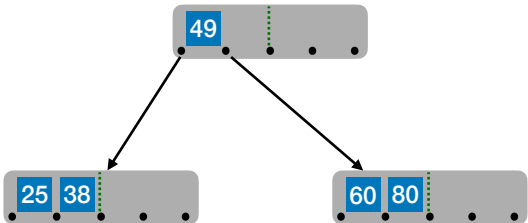
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

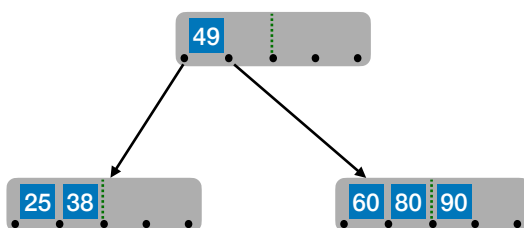


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

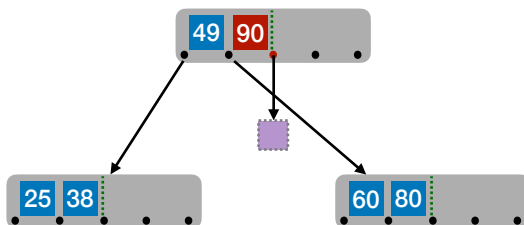


新元素一定是插入到最底层“终端节点”，用“查找”来确定插入位置

B树的插入



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

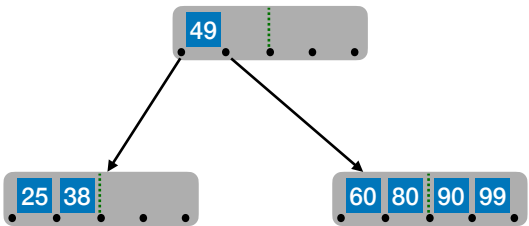


错误示范

注意：B树的失败结点只能出现在最下面一层

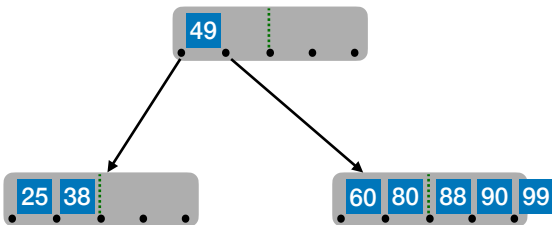
B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入

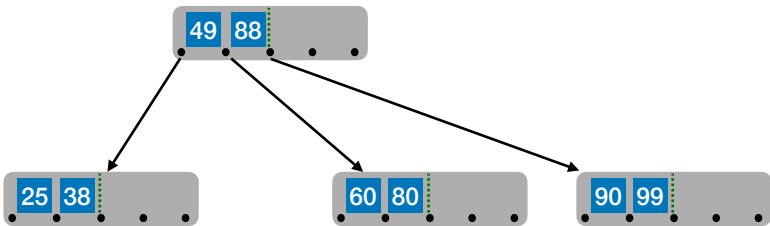
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

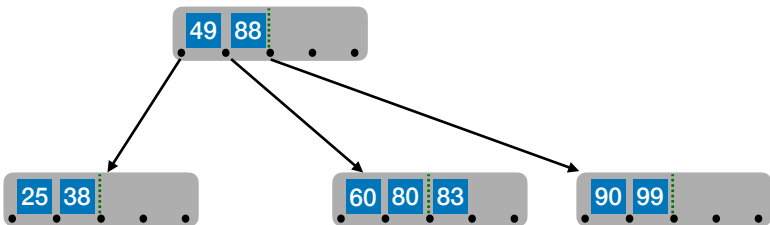
B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



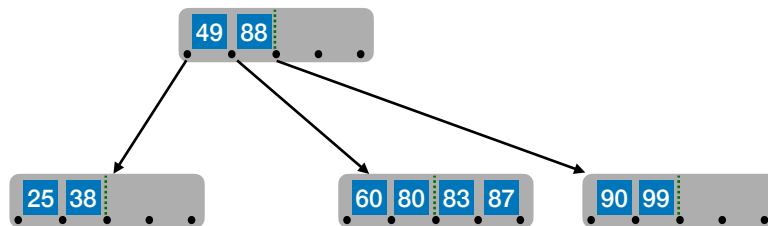
B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

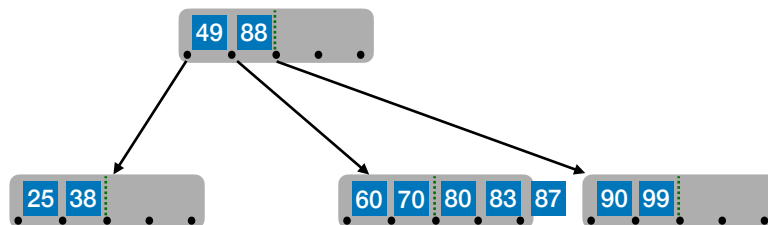


B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



思考：80要放到父节点中，放在哪个位置合适？



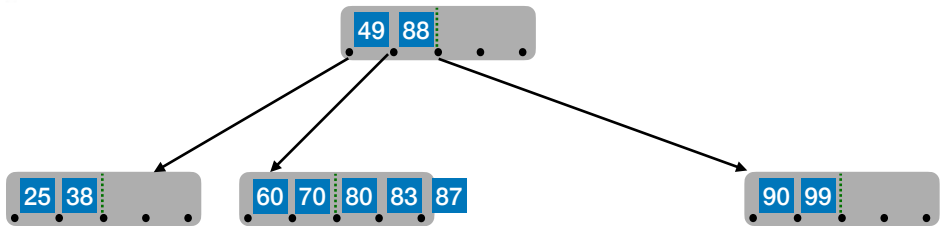
在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



思考：80要放到父节点中，放在哪个位置合适？

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

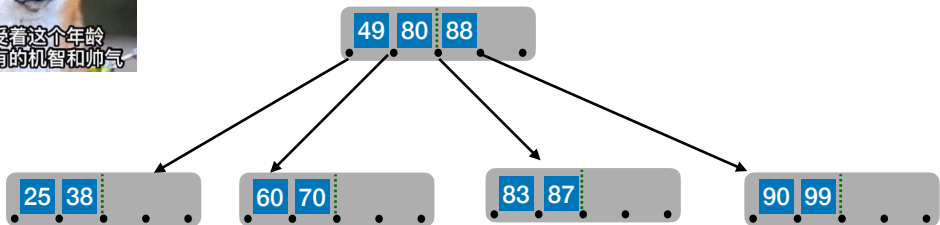


在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入



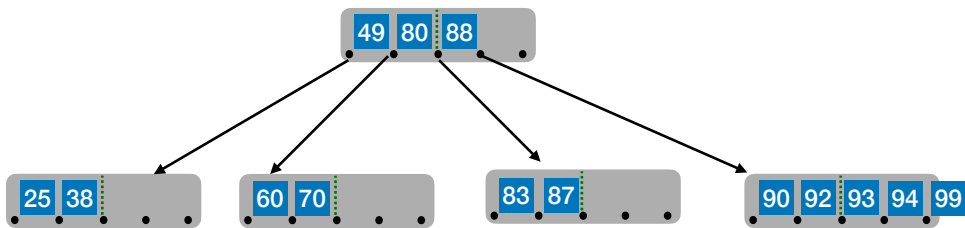
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

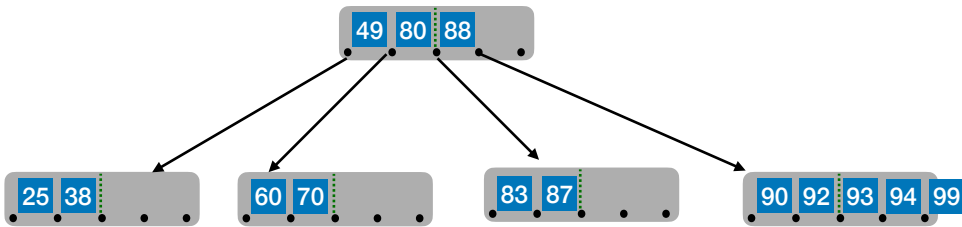
B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



B树的插入

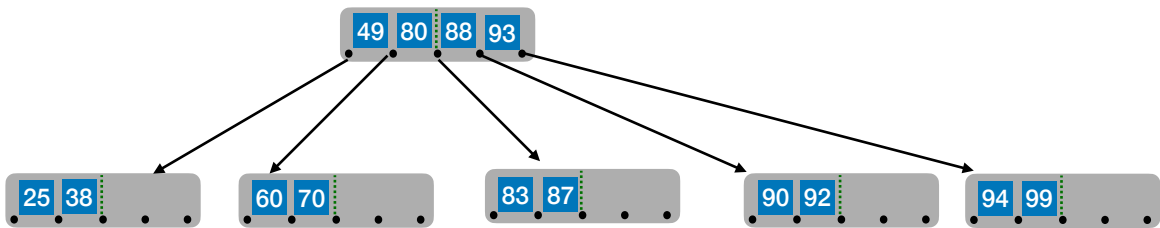
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入

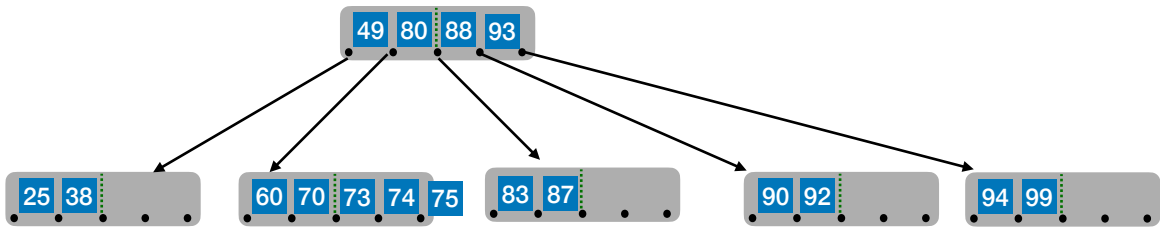
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入

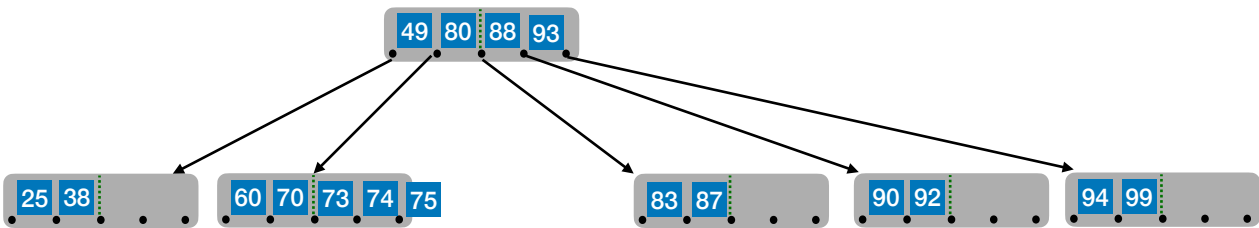
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入

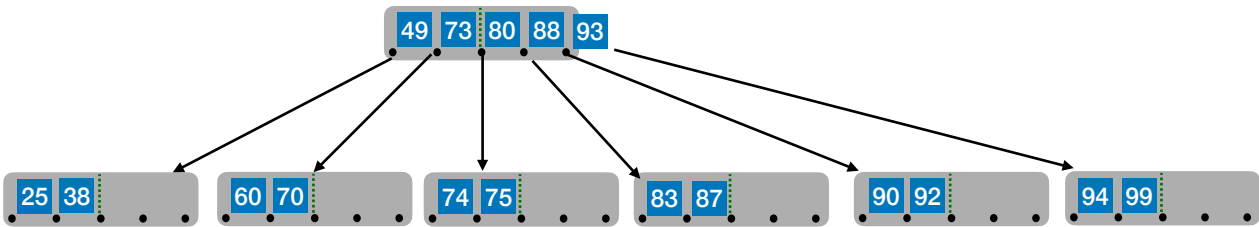
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入

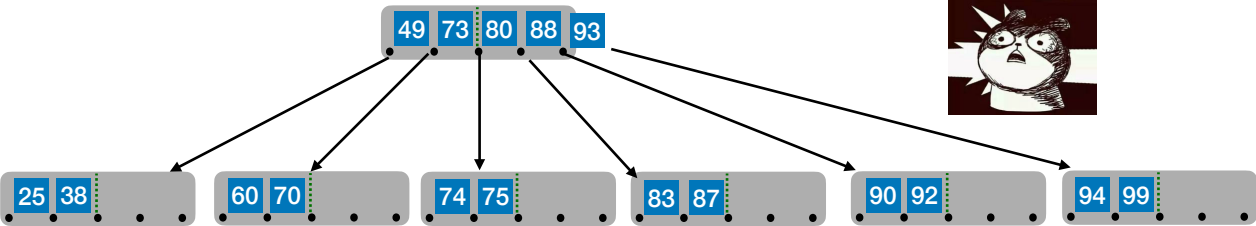
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点

B树的插入

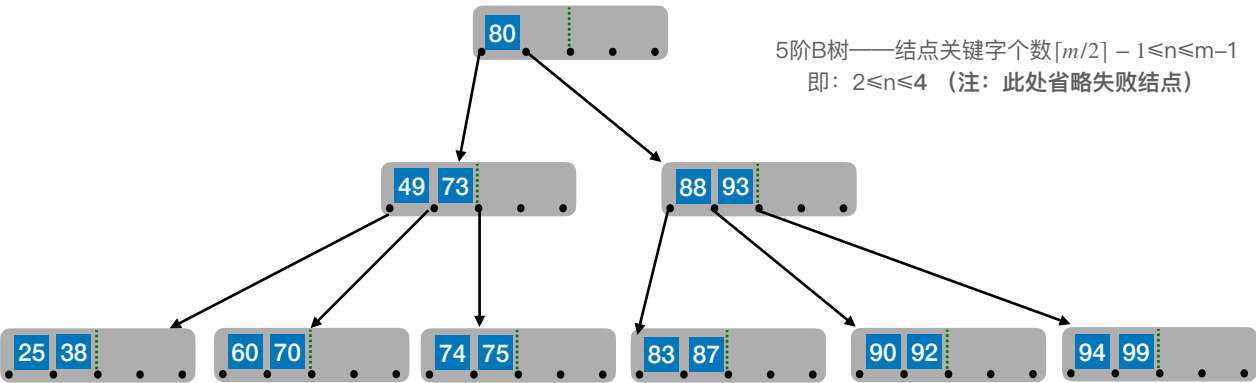
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点。若此时导致其父结点的关键字数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

B树的插入

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点。若此时导致其父结点的关键字数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

B树的插入

核心要求：

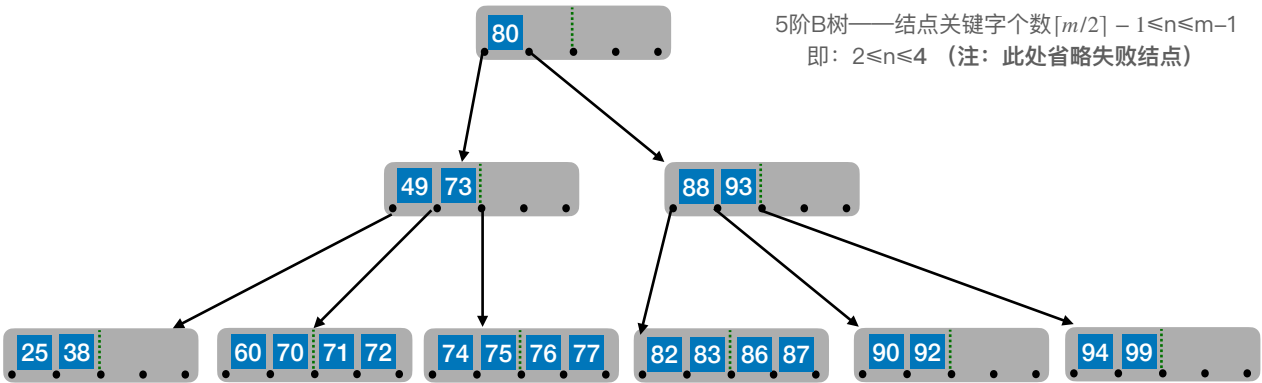
- ①对m阶B树——除根节点外，结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
- ②子树 $0 < \text{关键字}1 < \text{子树}1 < \text{关键字}2 < \text{子树}2 < \dots$

新元素一定是插入到最底层“终端节点”，用“查找”来确定插入位置

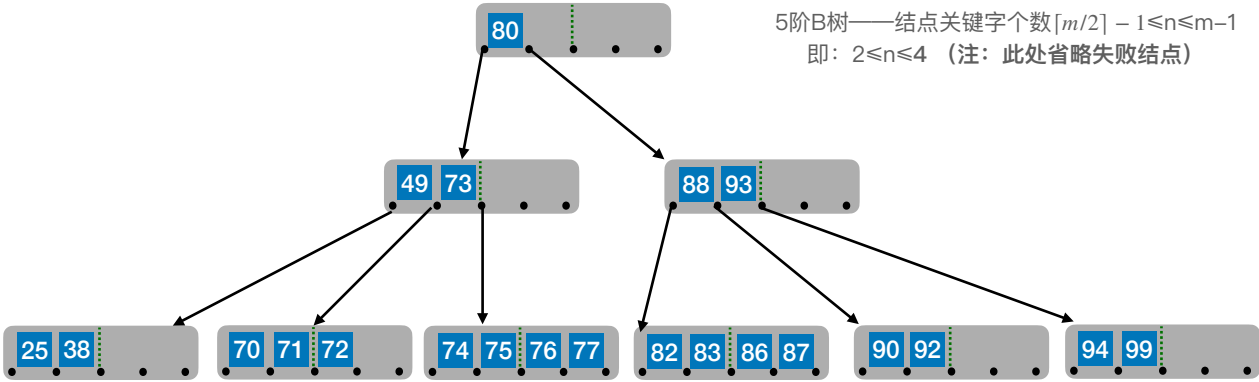
在插入key后，若导致原结点关键字数超过上限，则从中间位置（ $\lceil m/2 \rceil$ ）将其中的关键字分为两部分，左部分包含的关键字放在原结点中，右部分包含的关键字放到新结点中，中间位置（ $\lceil m/2 \rceil$ ）的结点插入原结点的父结点。若此时导致其父结点的关键字数也超过了上限，则继续进行这种分裂操作，直至这个过程传到根结点为止，进而导致B树高度增1。

B树的删除

5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

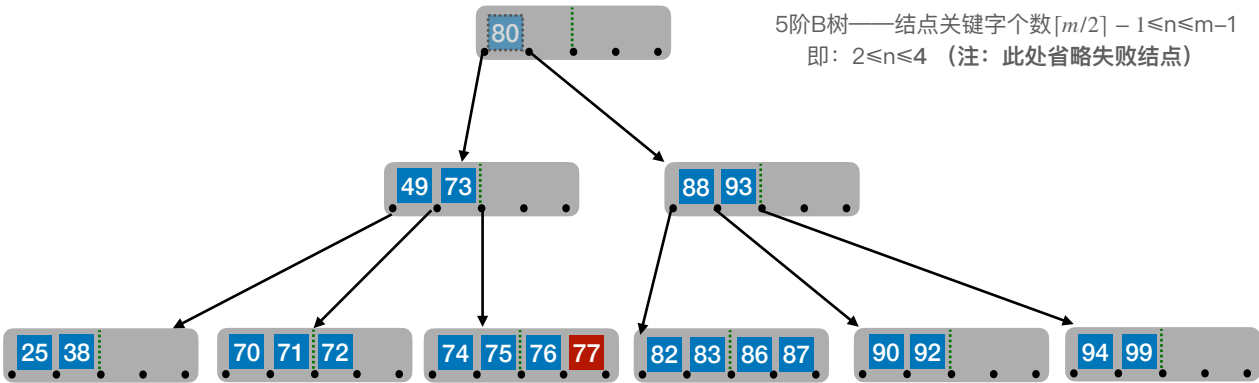


B树的删除



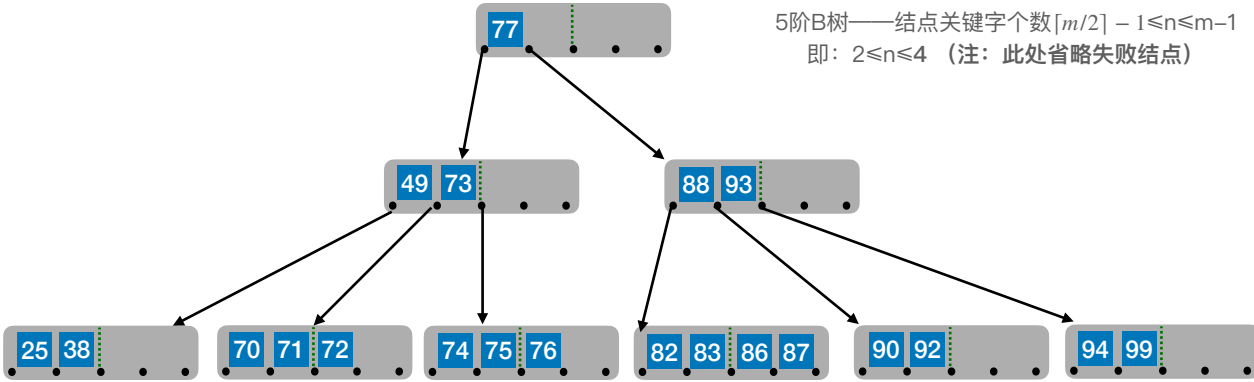
若被删除关键字在**终端节点**，则直接删除该关键字（要注意节点关键字个数是否低于下限 $\lceil m/2 \rceil - 1$ ）

B树的删除



若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字
直接前驱：当前关键字**左侧**指针所指子树中“最右下”的元素

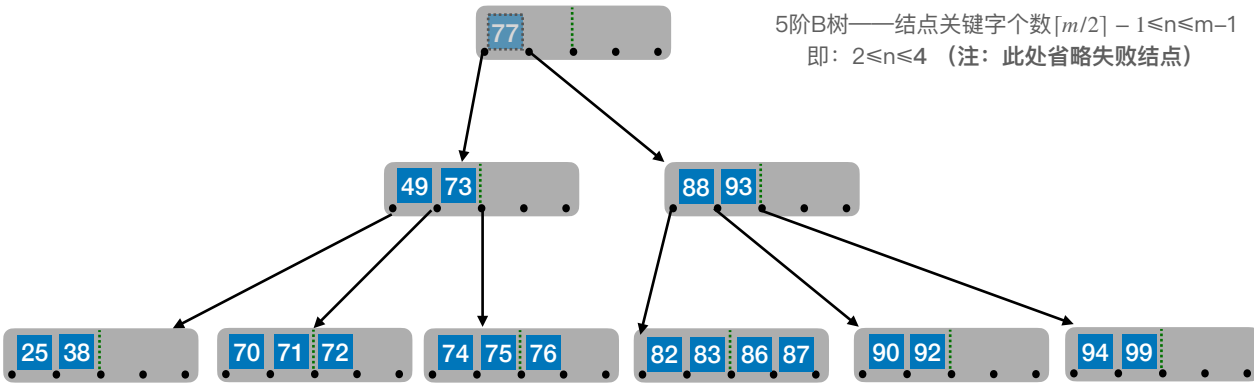
B树的删除



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字
直接前驱：当前关键字**左侧**指针所指子树中“最右下”的元素

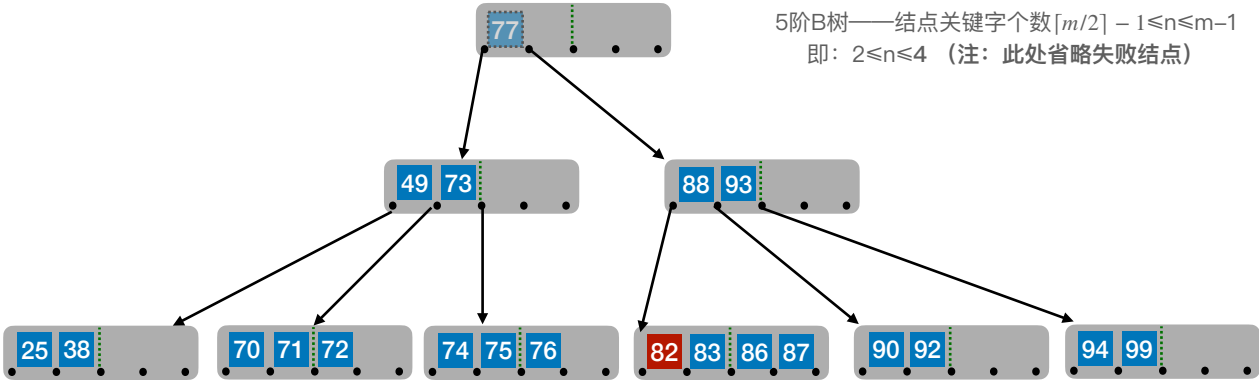
B树的删除



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

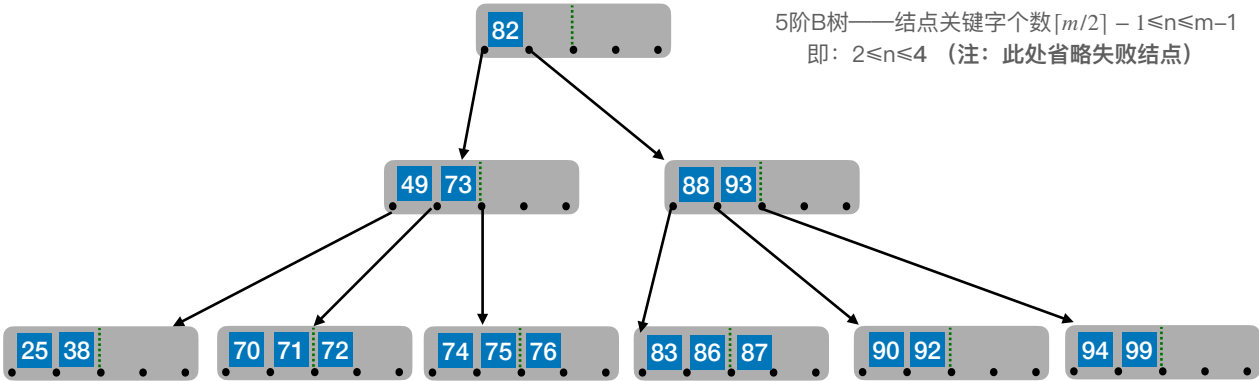
若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字
直接前驱：当前关键字**左侧**指针所指子树中“最右下”的元素

B树的删除



若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字
直接前驱：当前关键字**左侧**指针所指子树中“**最右下**”的元素
直接后继：当前关键字**右侧**指针所指子树中“**最左下**”的元素

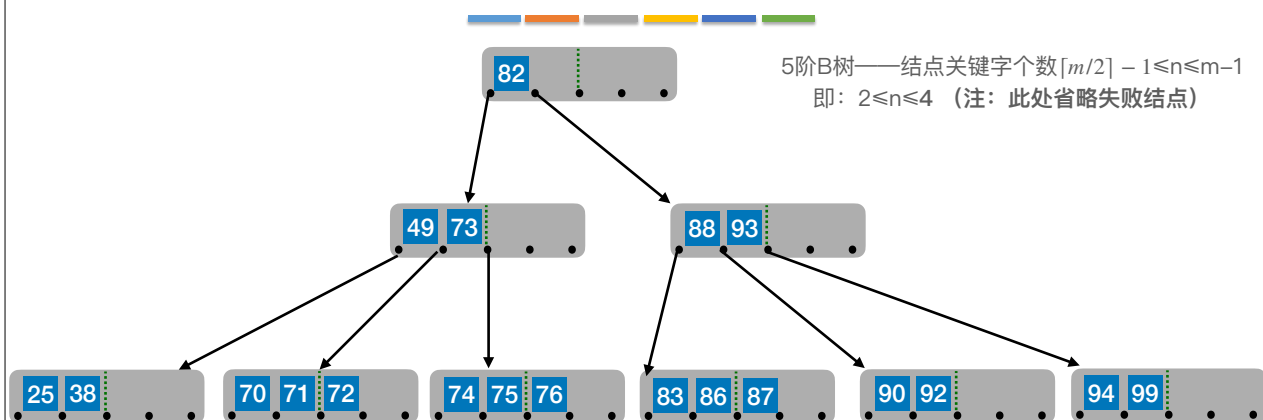
B树的删除



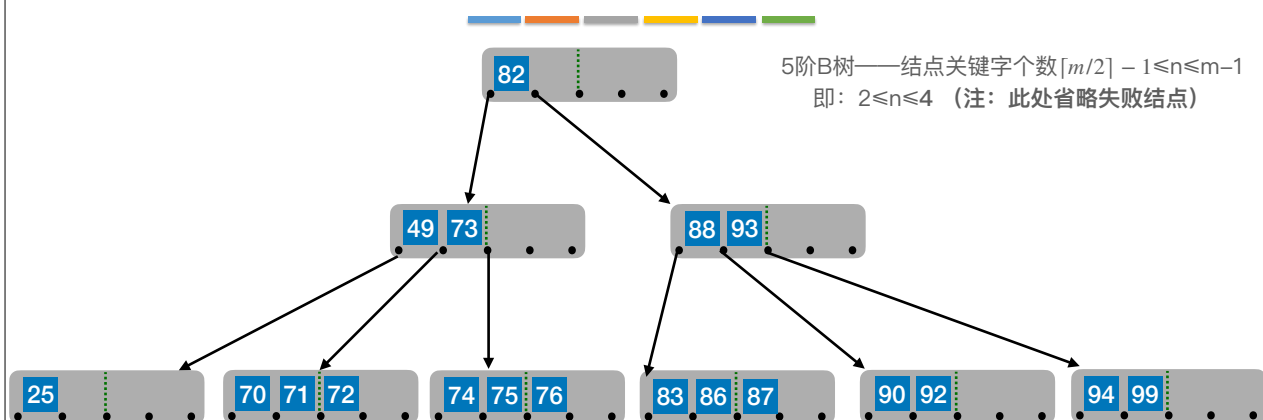
若被删除关键字在**非终端节点**，则用直接前驱或直接后继来替代被删除的关键字
直接前驱：当前关键字**左侧**指针所指子树中“**最右下**”的元素
直接后继：当前关键字**右侧**指针所指子树中“**最左下**”的元素

对非终端结点关键字的删除，必然可以转化为对终端结点的删除操作

B树的删除

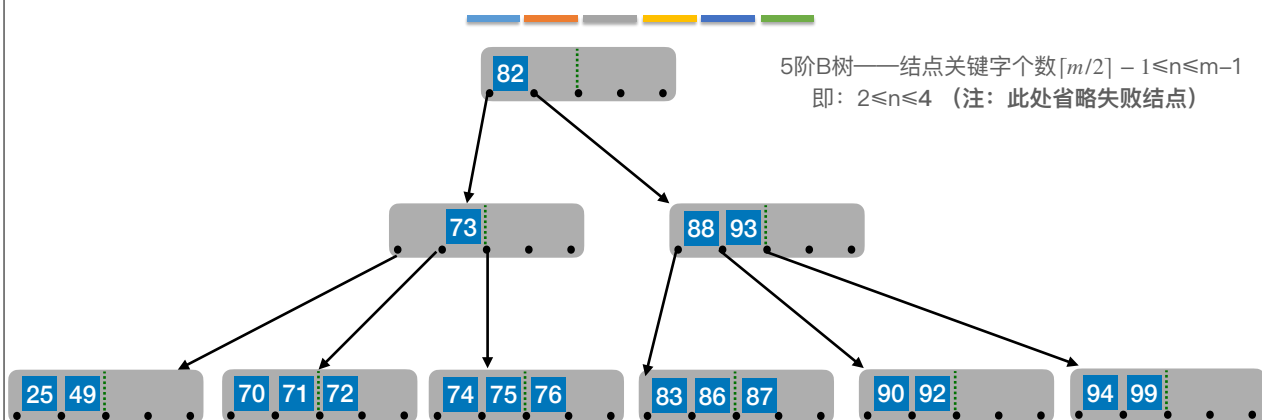


B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

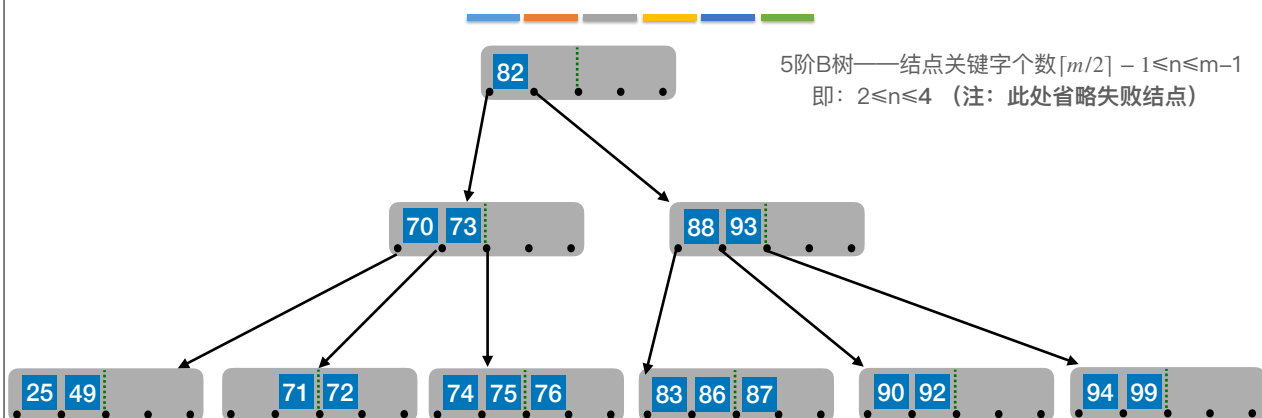
B树的删除



在？借点钱？

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

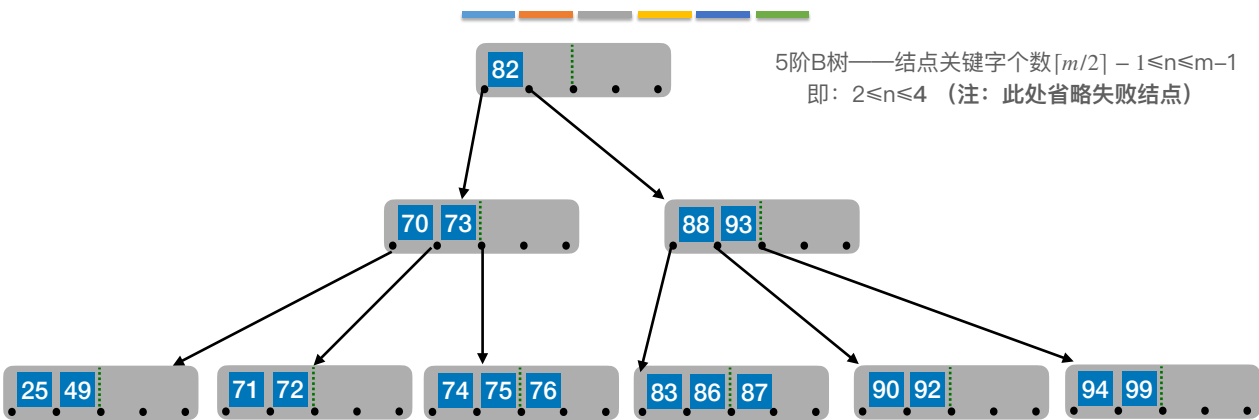
B树的删除



在？借点钱？

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

B树的删除



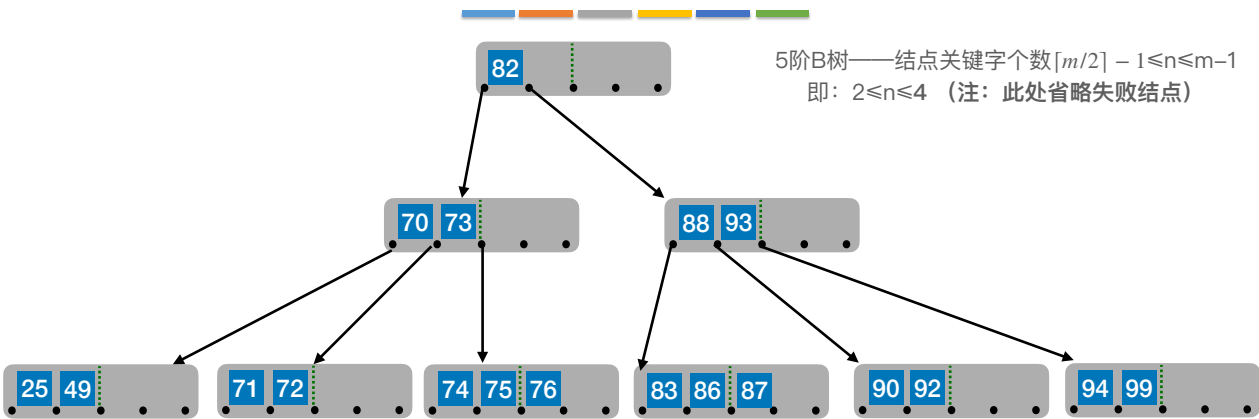
5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）



在？借点钱？

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）
说白了，当**右兄弟很宽裕**时，用当前结点的**后继**、**后继的后继**来填补空缺

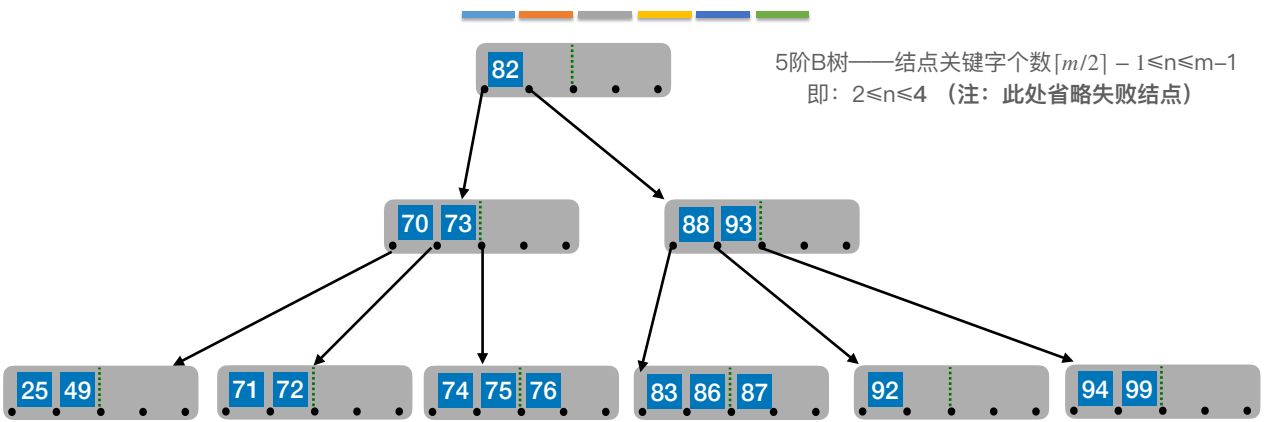
B树的删除



5阶B树——结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
即： $2 \leq n \leq 4$ （注：此处省略失败结点）

兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

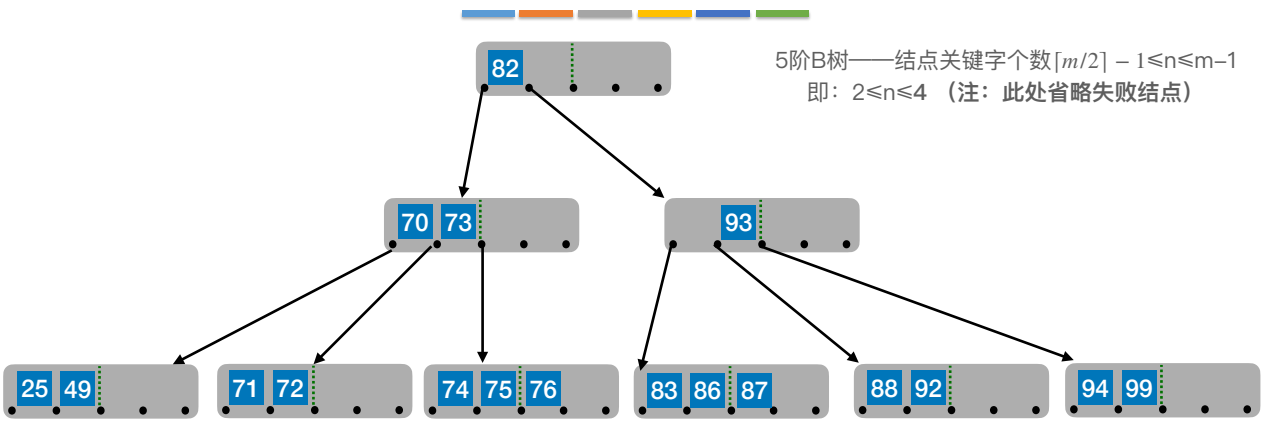
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

当左兄弟很宽裕时，用当前结点的前驱、前驱的前驱来填补空缺

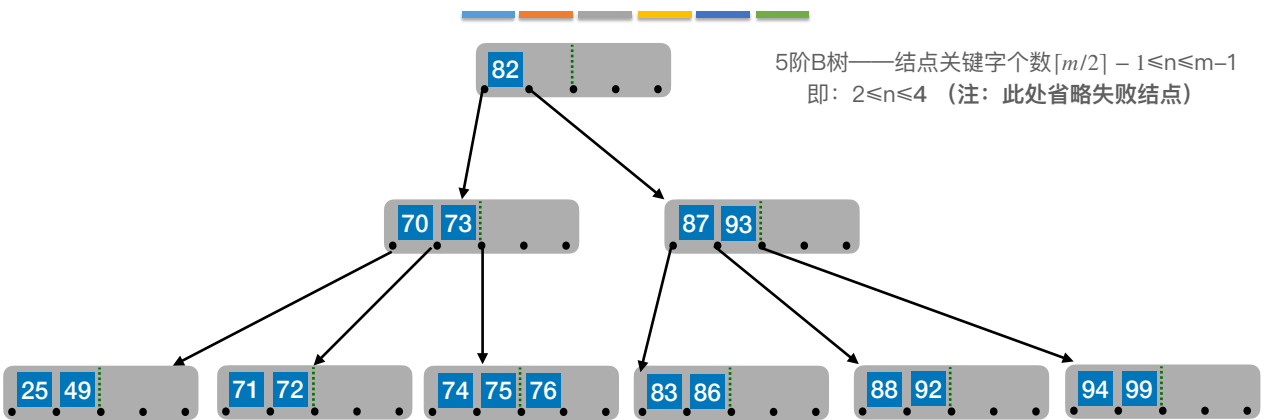
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

当左兄弟很宽裕时，用当前结点的前驱、前驱的前驱来填补空缺

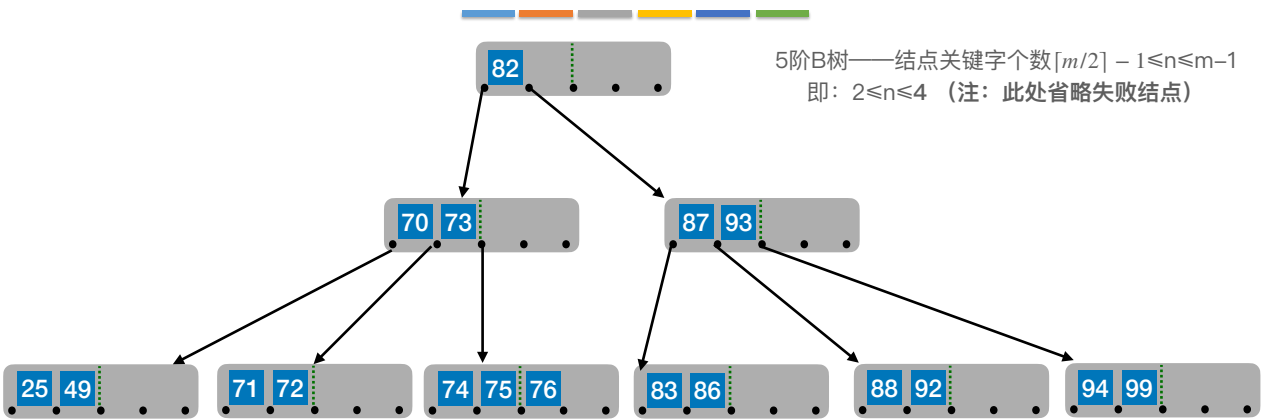
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

当左兄弟很宽裕时，用当前结点的前驱、前驱的前驱来填补空缺

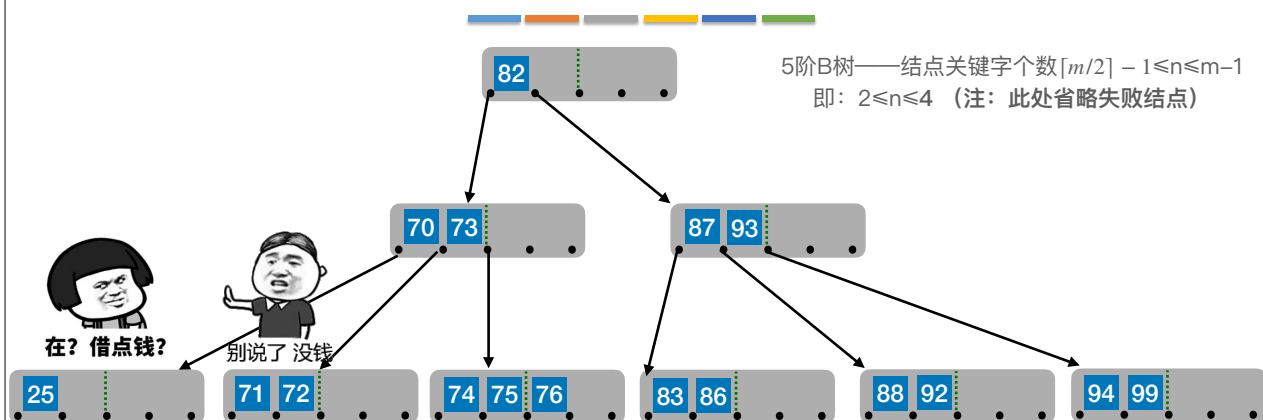
B树的删除



兄弟够借。若被删除关键字所在结点删除前的关键字个数低于下限，且与此结点右（或左）兄弟结点的关键字个数还很宽裕，则需要调整该结点、右（或左）兄弟结点及其双亲结点（父子换位法）

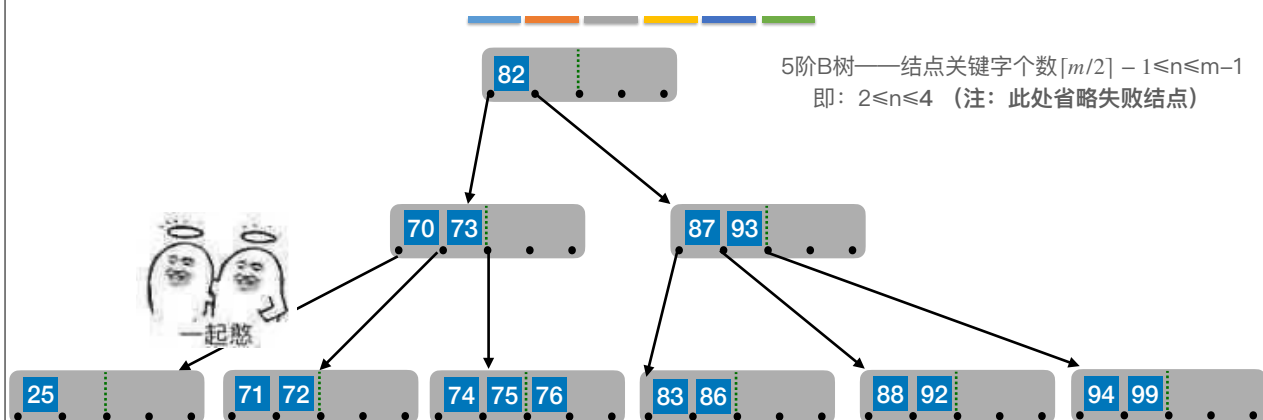
本质：要永远保证 子树0<关键字1<子树1<关键字2<子树2<....

B树的删除

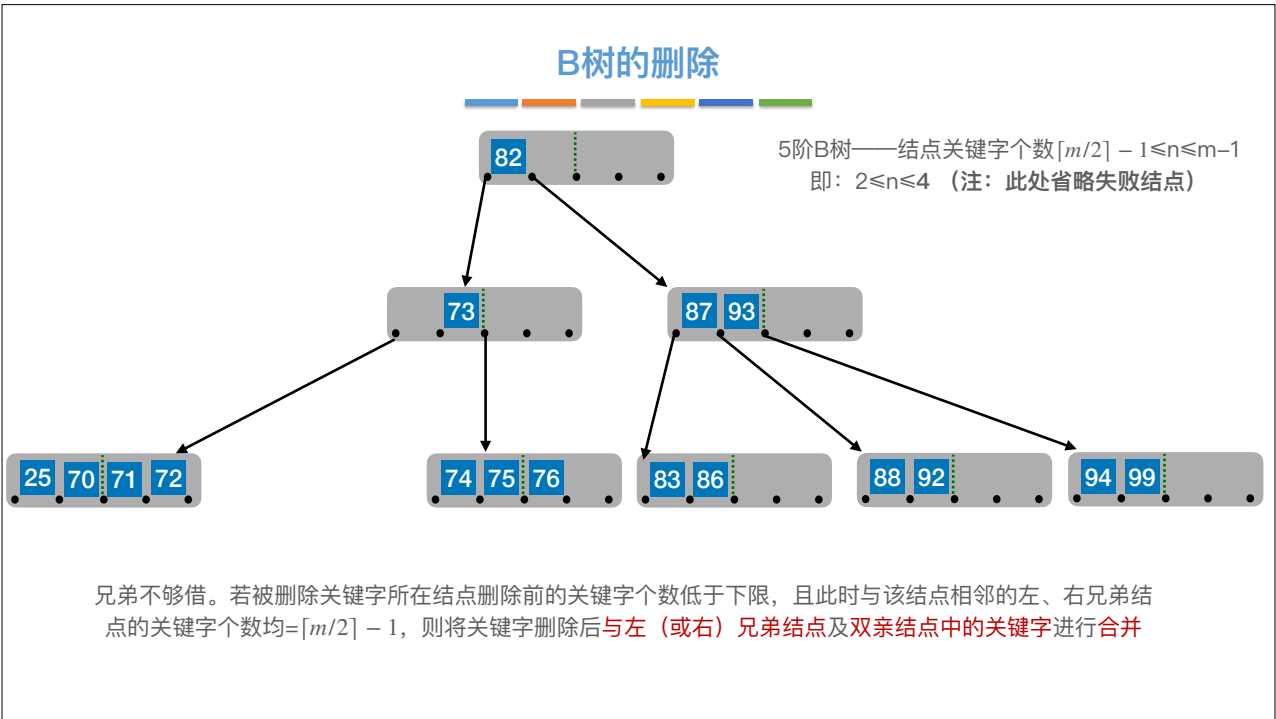
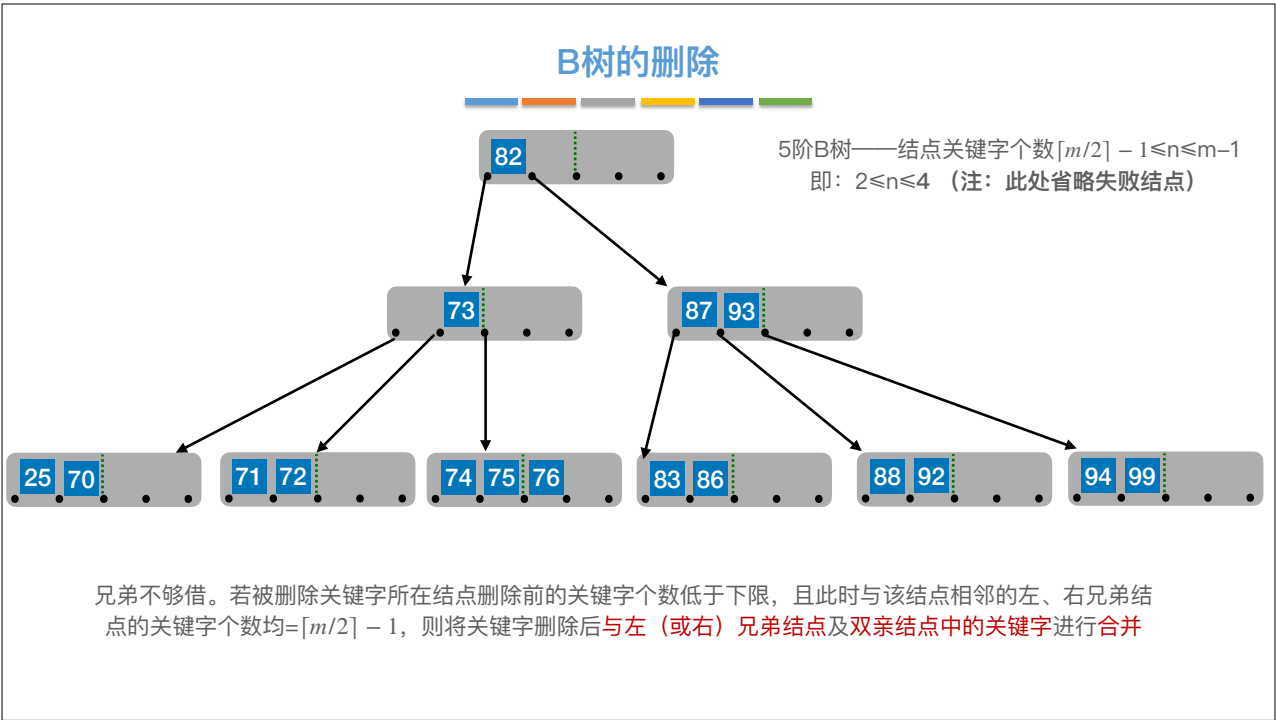


兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均为 $\lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

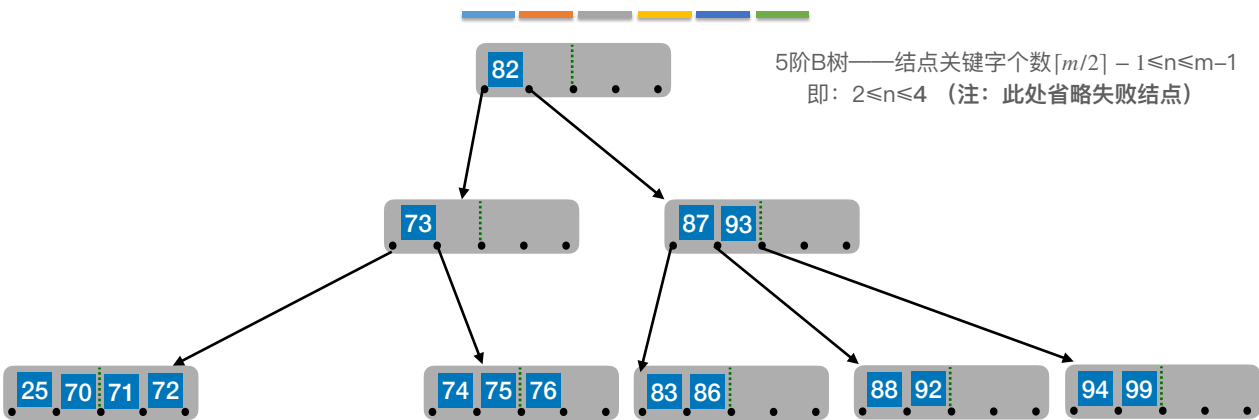
B树的删除



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均为 $\lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

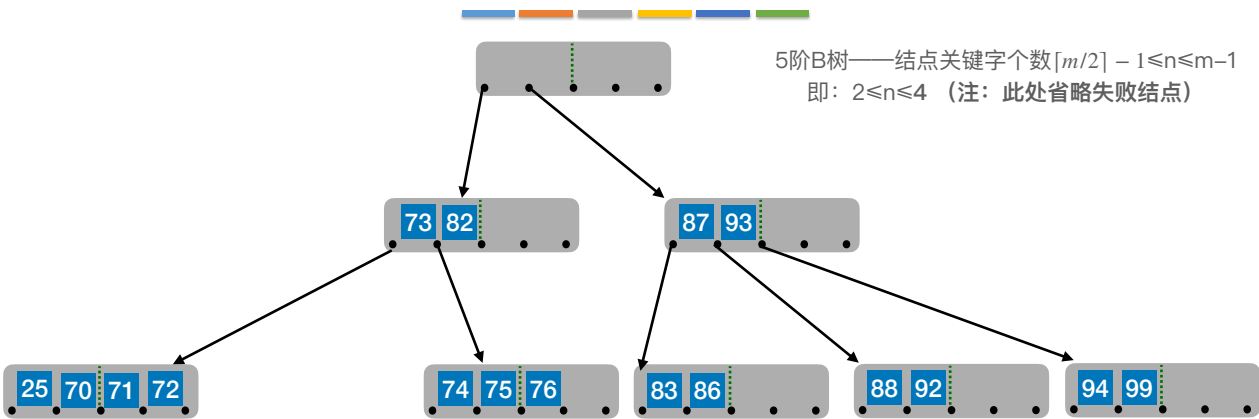


B树的删除

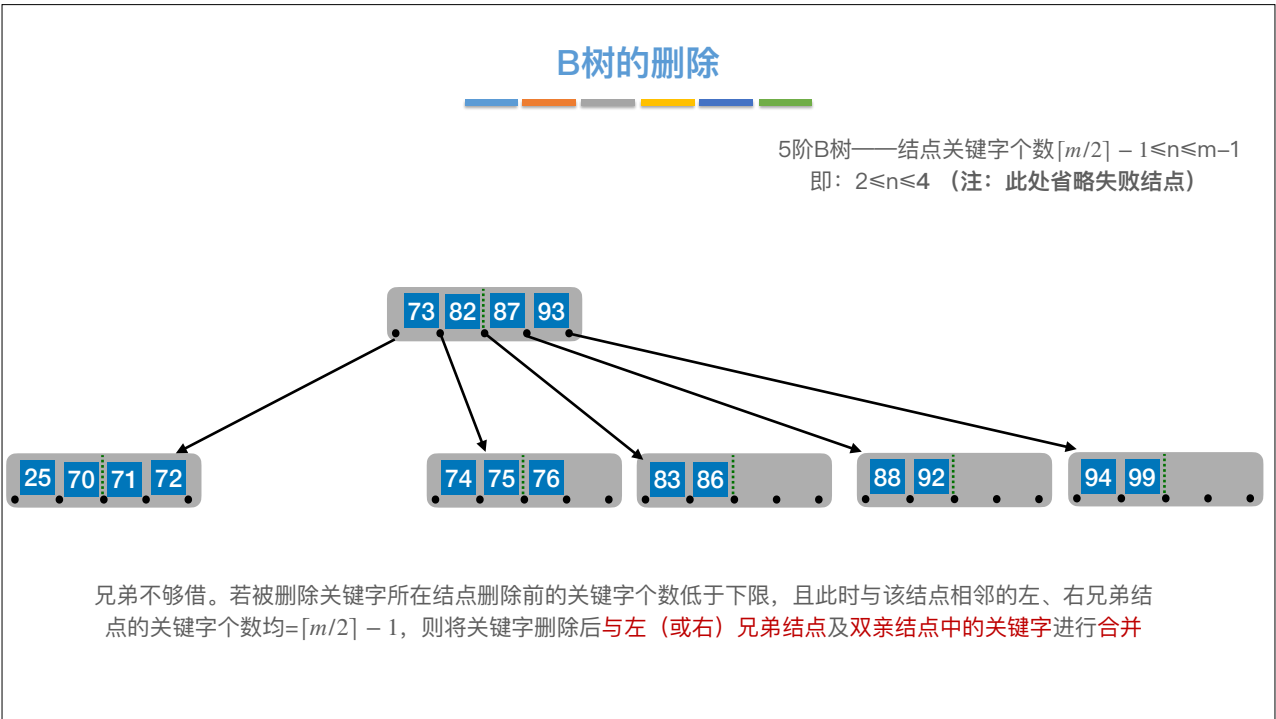
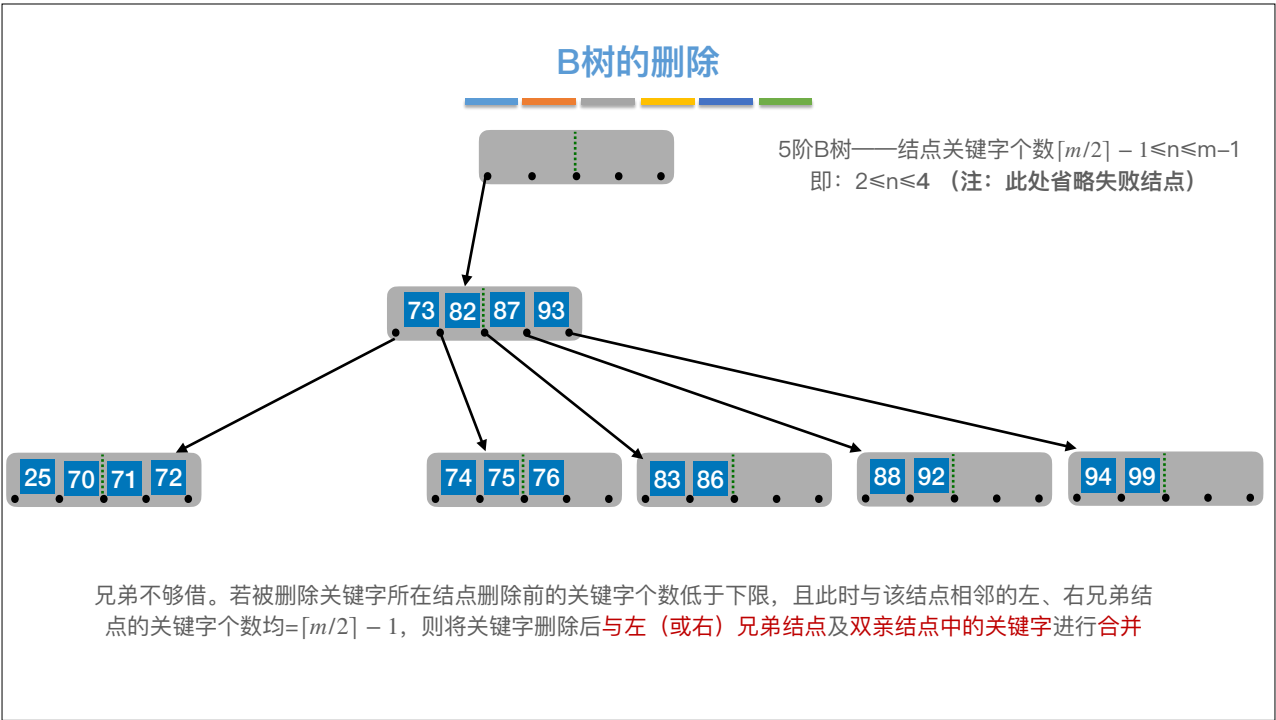


兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

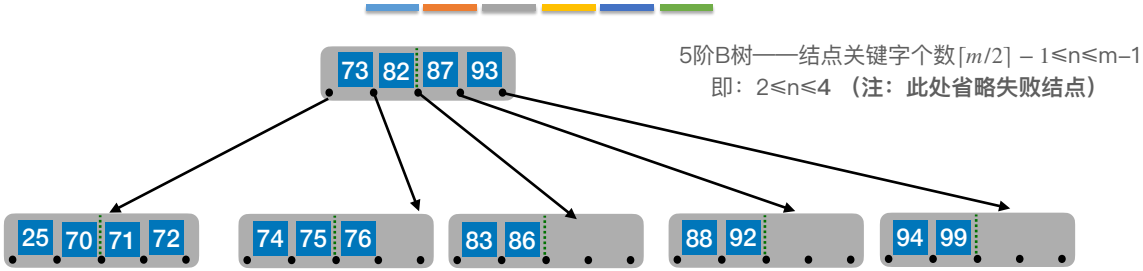
B树的删除



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并



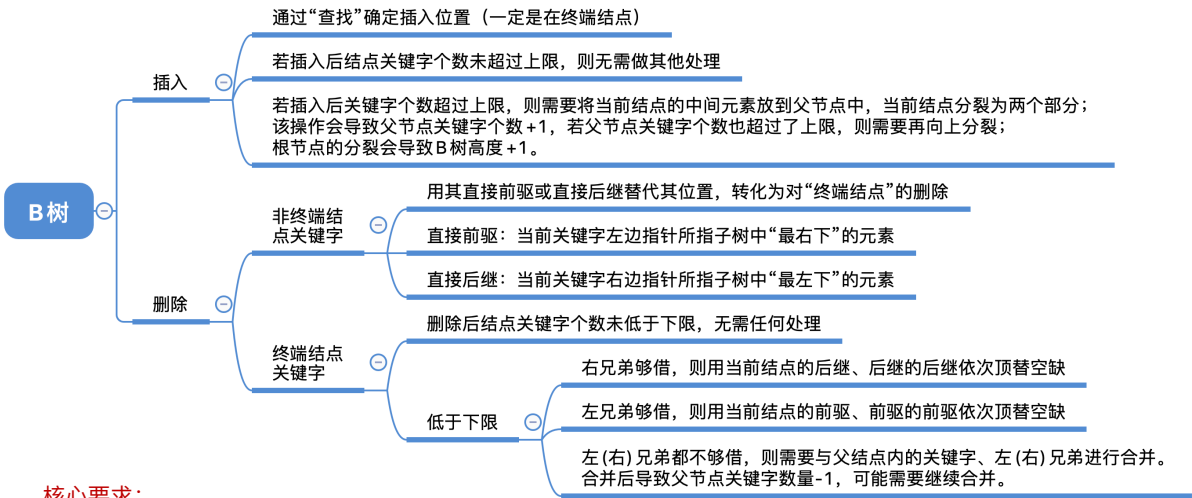
B树的删除



兄弟不够借。若被删除关键字所在结点删除前的关键字个数低于下限，且此时与该结点相邻的左、右兄弟结点的关键字个数均 $= \lceil m/2 \rceil - 1$ ，则将关键字删除后与左（或右）兄弟结点及双亲结点中的关键字进行合并

在合并过程中，双亲结点中的关键字个数会减1。若其双亲结点是根结点且关键字个数减少至0（根结点关键字个数为1时，有2棵子树），则直接将根结点删除，合并后的新结点成为根；若双亲结点不是根结点，且关键字个数减少到 $\lceil m/2 \rceil - 2$ ，则又要与它自己的兄弟结点进行调整或合并操作，并重复上述步骤，直至符合B树的要求为止。

知识回顾与重要考点



核心要求：

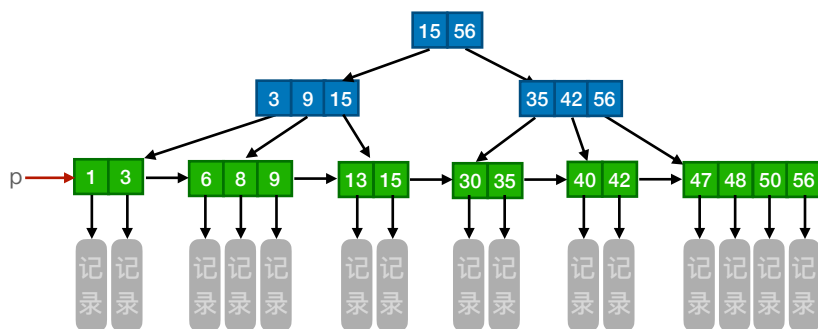
- ①对m阶B树——除根节点外，结点关键字个数 $\lceil m/2 \rceil - 1 \leq n \leq m-1$
- ②子树0<关键字1<子树1<关键字2<子树2<....

本节内容

B+树

王道考研/CSKAOYAN.COM

B+树

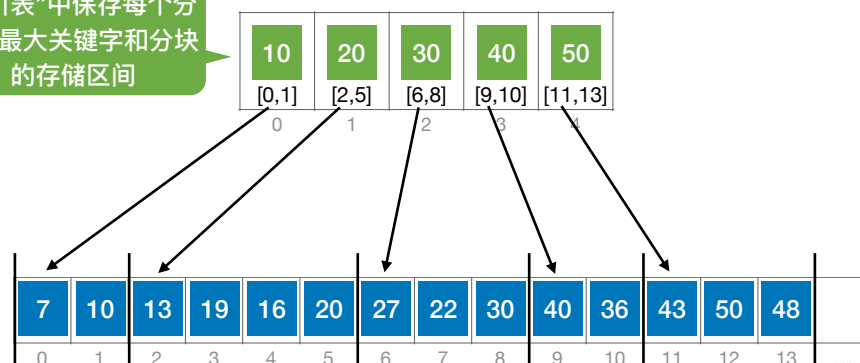


注: 以上是一棵4阶B+树

王道考研/CSKAOYAN.COM

对比：分块查找

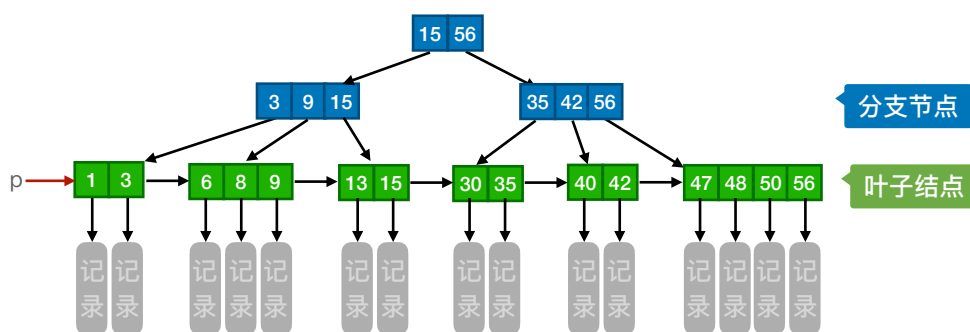
“索引表”中保存每个分块的最大关键字和分块的存储区间



特点：块内无序、块间有序

王道考研/CSKAOYAN.COM

B+树



注：以上是一棵4阶B+树

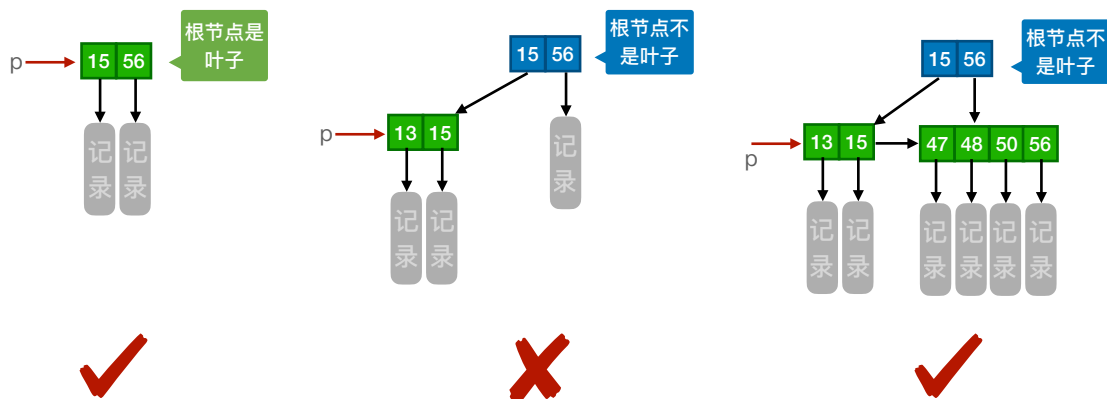
一棵m阶的B+树需满足下列条件：

- 1) 每个分支结点最多有m棵子树（孩子结点）。
- 2) 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。
- 3) 结点的子树个数与关键字个数相等。
- 4) 所有叶结点包含全部关键字及指向相应记录的指针，叶结点中将关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接起来。
- 5) 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针。

支持顺序查找

王道考研/CSKAOYAN.COM

B+树

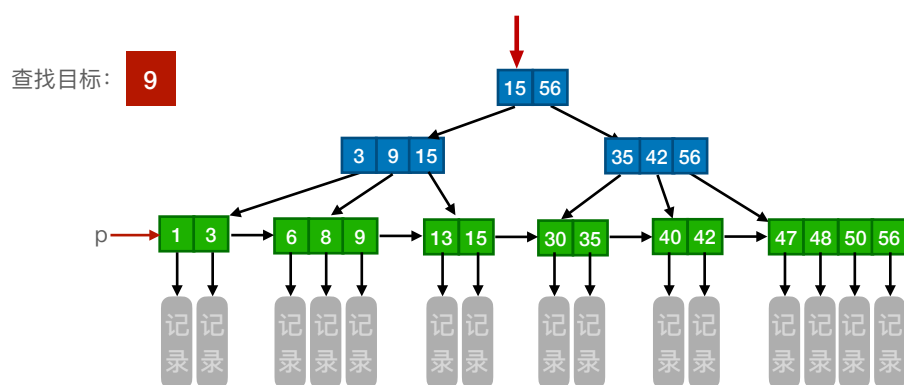


可以理解为: 要追求“绝对平衡”, 即所有子树高度要相同

2) 非叶根结点至少有两棵子树, 其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树。

王道考研/CSKAOYAN.COM

B+树的查找

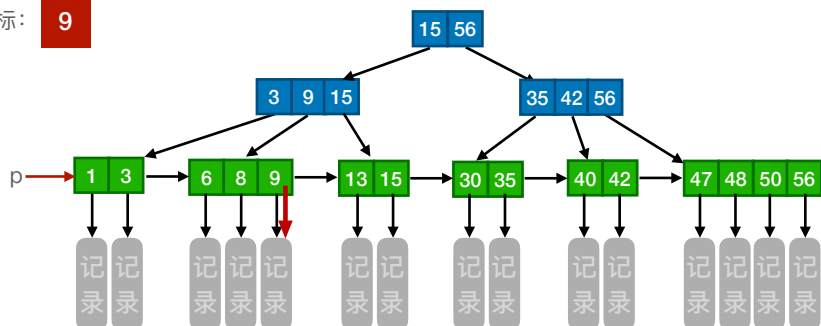


多路查找

王道考研/CSKAOYAN.COM

B+树的查找

查找目标: 9

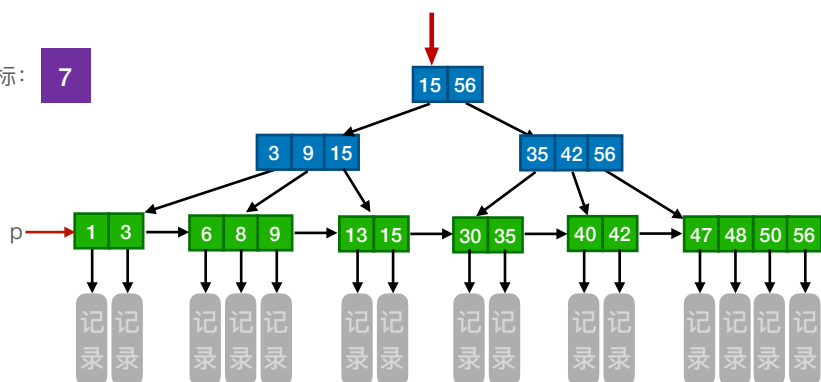


多路查找

王道考研/CSKAOYAN.COM

B+树的查找

查找目标: 7

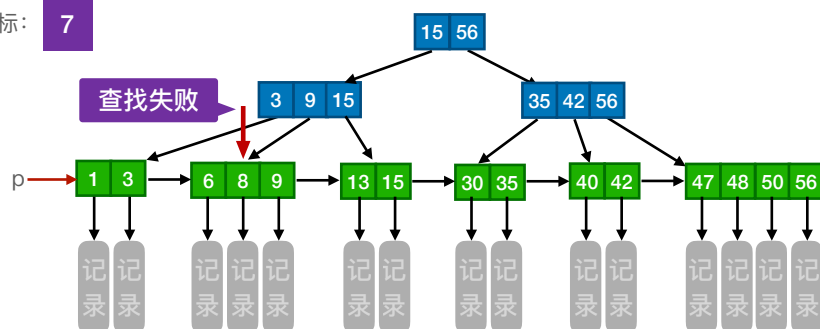


多路查找

王道考研/CSKAOYAN.COM

B+树的查找

查找目标：7



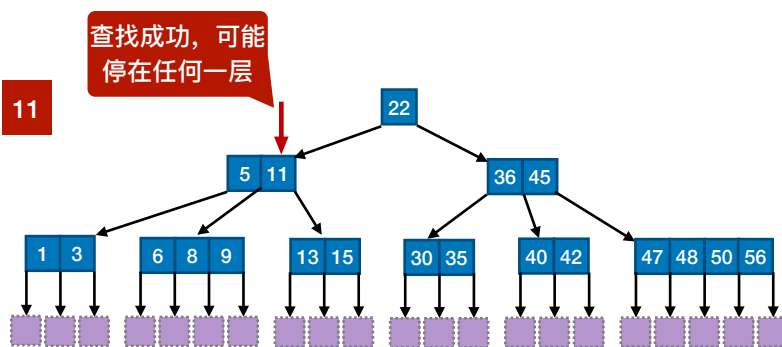
多路查找

B+树中，无论查找成功与否，最终一定都要走到最下面一层结点

王道考研/CSKAOYAN.COM

对比：B树的查找

查找目标：11

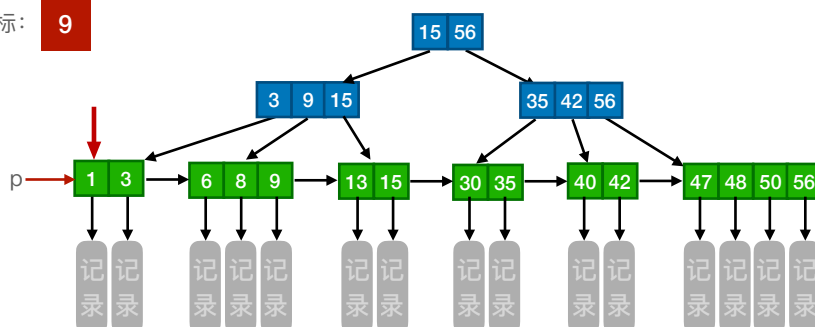


注：以上是一棵5阶B树

王道考研/CSKAOYAN.COM

B+树的查找

查找目标：9

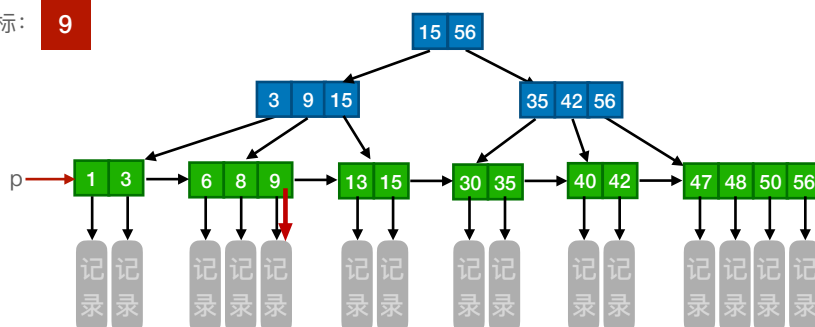


顺序查找

王道考研/CSKAOYAN.COM

B+树的查找

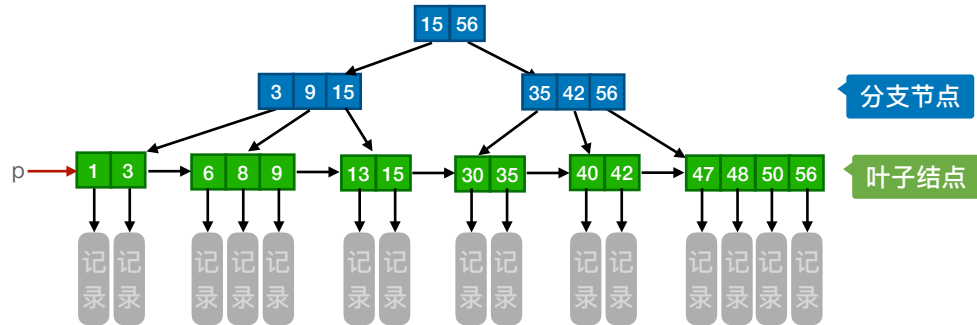
查找目标：9



顺序查找

王道考研/CSKAOYAN.COM

B+树 VS B树



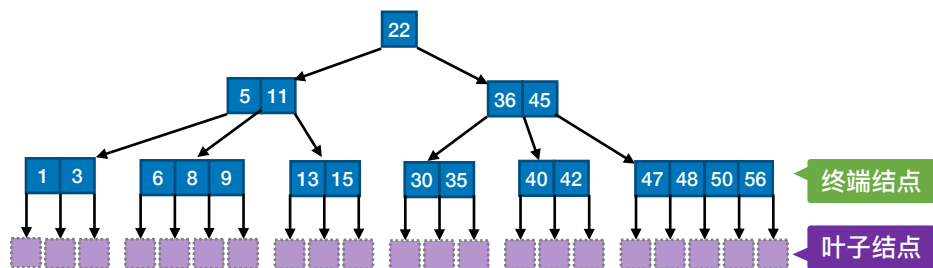
注：以上是一棵4阶B+树

m阶B+树：

- 1) 结点中的n个关键字对应n棵子树

王道考研/CSKAOYAN.COM

B+树 VS B树



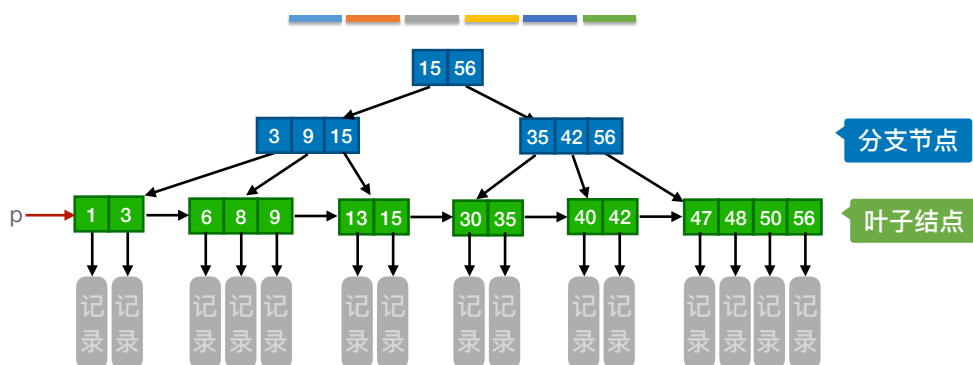
注：以上是一棵5阶B树

m阶B树：

- 1) 结点中的n个关键字对应n+1棵子树

王道考研/CSKAOYAN.COM

B+树 VS B树



注：以上是一棵4阶B+树

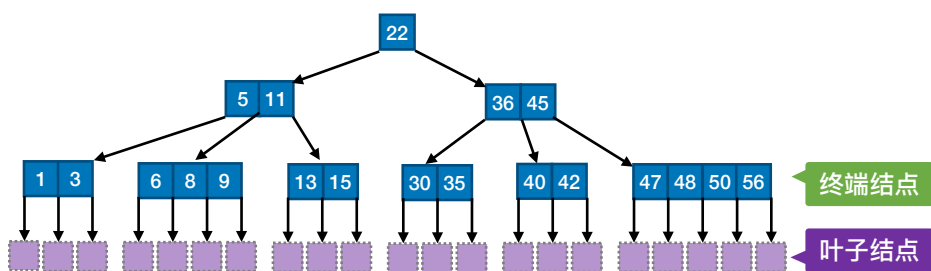
m阶B+树：

2) 根节点的关键字数 $n \in [1, m]$

其他节点的关键字数 $n \in [\lceil m/2 \rceil, m]$

王道考研/CSKAOYAN.COM

B+树 VS B树



注：以上是一棵5阶B树

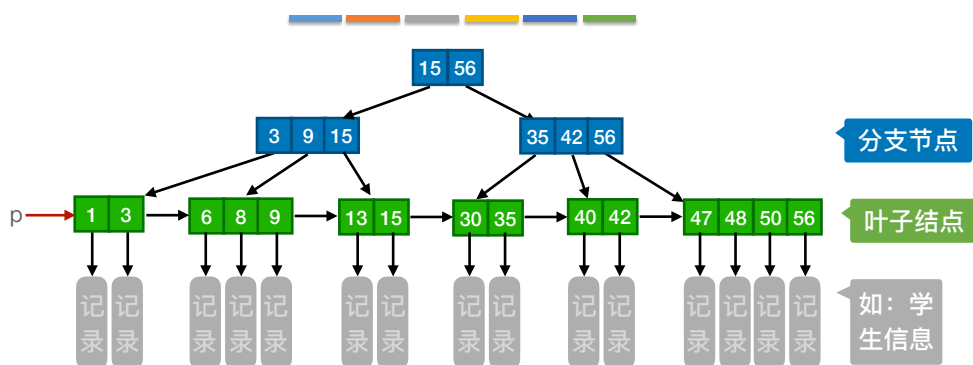
m阶B树：

2) 根节点的关键字数 $n \in [1, m-1]$ 。

其他节点的关键字数 $n \in [\lceil m/2 \rceil - 1, m-1]$

王道考研/CSKAOYAN.COM

B+树 VS B树



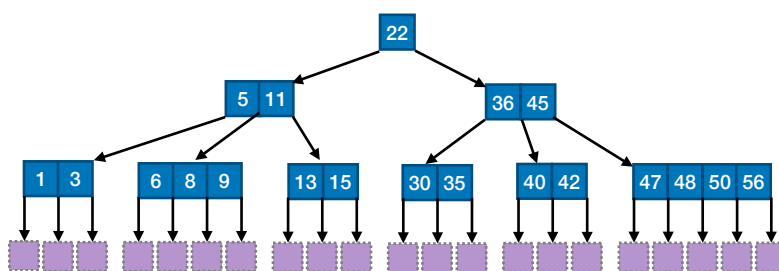
注：以上是一棵4阶B+树

m阶B+树：

3) 在B+树中，叶结点包含全部关键字，非叶结点中出现过的关键字也会出现在叶结点中

王道考研/CSKAOYAN.COM

B+树 VS B树



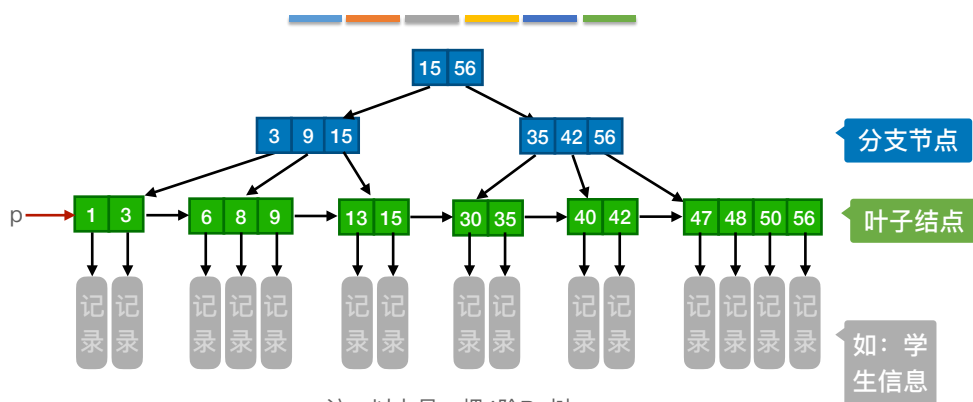
注：以上是一棵5阶B树

m阶B树：

3) 在B树中，各结点中包含的关键字是不重复的

王道考研/CSKAOYAN.COM

B+树 VS B树

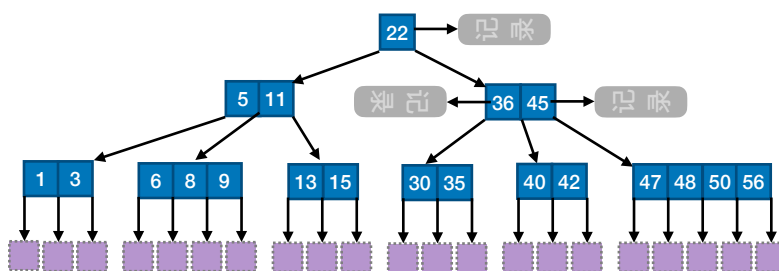


m阶B+树：

4) 在B+树中，叶结点包含信息，所有非叶结点仅起索引作用，非叶结点中的每个索引项只含有对应子树的最大关键字和指向该子树的指针，不含有该关键字对应记录的存储地址。

王道考研/CSKAOYAN.COM

B+树 VS B树

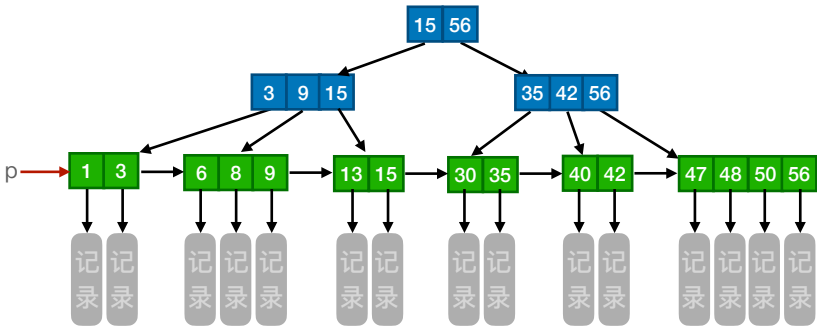


m阶B树：

4) B树的结点中都包含了关键字对应的记录的存储地址

王道考研/CSKAOYAN.COM

B+树 VS B树



典型应用：关系型数据库的“索引”（如MySQL）

在B+树中，非叶结点不含有该关键字对应记录的存储地址。
可以使一个磁盘块可以包含更多个关键字，使得B+树的阶更大，树高更矮，
读磁盘次数更少，查找更快



WHY?

王道考研/CSKAOYAN.COM

知识回顾与重要考点

	m阶B树	m阶B+树
类比	二叉查找树的进化——>m叉查找树	分块查找的进化——>多级分块查找
关键字与分叉	n个关键字对应n+1个分叉（子树）	n个关键字对应n个分叉
结点包含的信息	所有结点中都包含记录的信息	只有最下层叶子结点才包含记录的信息（可使树更矮）
查找方式	不支持顺序查找。查找成功时，可能停在任何一层结点，查找速度“不稳定”	支持顺序查找。查找成功或失败都会到达最下一层结点，查找速度“稳定”
相同点	除根节点外，最少 $\lceil m/2 \rceil$ 个分叉（确保结点不要太“空”） 任何一个结点的子树都要一样高（确保“绝对平衡”）	

王道考研/CSKAOYAN.COM

本节内容

散列查找

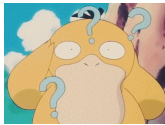
王道考研/CSKAOYAN.COM

散列表 (Hash Table)

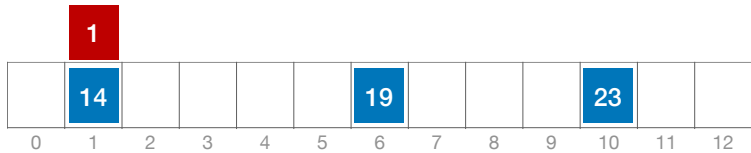
散列表 (Hash Table)，又称**哈希表**。是一种数据结构，特点是：数据元素的**关键字**与其**存储地址**直接相关

如何建立“关键字”与“存储地址”的联系？

通过“散列函数（哈希函数）”： $Addr=H(key)$



例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



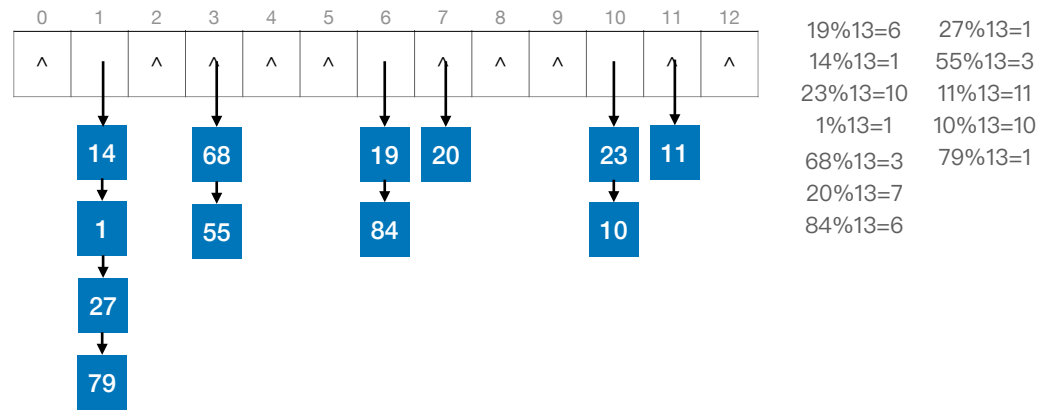
$19\%13=6$
 $14\%13=1$
 $23\%13=10$
 $1\%13=1$

若不同的关键字通过散列函数映射到同一个值，则称它们为“**同义词**”
通过散列函数确定的位置已经存放了其他元素，则称这种情况为“**冲突**”

王道考研/CSKAOYAN.COM

处理冲突的方法——拉链法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$

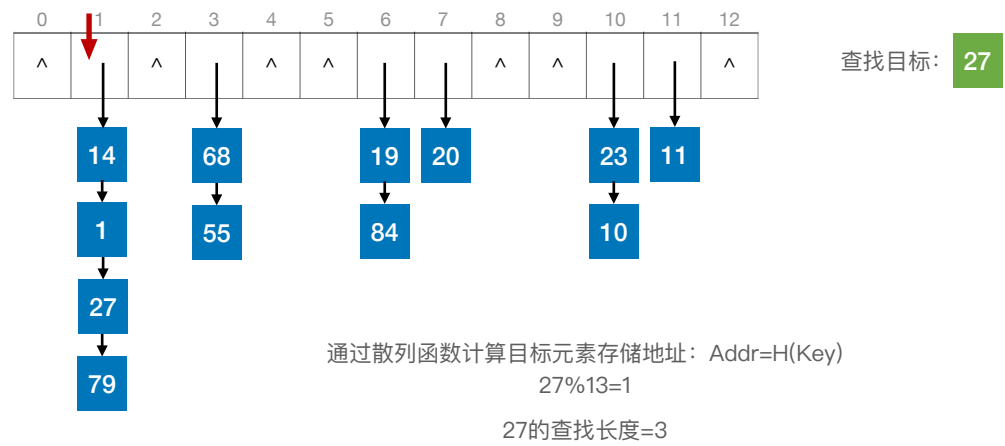


用**拉链法**（又称链接法、链地址法）处理“冲突”：把所有“同义词”存储在一个链表中

王道考研/CSKAOYAN.COM

散列查找

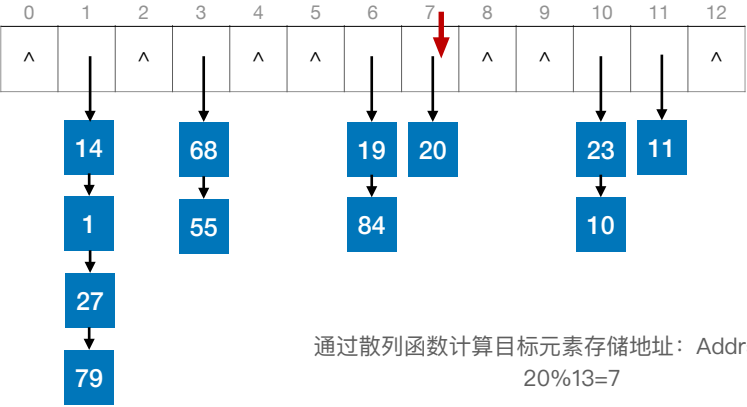
例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



王道考研/CSKAOYAN.COM

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



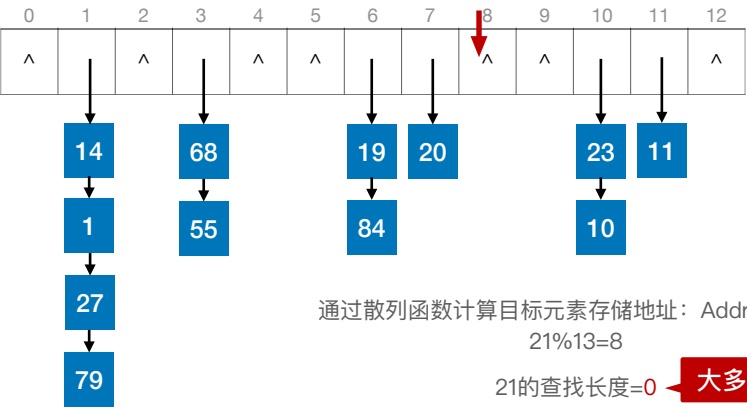
查找目标：20

通过散列函数计算目标元素存储地址： $Addr=H(Key)$
 $20\%13=7$
20的查找长度=1

王道考研/CSKAOYAN.COM

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



查找目标：21

通过散列函数计算目标元素存储地址： $Addr=H(Key)$
 $21\%13=8$

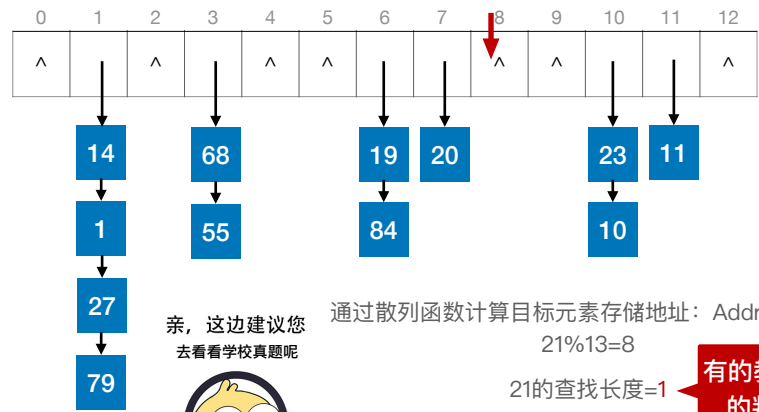
21的查找长度=0 大多数学校的计算方法

查找长度——在查找运算中，需要对比关键字的次数称为查找长度

王道考研/CSKAOYAN.COM

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



查找目标：21

亲，这边建议您去看看学校真题呢



通过散列函数计算目标元素存储地址：Addr=H(Key)

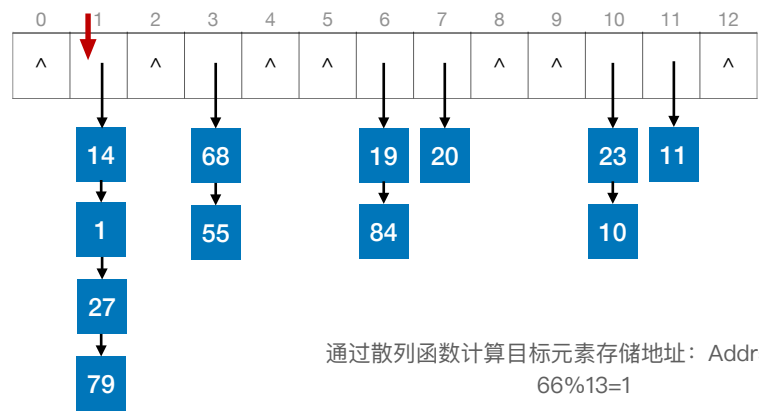
$$21\%13=8$$

21的查找长度=1

有的教材也会把“空指针”的判定算作一次比较

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



查找目标：66

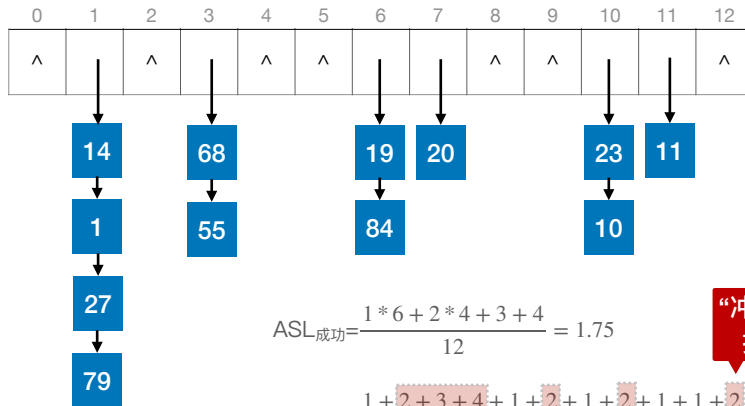
通过散列函数计算目标元素存储地址：Addr=H(Key)

$$66\%13=1$$

66的查找长度=4

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key) = key \% 13$



$$ASL_{成功} = \frac{1 * 6 + 2 * 4 + 3 + 4}{12} = 1.75$$

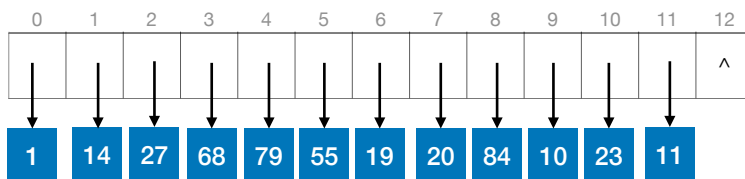
$$ASL_{成功} = \frac{1 + 2 + 3 + 4 + 1 + 1 + 2 + 1 + 1 + 2 + 1 + 1 + 2 + 1}{12} = 1.75$$

“冲突”越多，查找效率越低

王道考研/CSKAOYAN.COM

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key) = key \% 13$



最理想情况：散列查找时间复杂度可到达 $O(1)$

$$ASL_{成功} = \frac{1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1}{12} = 1$$



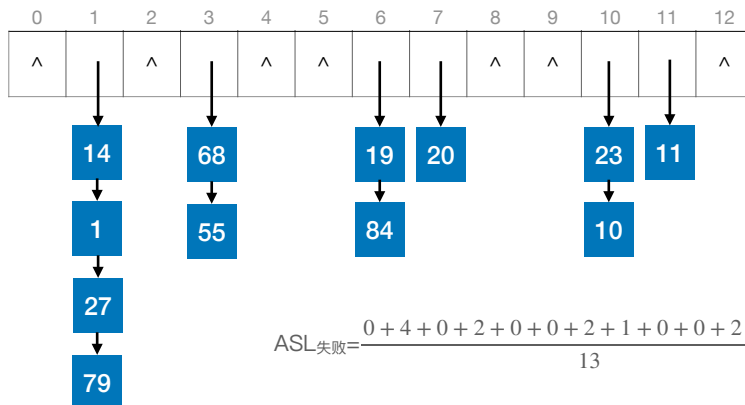
欲言又止 稍加思考

如何设计冲突更少的散列函数？

王道考研/CSKAOYAN.COM

散列查找

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



$$ASL_{失败} = \frac{0+4+0+2+0+0+2+1+0+0+2+1+0}{13} = 0.92$$

装填因子

装填因子 α = 表中记录数 / 散列表长度

装填因子会直接影响散列表的查找效率

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

除留余数法 —— $H(key) = key \% p$

散列表表长为 m ，取一个不大于 m 但最接近或等于 m 的质数 p

质数又称素数。指除了1和此整数自身外,不能被其他自然数整除的数

例：散列表表长13，散列函数 $H(key)=key\%13$

0	1	2	3	4	5	6	7	8	9	10	11	12

例：散列表表长15，散列函数 $H(key)=key\%13$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
													^	^

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

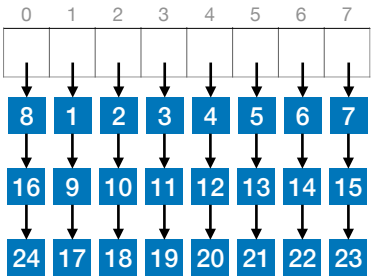
除留余数法 —— $H(key) = key \% p$

散列表表长为m，取一个不大于m但最接近或等于m的质数p

质数又称素数。指除了1和此整数自身外,不能被其他自然数整除的数

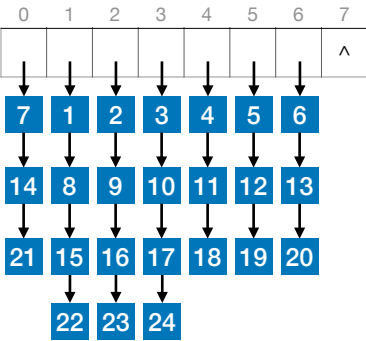
设：可能出现的关键字={1,2,3,4,5,6,7,8,9,10.....}

散列表表长8，散列函数 $H(key)=key\%8$



小朋友你是否有很多问号

散列表表长8，散列函数 $H(key)=key\%7$



王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

除留余数法 —— $H(key) = key \% p$

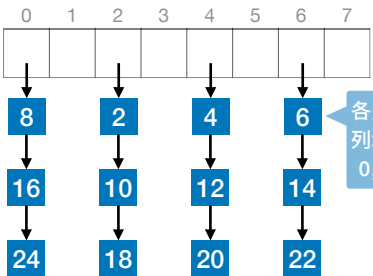
散列表表长为m，取一个不大于m但最接近或等于m的质数p

质数又称素数。指除了1和此整数自身外,不能被其他自然数整除的数

设：可能出现的关键字={2,4,6,8,10,12.....}

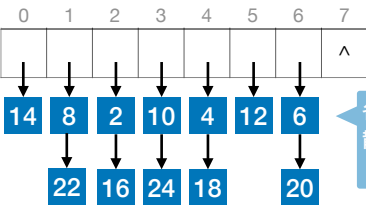
如：中国人更喜欢双数，车牌照双数更多

散列表表长8，散列函数 $H(key)=key\%8$



各关键字的散列地址集中在0, 2, 4, 6

散列表表长8，散列函数 $H(key)=key\%7$



各关键字的散列地址分布均匀

Why? ——用质数取模，分布更均匀，冲突更少。参见《数论》

Tips：散列函数的设计要结合实际的关键字分布特点来考虑，不要教条化

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

直接定址法 —— $H(key) = key$ 或 $H(key) = a*key + b$

其中，a和b是常数。这种方法计算最简单，且不会产生冲突。它适合关键字的分布基本连续的情况，若关键字分布不连续，空位较多，则会造成存储空间的浪费。

例：存储同一个班级的学生信息，班内学生学号为(1120112176~1120112205)
 $H(key) = key - 1120112176$

0	1	2	3	4	5	...	...	...	26	27	28	29
...176	...177	...178	...179	...180	...181	...	...	...	...202	...203	...204	...205

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

数字分析法 —— 选取数码分布较为均匀的若干位作为散列地址

设关键字是r进制数（如十进制数），而r个数码在各位上出现的频率不一定相同，可能在某些位上分布均匀一些，每种数码出现的机会均等；而在某些位上分布不均匀，只有某几种数码经常出现，此时可选取数码分布较为均匀的若干位作为散列地址。这种方法适合于已知的关键字集合，若更换了关键字，则需要重新构造新的散列函数。

例：以“手机号码”作为关键字设计散列函数

138XXX2875
138XXX1682
138XXX9125
.....
199XXX1684
199XXX1236

设计长度为10000的散列表，以手机号后四位作为散列地址

0	1	2	3	...	...	9998	9999

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

平方取中法——取关键字的平方值的中间几位作为散列地址。

具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀，适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数。

$$\begin{aligned} 1310^2 &= 1,716,100 \\ 1110^2 &= 1,232,100 \\ 1300^2 &= 1,690,000 \\ 1210^2 &= 1,464,100 \\ 1200^2 &= 1,440,000 \end{aligned}$$

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

平方取中法——取关键字的平方值的中间几位作为散列地址。

具体取多少位要视实际情况而定。这种方法得到的散列地址与关键字的每位都有关系，因此使得散列地址分布比较均匀，适用于关键字的每位取值都不够均匀或均小于散列地址所需的位数。

例：要存储整个学校的学生信息，以“身份证号”作为关键字设计散列函数



身份证号码规则：
前1、2位数字表示：所在省份的代码；
第3、4位数字表示：所在城市的代码；
第5、6位数字表示：所在区县的代码；
第7-14位数字表示：出生年、月、日；
第15、16位数字表示：所在地的派出所的代码；
第17位数字表示性别：奇数表示男性，偶数表示女性；
第18位数字是校检码。

0	1	2	3	...	...	...	99999

假设学生不超过十万人，可身份证号平方取中间5位

王道考研/CSKAOYAN.COM

常见的散列函数

设计目标——让不同关键字的冲突尽可能地少

例：要存储整个学校的学生信息，以“身份证号”作为关键字设计散列函数

0	1	2	3	...	...	...	999999
							999999
							999999



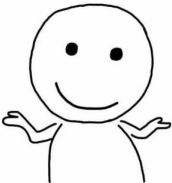
若散列表的长度为10000000000000000000（别数了，有18个0 🐶）

则可以直接用身份证号作为散列地址，且不可能有冲突，查找时间复杂度为 $O(1)$

散列查找是典型的“用空间换时间”的算法，只要散列函数设计的合理，则散列表越长，冲突的概率越低。

知识回顾与重要考点

下次一定



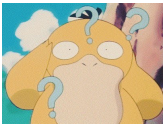
本节内容

散列查找

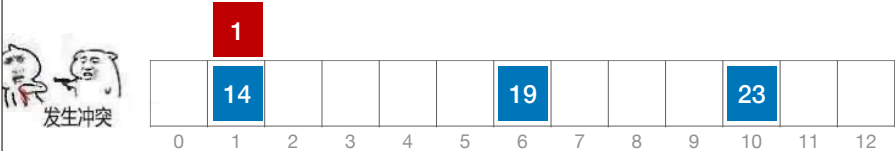
前情回顾

如何建立“关键字”与“存储地址”的联系？

通过“散列函数（哈希函数）”： $Addr=H(key)$



例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$

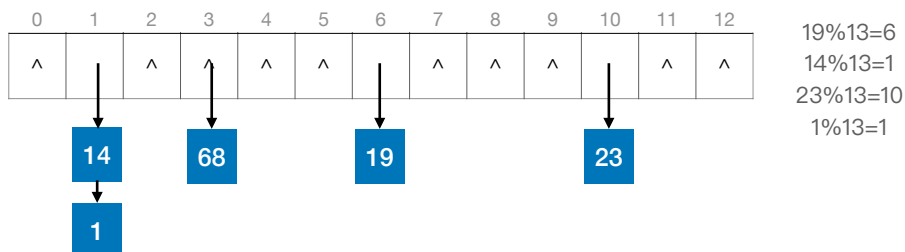


$19\%13=6$
 $14\%13=1$
 $23\%13=10$
 $1\%13=1$

若不同的关键字通过散列函数映射到同一个值，则称它们为“**同义词**”
通过散列函数确定的位置已经存放了其他元素，则称这种情况为“**冲突**”

处理冲突的方法——拉链法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$

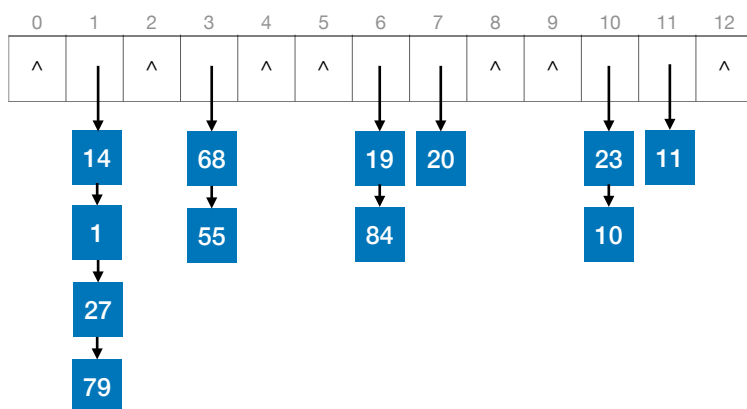


用**拉链法**（又称链接法、链地址法）处理“冲突”：把所有“同义词”存储在一个链表中

王道考研/CSKAOYAN.COM

处理冲突的方法——拉链法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



用**拉链法**（又称链接法、链地址法）处理“冲突”：把所有“同义词”存储在一个链表中

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法

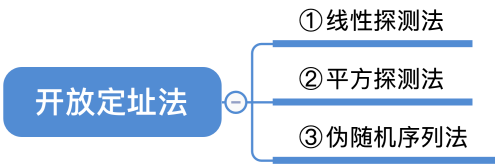
例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

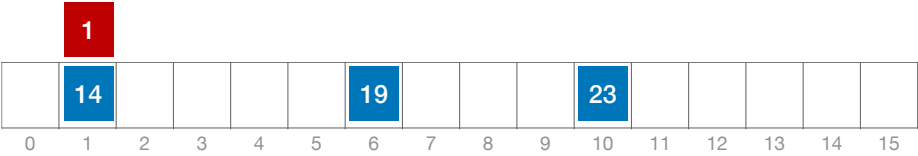
$$H_i = (H(key) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第*i*次发生冲突”



处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(key) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第*i*次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(key)=1\%13=1$ $H_0=(1+d_0)\%16=1$ **冲突** → $H_1=(1+d_1)\%16=2$ 发生第1次冲突后重新计算得到的哈希地址

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$

	14	1				19				23					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$$H(\text{key}) = 1 \% 13 = 1$$

$$H_0 = (1 + d_0) \% 16 = 1$$

冲突

$$H_1 = (1 + d_1) \% 16 = 2$$

发生第1次冲突后重新
计算得到的哈希地址

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$

	14	1	68			19	20			23					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

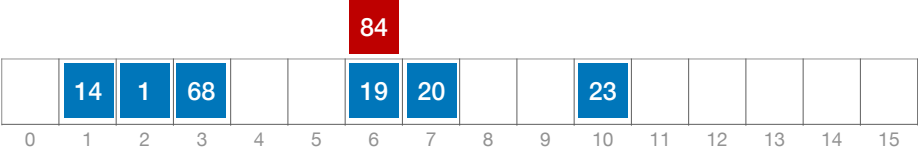
$$H(\text{key}) = 68 \% 13 = 3$$

$$20 \% 13 = 7$$

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

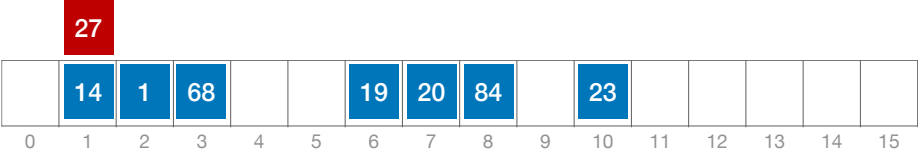
$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第*i*次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 84 \% 13 = 6$ $H_0 = (6 + 0) \% 16 = 6$ 冲突 $H_1 = (6 + 1) \% 16 = 7$ 冲突 $H_2 = 8$

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

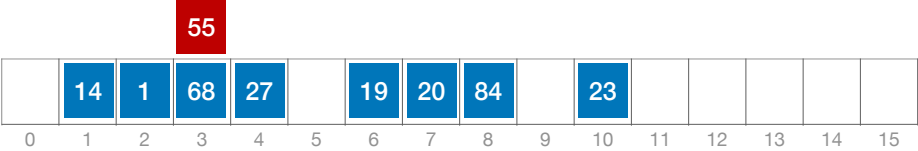
$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第*i*次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 27 \% 13 = 1$ 冲突 $H_1 = 2$ 冲突 $H_2 = 3$ 冲突 $H_3 = 4$

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(key) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第*i*次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(key)=55\%13=3$ **冲突** $H_1=4$ **冲突** $H_2=5$

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(key)=key\%13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(key) + d_i) \% m$$

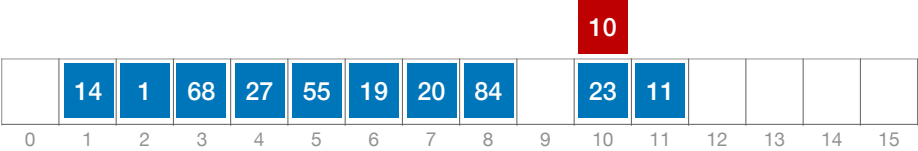
$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第*i*次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(key)=11\%13=11$

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

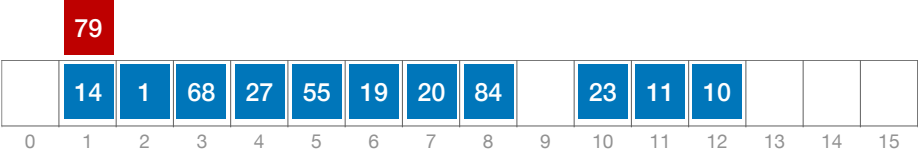
$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 10 \% 13 = 10$ 冲突 $\rightarrow H_1 = 11$ 冲突 $\rightarrow H_2 = 12$

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 79 \% 13 = 1$ 冲突 $\rightarrow H_1 = 2$ 冲突 $\rightarrow H_2 = 3$... 冲突 $\rightarrow H_8 = 9$

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$

												25			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空



$$H(25) = 25 \% 13 = 12$$

$$H_1 = (H(\text{key}) + 1) \% 16 = 13$$

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$

	14	1	68	27	55	19	20	84	79	23	11	10	25		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空



$$H(25) = 25 \% 13 = 12$$

$$H_1 = (H(\text{key}) + 1) \% 16 = 13$$

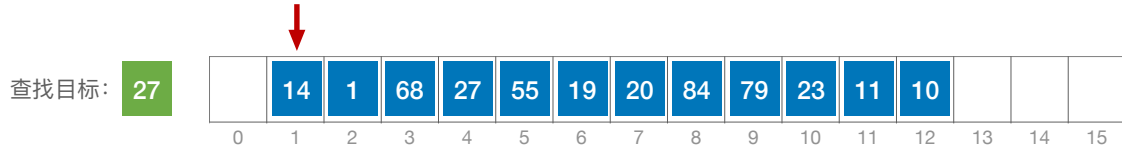
哈希函数值域[0,12]

冲突处理函数值域[0,15]

王道考研/CSKAOYAN.COM

查找操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 27 \% 13 = 1$ **冲突** $H_1 = 2$ **冲突** $H_2 = 3$ **冲突** $H_3 = 4$ **27的查找长度=4**

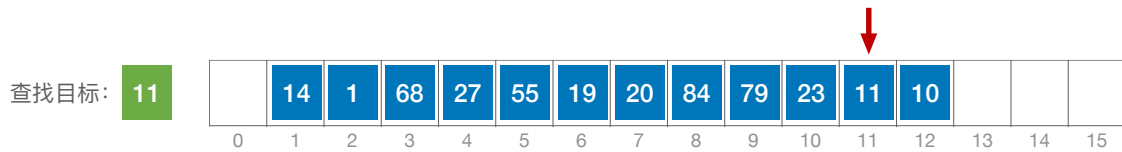
同义词、非同义词都需要被检查



王道考研/CSKAOYAN.COM

查找操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

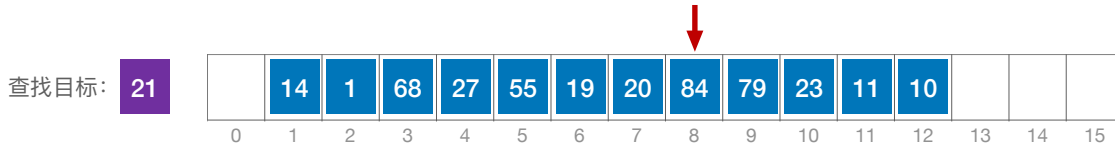
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 11 \% 13 = 11$ **11的查找长度=1**

王道考研/CSKAOYAN.COM

查找操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

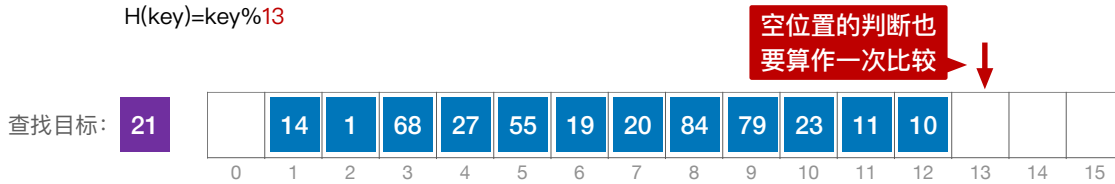
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 21 \% 13 = 8$ 冲突 → $H_1 = 9$ 冲突 → $H_2 = 10$ 冲突 → $H_3 = 11$ 冲突 → $H_4 = 12$ 冲突 → $H_5 = 13$
21的查找长度=6

王道考研/CSKAOYAN.COM

查找操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

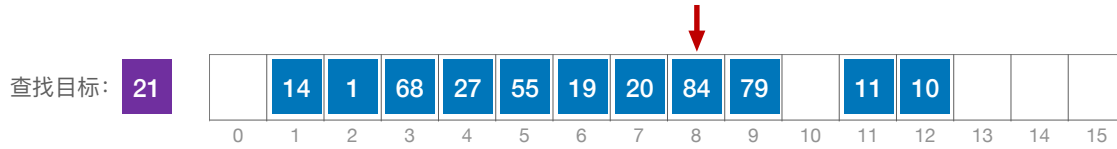
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 21 \% 13 = 8$ 冲突 → $H_1 = 9$ 冲突 → $H_2 = 10$ 冲突 → $H_3 = 11$ 冲突 → $H_4 = 12$ 冲突 → $H_5 = 13$
21的查找长度=6

王道考研/CSKAOYAN.COM

查找操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

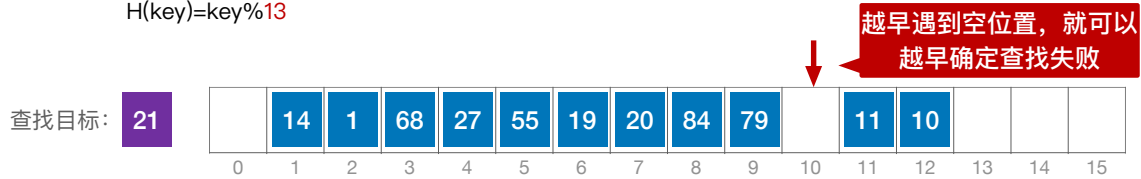
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$$H(\text{key}) = 21 \% 13 = 8 \xrightarrow{\text{冲突}} H_1 = 9 \xrightarrow{\text{冲突}} H_2 = 10$$

王道考研/CSKAOYAN.COM

查找操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

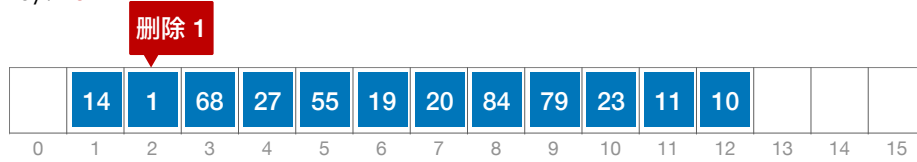
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$$H(\text{key}) = 21 \% 13 = 8 \xrightarrow{\text{冲突}} H_1 = 9 \xrightarrow{\text{冲突}} H_2 = 10 \quad \text{21的查找长度}=3$$

王道考研/CSKAOYAN.COM

删除操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

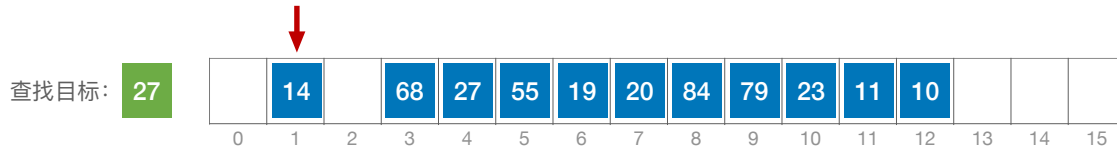
$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

王道考研/CSKAOYAN.COM

删除操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 27 \% 13 = 1$ **冲突** $\rightarrow H_1 = 2$

王道考研/CSKAOYAN.COM

删除操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

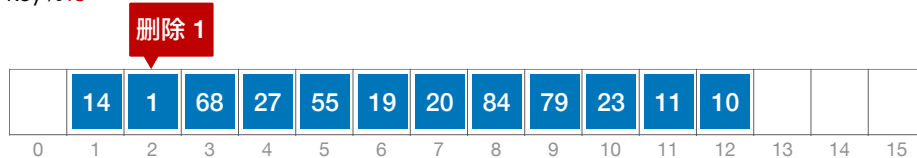
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$$H(\text{key}) = 27 \% 13 = 1 \quad \xrightarrow{\text{冲突}} \quad H_1 = 2$$

王道考研/CSKAOYAN.COM

删除操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

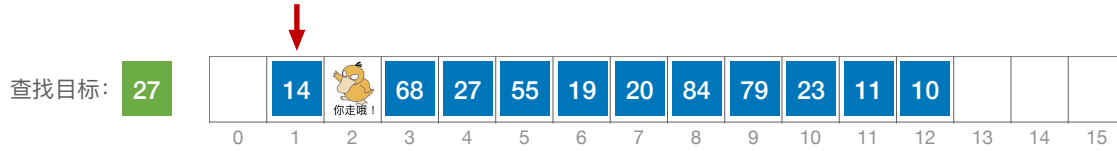
①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

注意：采用“开放定址法”时，删除结点不能简单地将被删结点的空间置为空，否则将截断在它之后填入散列表的同义词结点的查找路径，可以做一个“删除标记”，进行逻辑删除

王道考研/CSKAOYAN.COM

删除操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 27 \% 13 = 1$ **冲突** $\rightarrow H_1 = 2$ **冲突** $\rightarrow H_2 = 3$ **冲突** $\rightarrow H_3 = 4$ **27的查找长度=4**

王道考研/CSKAOYAN.COM

删除操作

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$



所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示**散列表表长**； d_i 为**增量序列**； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

$H(\text{key}) = 79 \% 13 = 1$ **冲突** $\rightarrow H_1 = 2$ **冲突** $\rightarrow H_2 = 3$... **冲突** $\rightarrow H_8 = 9$



王道考研/CSKAOYAN.COM

查找效率分析 (ASL)

例: 有一堆数据元素, 关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}, 散列函数 $H(\text{key}) = \text{key} \% 13$

	14	1	68	27	55	19	20	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**, 是指可存放新表项的空闲地址既向它的同义词表项开放, 又向它的非同义词表项开放。其数学递推公式为:

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$, m 表示散列表表长; d_i 为增量序列; i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$; 即发生冲突时, 每次往后探测相邻的下一个单元是否为空

19%13=6——1次 27%13=1——4次 55%13=3——3次
14%13=1——1次 68%13=3——1次 11%13=11——1次
23%13=10——1次 20%13=7——1次 10%13=10——3次
1%13=1——2次 84%13=6——3次 79%13=1——9次

$$ASL_{\text{成功}} = \frac{1 + 1 + 1 + 2 + 4 + 1 + 1 + 3 + 3 + 1 + 3 + 9}{12} = 2.5$$

王道考研/CSKAOYAN.COM

查找效率分析 (ASL)

例: 有一堆数据元素, 关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}, 散列函数 $H(\text{key}) = \text{key} \% 13$

	14	1	68	27	55	19	20	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**, 是指可存放新表项的空闲地址既向它的同义词表项开放, 又向它的非同义词表项开放。其数学递推公式为:

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$, m 表示散列表表长; d_i 为增量序列; i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$; 即发生冲突时, 每次往后探测相邻的下一个单元是否为空

$$ASL_{\text{失败}} = \frac{1 + 13 + 12 + 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2}{13} = 7$$

初次探测的地址 H_0 只可能在[0,12]

王道考研/CSKAOYAN.COM

查找效率分析 (ASL)

例：有一堆数据元素，关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}，散列函数 $H(\text{key}) = \text{key} \% 13$

	14	1	68	27	55	19	20	84	79	23	11	10			
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

①**线性探测法**—— $d_i = 0, 1, 2, 3, \dots, m-1$ ；即发生冲突时，每次往后探测相邻的下一个单元是否为空

线性探测法很容易造成同义词、非同义词的“**聚集（堆积）**”现象，严重影响查找效率

产生原因——冲突后再探测一定是放在某个连续的位置

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法

例：散列函数 $H(\text{key}) = \text{key} \% 13$ ，采用平方探测法处理冲突

							84	71																				
							58	45																				
							32	19																				
							6																					
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26		

开放定址法： $H_i = (H(\text{key}) + d_i) \% m$

$i = 0, 1, 2, \dots, k$ ($k \leq m - 1$)， m 表示散列表表长； d_i 为增量序列；

②**平方探测法**。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 时，称为平方探测法，又称**二次探测法**其中 $k \leq m/2$

$$d_0 = 0$$

$$d_1 = 1$$

$$d_2 = -1$$

$$d_3 = 4$$

$$d_4 = -4$$

$$d_5 = 9$$

$$d_6 = -9$$

....

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法



例：散列函数 $H(\text{key}) = \text{key} \% 13$ ，采用平方探测法处理冲突

		58			32	6	19			45				71										84		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

开放定址法： $H_i = (H(\text{key}) + d_i) \% m$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列；

②平方探测法。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 时，称为平方探测法，又称二次探测法其中 $k \leq m/2$

$$d_0 = 0$$

$$d_1 = 1$$

$$d_2 = -1$$

$$d_3 = 4$$

$$d_4 = -4$$

$$d_5 = 9$$

$$d_6 = -9$$

....

注意负数的模运算， $(-3) \% 27 = 24$ ，而不是3

《数论》中模运算的规则： $a \text{ MOD } m == (a + km) \text{ MOD } m$ ，其中， k 为任意整数

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法



例：散列函数 $H(\text{key}) = \text{key} \% 13$ ，采用平方探测法处理冲突

查找目标：71

		58			32	6	19			45				71										84		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26

开放定址法： $H_i = (H(\text{key}) + d_i) \% m$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列；

②平方探测法。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 时，称为平方探测法，又称二次探测法其中 $k \leq m/2$

$$d_0 = 0$$

$$d_1 = 1$$

$$d_2 = -1$$

$$d_3 = 4$$

$$d_4 = -4$$

$$d_5 = 9$$

$$d_6 = -9$$

....

平方探测法：比起线性探测法更不易产生“聚集（堆积）”问题

王道考研/CSKAOYAN.COM

处理冲突的方法——开放定址法

表长 $m=7$

若当前 $H(\text{key})=5$

经过7次探测可以探测到所有位置

0	1	2	3	4	5	6

$d_0 = 0 \quad H_0 = 5$
 $d_1 = 1 \quad H_1 = 6$
 $d_2 = -1 \quad H_2 = 4$
 $d_3 = 4 \quad H_3 = 2$
 $d_4 = -4 \quad H_4 = 1$
 $d_5 = 9 \quad H_5 = 0$
 $d_6 = -9 \quad H_6 = 3$

表长 $m=8$

经过8次探测不能探测到所有位置

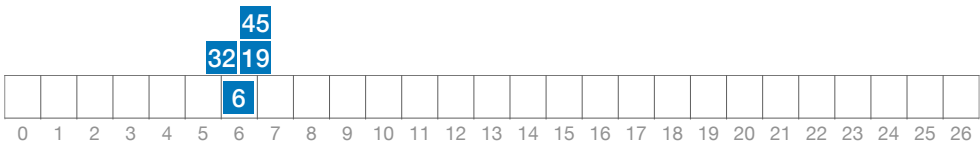
0	1	2	3	4	5	6	7

$d_0 = 0 \quad H_0 = 5$
 $d_1 = 1 \quad H_1 = 6$
 $d_2 = -1 \quad H_2 = 4$
 $d_3 = 4 \quad H_3 = 1$
 $d_4 = -4 \quad H_4 = 1$
 $d_5 = 9 \quad H_5 = 6$
 $d_6 = -9 \quad H_6 = 4$
 $d_7 = 16 \quad H_7 = 5$

开放定址法： $H_i = (H(\text{key}) + d_i) \% m$
 $i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列；
②平方探测法。当 $d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 时，称为平方探测法，又称二次探测法其中 $k \leq m/2$
非重点小坑：散列表长度 m 必须是一个可以表示成 $4j + 3$ 的素数，才能探测到所有位置

处理冲突的方法——开放定址法

例：散列函数 $H(\text{key}) = \text{key} \% 13$ ，采用平方探测法处理冲突



开放定址法： $H_i = (H(\text{key}) + d_i) \% m$
 $i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列；
③伪随机序列法。 d_i 是一个伪随机序列，如 $d_i = 0, 5, 24, 11, \dots$

处理冲突的方法——开放定址法

所谓**开放定址法**，是指可存放新表项的空闲地址既向它的同义词表项开放，又向它的非同义词表项开放。其数学递推公式为：

$$H_i = (H(\text{key}) + d_i) \% m$$

$i = 0, 1, 2, \dots, k \ (k \leq m - 1)$ ， m 表示散列表表长； d_i 为增量序列； i 可理解为“第 i 次发生冲突”

开放定址法	① 线性探测法	$d_i = 0, 1, 2, 3, \dots, m-1$
	② 平方探测法	$d_i = 0^2, 1^2, -1^2, 2^2, -2^2, \dots, k^2, -k^2$ 其中 $k \leq m/2$
	③ 伪随机序列法	$d_i = \text{某个伪随机序列}$

注意：采用“开放定址法”时，删除结点不能简单地将被删结点的空间置为空，否则将截断在它之后填入散列表的同义词结点的查找路径，可以做一个“删除标记”，进行逻辑删除

处理冲突的方法——再散列法

严蔚敏《数据结构》

再散列法（再哈希法）：除了原始的散列函数 $H(\text{key})$ 之外，多准备几个散列函数，当散列函数冲突时，用下一个散列函数计算一个新地址，直到不冲突为止：

$$H_i = RH_i(\text{Key}) \quad i=1,2,3,\dots,k$$

王道《数据结构》

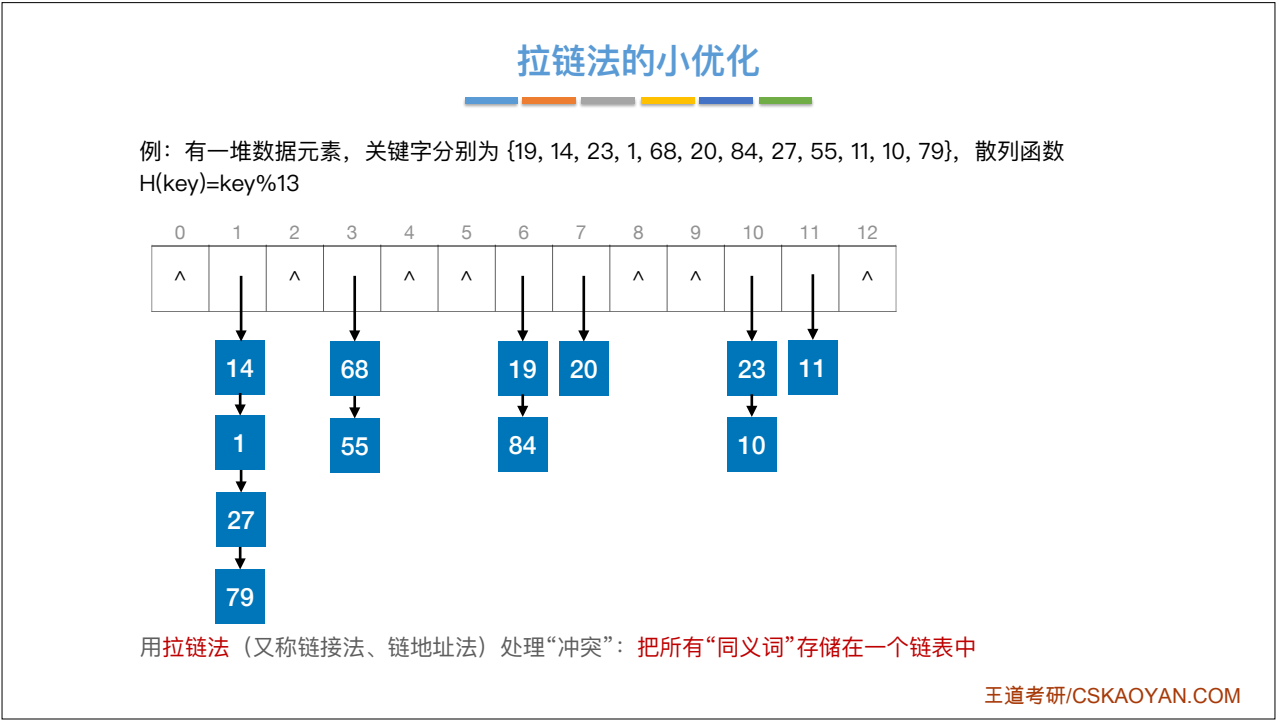
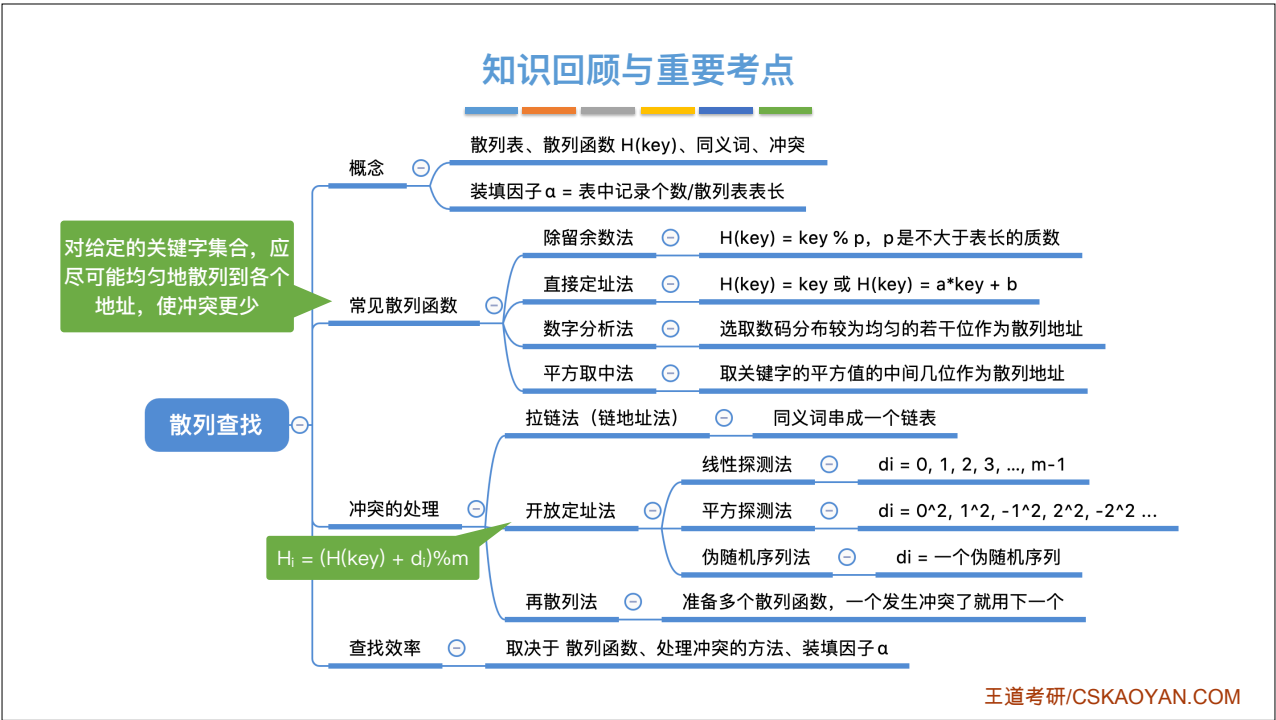
3) 再散列法。当 $d_i = \text{Hash}_2(\text{key})$ 时，称为再散列法，又称双散列法。需要使用两个散列函数，当通过第一个散列函数 $H(\text{key})$ 得到的地址发生冲突时，则利用第二个散列函数 $\text{Hash}_2(\text{key})$ 计算该关键字的地址增量。它的具体散列函数形式如下：↵

$$H_i = (H(\text{key}) + i \times \text{Hash}_2(\text{key})) \% m$$

初始探测位置 $H_0 = H(\text{key}) \% m$ 。 i 是冲突的次数，初始为 0。在散列法中，最多经过 $m - 1$ 次探测就会遍历表中所有位置，回到 H_0 位置。↵

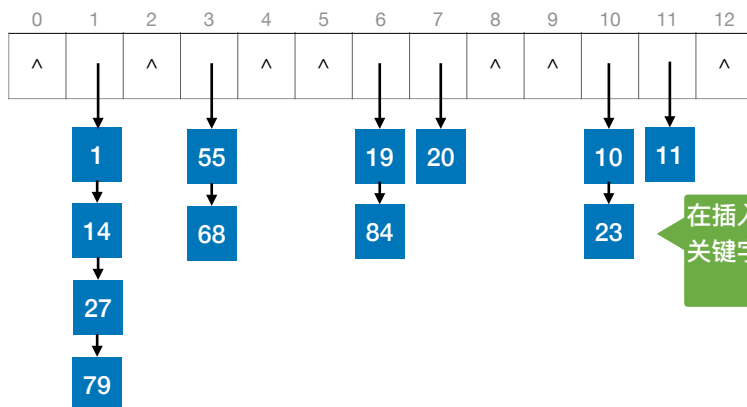


疑惑 却不说



拉链法的小优化

例: 有一堆数据元素, 关键字分别为 {19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79}, 散列函数 $H(\text{key}) = \text{key} \% 13$



在插入新元素时, 保持关键字有序, 可微微提高查找效率

用**拉链法** (又称链接法、链地址法) 处理“冲突”: 把所有“同义词”存储在一个链表中